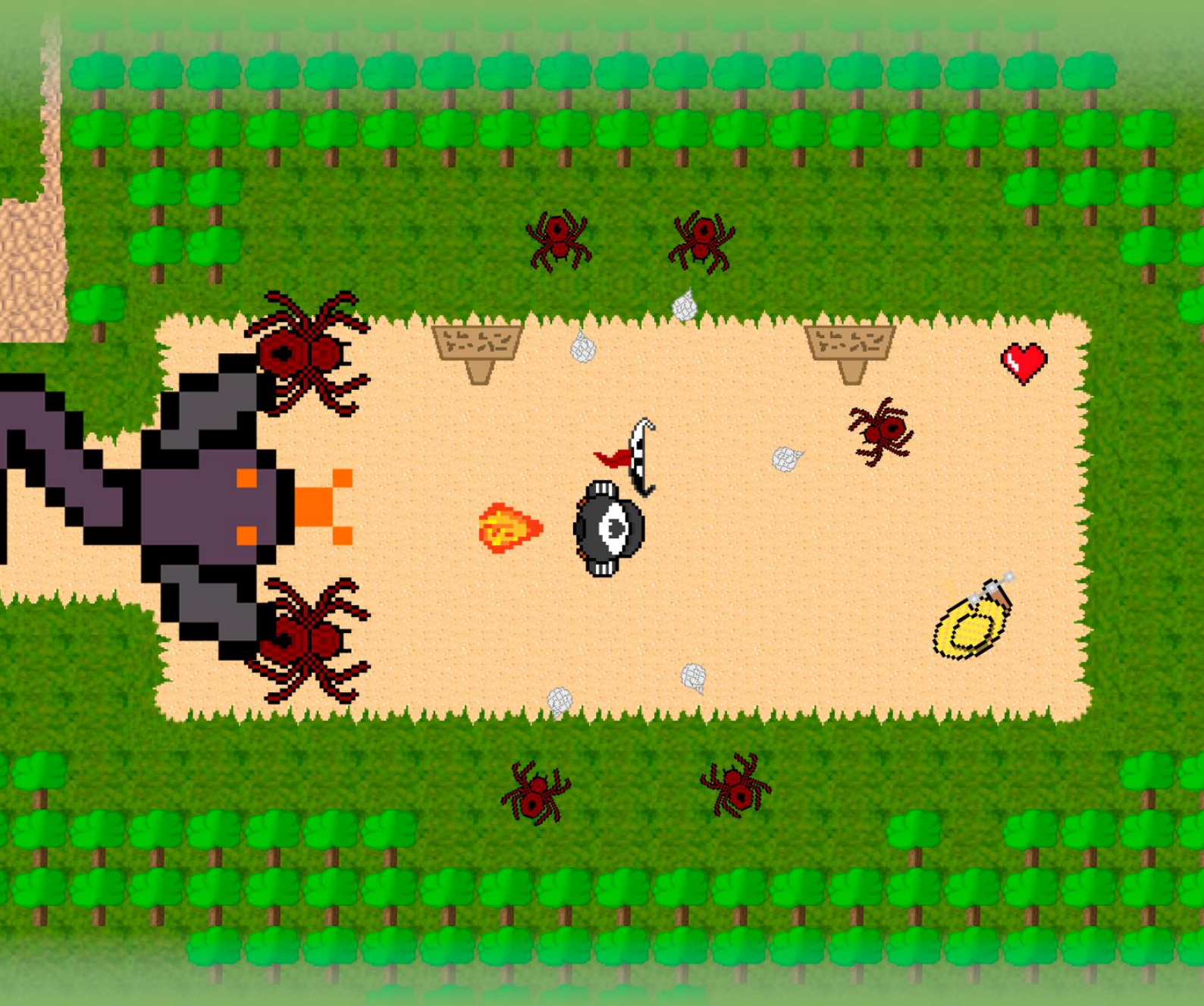


**TEKNIKFAG**  
**DDU2**



# **EKSAMENSPROJEKT**

## **SPIILUDVIKLING**



**MEDLEMMER:**  
**GUSTAV A. L. KENSØ**  
**RASMUS A. CHRISTIANSEN**  
**SIMON SAXILD DANIELSEN**

|                                              |    |
|----------------------------------------------|----|
| <b>Indledning</b>                            | 4  |
| <b>Produktbeskrivelse</b>                    | 5  |
| <b>Kravspecifikation</b>                     | 6  |
| MoSCoW-modellen                              | 6  |
| <b>Design - Uddybning af krav</b>            | 7  |
| Skitser                                      | 9  |
| <b>Planlægning</b>                           | 11 |
| <b>Scrum</b>                                 | 11 |
| Ansvarsområder                               | 11 |
| Sprints                                      | 12 |
| Gantt-kort                                   | 12 |
| Trello                                       | 13 |
| Programmeringssprog og udviklingsmiljø       | 14 |
| <b>Produktudvikling</b>                      | 15 |
| <b>Grafiske elementer</b>                    | 15 |
| Monstre, spiller, fjender, spells og diverse | 15 |
| Oprettelse af sprites og spritesheet         | 16 |
| Spritesheet - Baneopbygning                  | 17 |
| <b>Programmeringselementer</b>               | 18 |
| Unity-elementer                              | 18 |
| Singleplayer                                 | 21 |
| Player & monster valg                        | 21 |
| Player og monster instans                    | 23 |
| Player                                       | 25 |
| Player movement & animation                  | 29 |
| Spells                                       | 34 |
| Enemies                                      | 40 |
| Quest                                        | 44 |
| Multiplayer                                  | 47 |
| Hvad er Photon PUN?                          | 47 |
| Hvorfor valgte vi dette?                     | 47 |
| Start eller tilslut et online session-room   | 47 |
| Styring af spillerens bevægelser             | 52 |
| Var PUN det rigtige valg?                    | 53 |
| <b>Produktfremvisning</b>                    | 55 |
| Game hub                                     | 55 |
| Multiplayer                                  | 56 |
| Singleplayer                                 | 57 |

|                                                               |    |
|---------------------------------------------------------------|----|
| <b>Spilanalyse, med MDA framework og Bartles spillertyper</b> | 61 |
| <b>Tænke-højt test &amp; spørgeskema</b>                      | 64 |
| Resultater på testen                                          | 66 |
| Deltest 1 & 2                                                 | 66 |
| Deltest 3                                                     | 66 |
| Vigtige observationer                                         | 67 |
| Hvordan arbejder vi videre herfra?                            | 68 |
| <b>Projektevaluering</b>                                      | 69 |
| <b>Konklusion</b>                                             | 70 |
| <b>Litteraturliste</b>                                        | 71 |
| <b>Bilag</b>                                                  | 73 |

# Indledning

Vi har alle prøvet at spille et videospil. Om det er telefonspil, computerspil eller konsolspil er ligegyldigt, fordi de har allesammen nogle centrale egenskaber til fælles. Spil er med til at bringe familie og venner tættere sammen. Samtidig giver det mulighed for at møde nye mennesker og skabe nye relationer på tværs af landegrænser og kontinenter. Udover de sociale muligheder, så giver spil også kognitive fordele, da man som spiller i situationer skal tænke ud af boksen. Vi vil i vores projekt udvikle et fuld funktionelt spil med et klart og tydeligt mål. Undervejs i rapporten vil vi både komme ind på vores projektstyring og produktudvikling samt test og evaluering af spillet. I projektstyringen vil vi lægge vægt på de valg af metoder vi har valgt, samt vores planlægningsdokumenter. En stor del af vores planlægningsselementer er essentielle at forstå for at sætte sig ind i vores udviklingsprocess. Vi vil specielt komme ind på vores brug af Scrum. I produktudviklingsprocessen vil vi gennemgå og forklare de mest essentielle scripts og kode elementer for vores endelige produkt. Da vi har et godt stykke over 2000 linjer kode samlet set, vil vi kun gennemgå de mest centrale dele af vores kode. Vi vil også forklare vigtige elementer fra vores udviklingsmiljø "Unity 2D". Dette gør vi for at give bedre indblik i hvordan det er at programmere spil i Unity. Til at evaluere på vores process og produkt vil vi foretage en bred og fyldestgørende test af det endelige spil som produkt. Denne test skal være grundlag for feedback til fremtidig videreudvikling af produktet. For yderligere at evaluere på hvorvidt vi opnåede de opsatte krav og elementer for vores spil, vil vi foretage en dybdegående spilanalyse. Denne analyse vil til dels evaluere på spilelementer og målgruppe.



# Produktbeskrivelse

Målgruppen for vores spil har ingen speciel aldersgruppe. Vores målgruppe kommer dog til at være baseret på spillertyperne; "Achiever" og "Explorer". Man kan til dels også argumentere for "killers". Vi vil gerne opnå et produkt der kan appellere til disse tre spillertyper. Genremæssigt, kommer vi mest til at appellere til fantasy-genren. Så personer der godt kan lide fantasy som samtidigt falder under spillertyperne "Achiever" og "Explorer", vil være den primære målgruppe for vores spil.

I vores spil, kan man vælge enten at spille singleplayer eller multiplayer. Herefter bliver man placeret ind i en verden fyldt med monstre. Inden da, så får man dog mulighed for at vælge sin egen karakter og sit eget monster. Herefter går det ud på at man dræber andre monstre og, sideløbende løse missioner og quests givet af NPC'er, som kan sætte dig ind i landet situation og give dig objektet for din quest.

Spillet vil blive lavet i RPG-genren. Vores spil vil høre til RPG-genren, da man skal kunne skabe sin egen karakter og indsætte den i den virtuelle fantasy verden spillet foregår i, hvor man vil spille sin rolle som en person der går rundt og hjælper verdens beboere(NPC's).

Spilleren vælger et monster som skal følge og hjælpe spilleren

Kontrolleringen af spilleren udfoldes af en WASD og right, left, movement mekanik. Man kommer ikke til at kunne hoppe i dette spil, da det er et top down spil. Vi ser altså spilleren oppefra i tredjepersonsvinkel. Man kommer til at kunne select og kaste med magi på knapperne "1", "2", "3", "4" og "space".

Pocket-Monster kan være en ud af 4 forskellige typer, (psychic, poison, ghost, electric), hvor hver type har deres egen type af magiske spells. Monstrene kan kun udføre spells af samme type som de selv er. det vil sige at Playerens monster definere hvilke typer spells og angreb, playeren kan lave. Monstrene er også det der leverer den magiske energi til playeren som så kan bruges til at kaste spells, så på den måde er Pocket-monstrene omvarende magiske batterier. Yderligere, jo højere level Pocket-monsteret er, jo større magisk kapacitet har det og jo større og stærkere spells kan playeren kaste.

# Kravspekifikation

I dette stykke vil vi komme ind på vores opsætning af alle de krav vi har til vores endelige produkt/spil. Til dette vil vi bruge Moscow modellen, da den også fint lægger op til scrum-udviklingsmetoden, som vi også har tænkt os at gøre brug af. Disse krav har ændret sig fra dem der først fremgik i vores allerførste kravspekifikation.

## MoSCoW-modellen

Moscow modellen herunder, er primært til egen udvikling, men kan benyttes til videre udvikling af andre, i en fremtidig udviklingsindsats.

### **Must have:**

- En spiller
- Bevægelse til spiller
- Spilleren har et simpelt kampsystem
- En fjende
- Simpel bane
- Multiplayer

### **Should have:**

- Basic attack
- Avanceret kampsystem med elementbaserede angreb.
- Avancerede fjender.
- Noget Selvproduceret grafik.
- Pænt GUI, med healthbar og spells.
- Boss fight
- Questsystem

### **Could have:**

- Baggrundsmusik og andre lydeffekter.
- Alt Grafik Selvproduceret
- Typerne skal have større eller mindre effekt mod hinanden
- Forskellige biomes der indretter sig efter monster typerne
- Skill-tree

### **Will not have:**

- En live chat
- Level system med XP

# Design - Uddybning af krav

## Must have

**En spiller:** En grafisk figur hvor kameraet er centreret på spillerens position.

**Bevægelse til spiller:** Spilleren skal kunne bevæge sig op, ned til højre og til venstre. Spilleren skal ses oppefra, da spillet er i TopDown-format. Derudover skal man også kunne gå skråt til alle sider, hvis man holder to knapper inde på samme tid. Spilleren skal kunne rotere ved at benytte venstre og højre piletaster. Spilleren skal ikke kunne hoppe, da spillet bliver set oppefra.

**Spilleren har et simpelt kampsystem:** Spilleren skal kunne kaste et spell, som bliver sendt afsted foran spilleren. Spillet skal have en bestemt mængde skade. Hvis et spell rammer en væg, fjende eller lignende, så skal spell projektilet fjernes.

**En fjende:** En fjende der kan ses grafisk og kan skade spilleren ved at sende skade via et projektil som der kolliderer med spilleren. Projektilets retning skal beregnes ved at tage spillerens nuværende position, og bruge de koordinater til en vektor.

**Simpel bane:** En grafisk bane med lånt grafik. Den skal indeholde få kollisioner med sten eller lignende. Banen skal grafisk set være i TopDown-format.

**Multiplayer:** To spillere skal kunne tilslutte sig samme bane og kunne se hinanden bevæge sig rundt på denne bane. Metoden til multiplayer er ligegyldig, så længe at det består af en dedikeret server, som ikke er afhængig af én spiller.

## Should have

**Basic attack:** Et angreb der grafisk kan ses flyve fra spillerens position, mod den retning som spilleren vender. Projektilet skal instantiates som et prefab med et Collider 2D komponent, således at det kan forsvinde hvis det rammer en kant, fjende eller lignende.

**Avanceret kampsystem med elementbaserede angreb:** Et kampsystem med flere forskellige projektiler der har forskellige skade-variabler. Disse projektiler skal laves ud fra forskellige naturlige elementer. Der skal laves fire projektiler; (**Fireball**: 10 damage: 1 sec cooldown, **Poison Dart**: 5 damage: 0,2 sec cooldown, **Electric Bolt**: Stun target 5 sec: cooldown 15 sec, **Puke Valley**: 40 damage: 3 sec cooldown: Note slow projectile and short range).

**Avancerede fjender:** Fjenderne skal kunne følge efter spilleren når de kommer ind for en bestemt afstand. Når spilleren kommer ind for en bestemt afstand, skal fjenderne begynde at skyde projektiler mod spilleren. Derudover skal fjenderne begynde at trække sig og skyde samtidigt når spilleren kommer for tæt på.

**Noget Selvproduceret grafik:** Noget af den lånte grafik skal erstattes af selvproduceret grafik. Første prioritet vil være at lave eget sprite-sheet. Dette skal indeholde, græs, grus til sti, vand, bjergsider og en slags hegn.

**Pæn GUI, med healthbar og spells:** En pæn GUI hvor man kan se spillerens liv i form af en progressiv healthbar. Derudover skal der også være et sted på skærmen, hvor det vil være muligt for spilleren at se hvilket spell der er markeret.

**Questsystem:** Der skal oprettes endnu en bane separat fra den man starter på. Denne bane skal bygges op så den er fyldt med forhindringer i form af fjender og et tydeligt mål. Derudover skal der indsættes en "NPC"(Non-Player Character), der skal give og sende spilleren afsted på sin quest. Questen skal gives via et pop-up vindue inde i spillet, når man kommer tæt på NPC'en.

**Boss fight:** Til sidst i questbanen skal der opstilles et lukket rum med en meget svær fjende, som spilleren skal kæmpe imod for at gennemføre questen.

### Could have

**Baggrundsmusik og andre lydeffekter:** Baggrundsmusik bestående af rolig royalty free musik. Lydeffekter når spilleren eller en fjende tager skade, for bedre indikation.

**Alt Grafik Selvproduceret:** Alt grafik selvproduceret; (Sprite-sheet, spiller, animationer, fjender, Healthbar, Actionbar til spells, spell-projektiler).

**Typerne skal have større eller mindre effekt mod hinanden:** Elementerne fra spellsne skal have større eller mindre skade, alt efter hvilken type fjender de rammer.

**Forskellige biomes der indretter sig efter monster typerne:** Forskellige grafiske naturområder der passer til fjendernes type.

**Skill-tree:** En menu hvor man vil kunne udvikle på sine spells og eventuelt vælge nye.

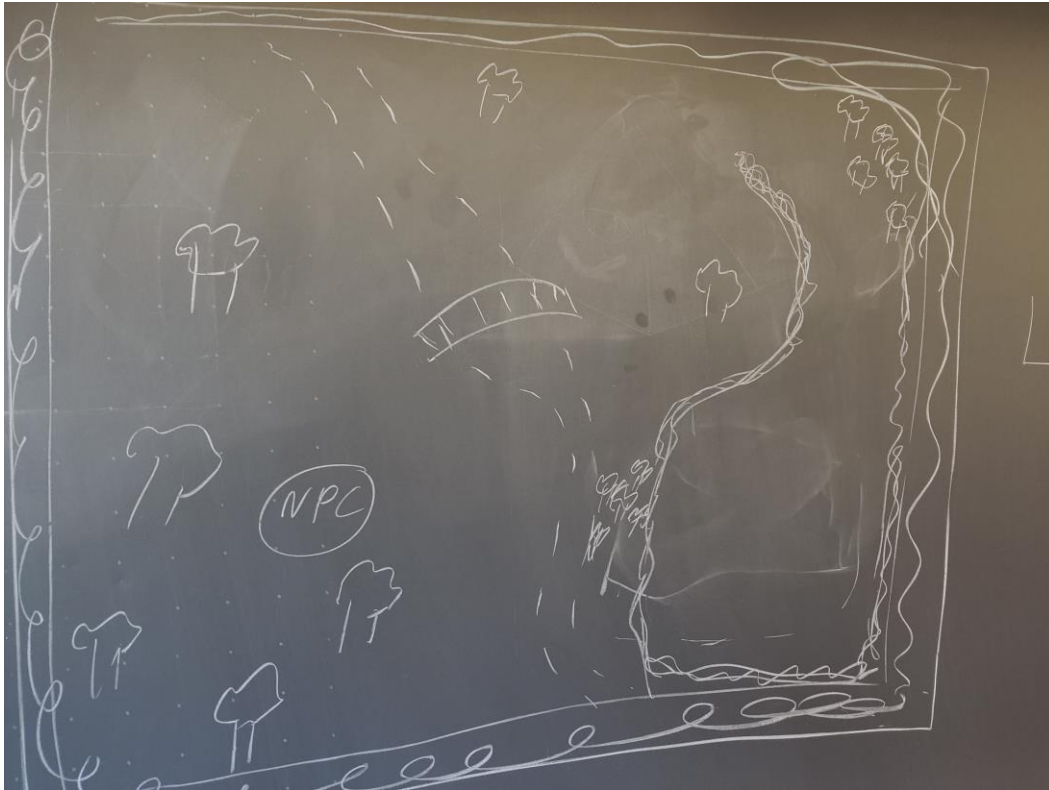
### Will not have

**En live chat:** En chat der skal placeres nede i venstre hjørne af skærmen. I dette chatvindue, skal man kunne kommunikere med andre spillere der spiller på samme server og er i samme bane/scene.

**Level system med XP:** Et system hvor man efter en hvis mængde XP-point kan level op. Hver gang en spillers level stiger vil spillerens skade og liv stige. Derudover skal spilleren optjene flere point i sit skill-tree. Mængden af XP til level to vil være 100. Mængden af XP til de næste par mange levels, kan beskrives med vækstfunktionen;  $f(x) = 100 * 1,15^x$ . XP-point vil optjenes via quests og blot at dræbe fjender. Jo stærkere fjender, desto mere XP.



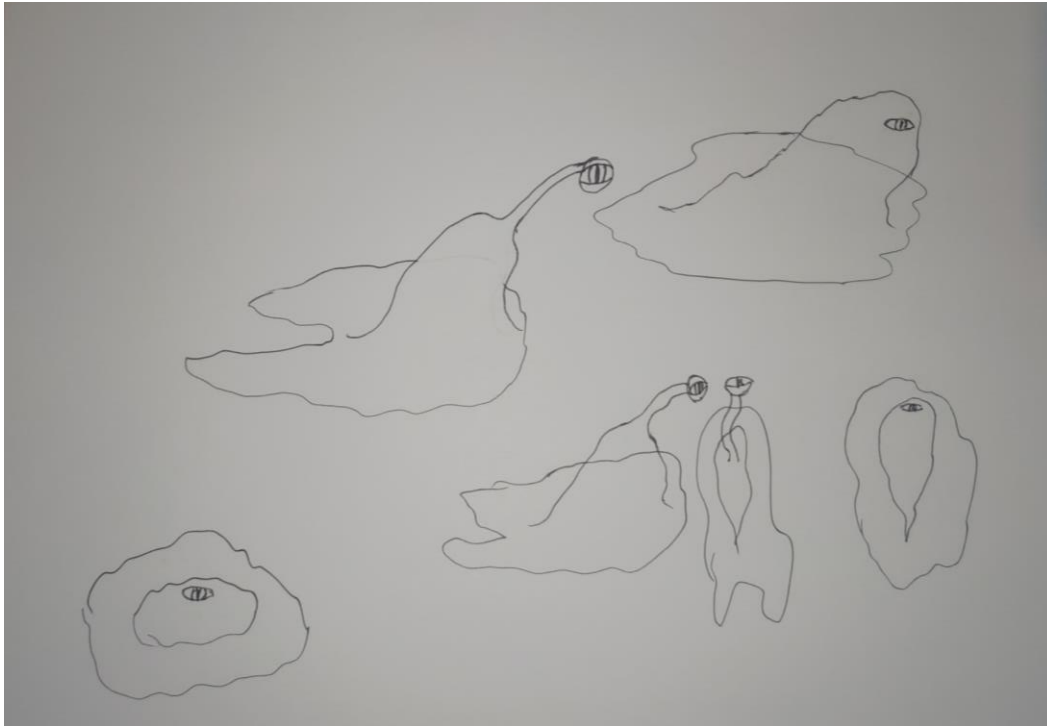
# Skitser



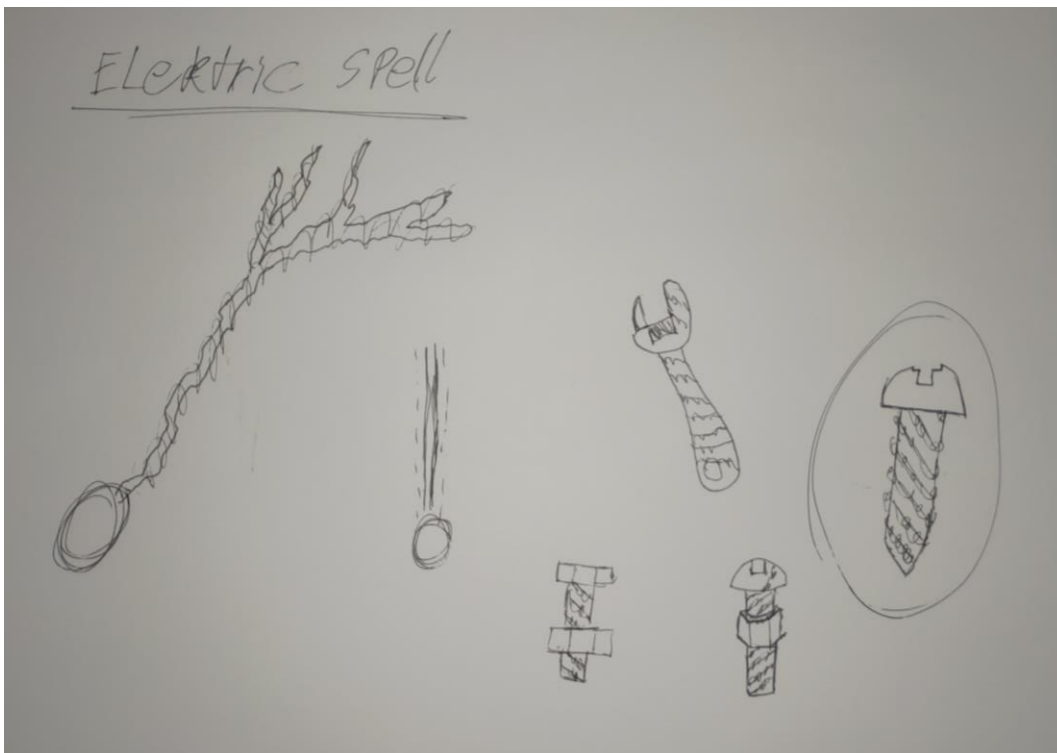
Skitse af questbanen.



Første skitse af Boss Slangen



De første skitser af hvad der senere er blevet til Poison monsteret.



De første skitser af hvad der senere er blevet til Elektric stun spell.

# Planlægning

I dette stykke vil vi beskrive lidt omkring den del af arbejdsprocessen der fandt sted inden selve produktudviklingen. Det er en stor fordel at have lagt en struktureret plan inden man skal i gang med at arbejde på at udvikle store spil. Der er mange elementer der skal laves og holdes styr på. På grund af dette har vi for eksempel valgt at projektstyre vores gruppearbejde med scrum.

## Scrum

Scrum er en meget agil og oplagt udviklingsmetode for et hold af spiludviklere. Metoden gør det nemt og overskueligt at prioritere sine arbejdsopgaver og planlægge hvornår hvilke arbejdsopgaver skal laves og være færdige. Vi har også lagt et stort fokus på at inddele vores opgaver i ansvarsområder.

## Ansvarsområder

| Roller                        | Gustav             | Rasmus             | Simon              |
|-------------------------------|--------------------|--------------------|--------------------|
| Multiplayer                   | Udfører hovedfokus | Udfører delvis     | Udfører ikke       |
| 2D Grafik                     | Udfører ikke       | Udfører delvis     | Udfører hovedfokus |
| Software leader               | Udfører ikke       | Udfører delvis     | Udfører hovedfokus |
| Software / scripts            | Udfører delvis     | Udfører hovedfokus | Udfører ikke       |
| Quest text og skilte          | Udfører hovedfokus | Udfører delvis     | Udfører ikke       |
| Scrummeeting ordstyrer        | Udfører ikke       | Udfører delvis     | Udfører hovedfokus |
| Project manager/product owner | Udfører delvis     | Udfører hovedfokus | Udfører ikke       |
| Audio effects                 | Udfører hovedfokus | Udfører delvis     | Udfører ikke       |

|                |
|----------------|
| Udfører ikke   |
| Udfører delvis |
| Udfører        |
| hovedfokus     |

**Multiplayer:** Programmering og implementering af multiplayer i spillet.

**2D Grafik:** Da vi i sidste ende håber på at lave en stor del af vores grafik selv, på baggrund af en erfaren gruppemedlem, så har vi valgt at tage det med som et vigtigt ansvarsområde.

**Software leader:** Skal sørge for at der er styr på programmeringen i en større sammenhæng. Det hele skal holdes overskueligt.

**Software/scripts:** Programmering af alle spillets scripts, altså byggestenene for programmet.

**Quest text og skilte:** Skal nedskrive kreative og informative tekststykker til skilte og quests.

**Scrum Meeting ordstyrer:** Skal i starten af hvert sprint og kort i starten af hver arbejdsdag afholdes et kort oplæg/møde med hele scrum teamet.

**Project manager/product owner:** Skal holde styr på projektet som en helhed.

**Audio effects:** Vi vil selv indtale et par lydeffekter der skal tydeliggøre interaktion i spillet.

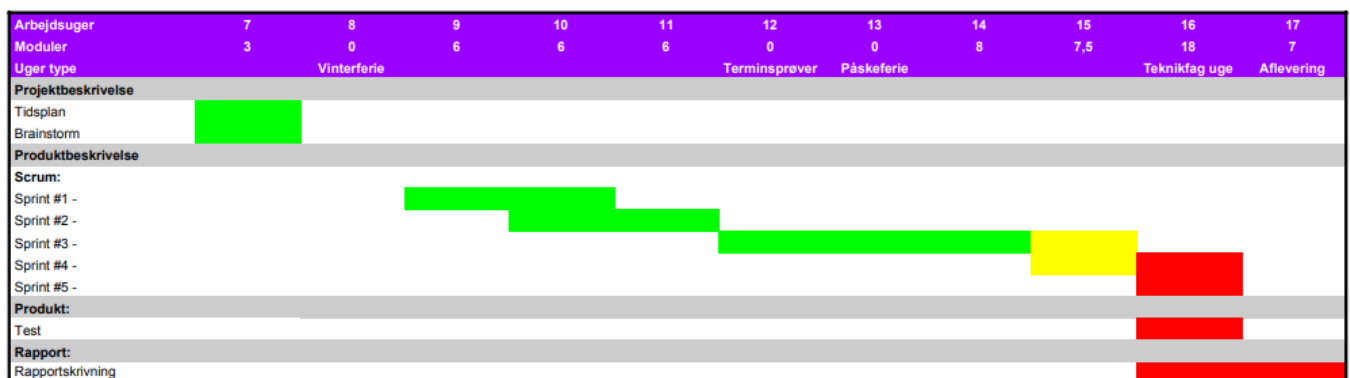
## Sprints

| Sprint 1                                                                      | Sprint 2                                                | Sprint 3                                                | Sprint 4                            | Sprint 5                    |
|-------------------------------------------------------------------------------|---------------------------------------------------------|---------------------------------------------------------|-------------------------------------|-----------------------------|
| Player<br>Player movement<br>Sprite sheet til<br>tilemap<br>Tilemap-opsætning | Monster til spiller<br>Collisions<br>Shooting<br>Fjende | Start menu<br>Pauseskærm<br>Multiplayer-<br>integration | Forskellige<br>bullets<br>Healthbar | Quest 1 NPC<br>Quest 1 bane |

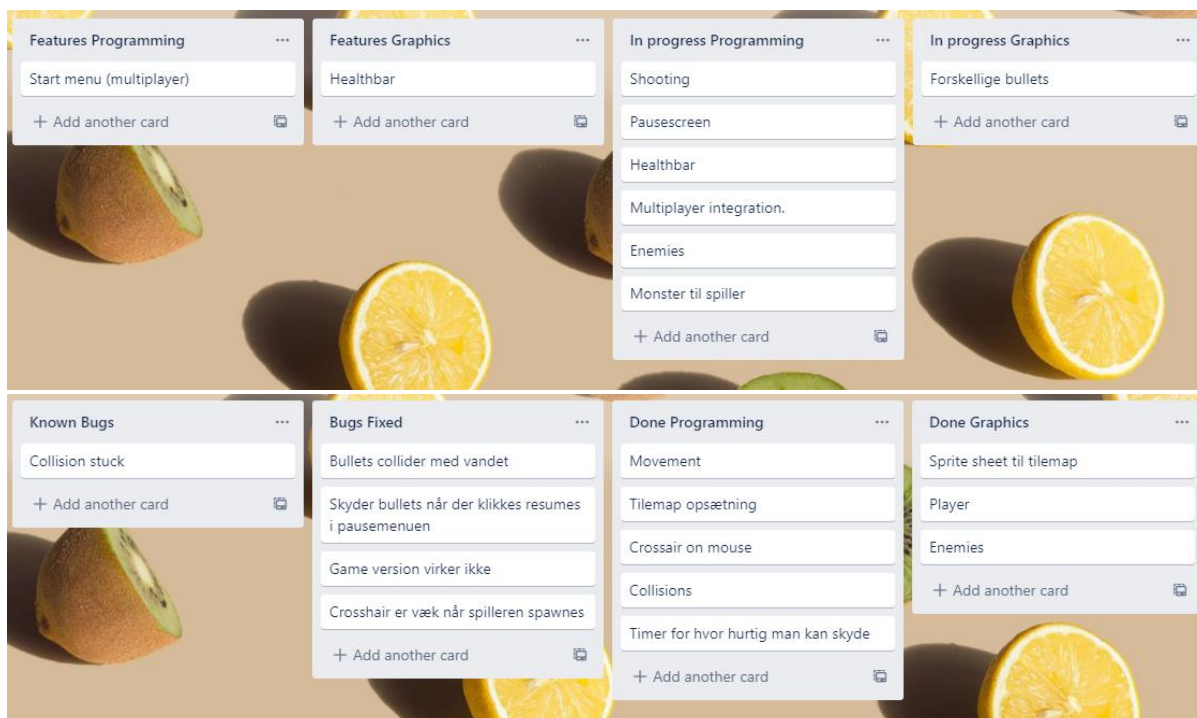
Vi har uddelt vores arbejdsopgaver i sprints således at vi har et overblik over hvad vi skal lave på et bestemt tidspunkt. Vi har prioriteret kravene efter vores MoSCoW-model. Det betyder at vores must have's ligger i de første sprints, da de er essentielle for spillet at have med. Herefter kommer vores should have's, da det er ting som vi også meget gerne skulle have med. Til sidst har vi vores could have's. De er lagt til sidst, da det ikke gør noget for spillets hovedfokus. Grunden til at multiplayer ikke er en af de første dele, er fordi vi regner med at skulle bruge rigtig meget til på at få dette implementeret i spillet.

## Gantt-kort

Herunder kan man se vores gantt-kort, som vi har brugt i løbet af processen til at planlægge vores arbejdsopgaver. Vi har lagt vores arbejdsuger ud på x-aksen og vores planlagte sprints ned ad y-aksen. Arbejdsopgaverne er inddelt i sprints, som selvfølgelig er sat i prioriteret rækkefølge ud fra vores kravspecifikation. Derudover har vi også oprettet to punkter der hedder test og rapportskrivning. Dette er igen blot for at få noget tidsperspektiv på hvornår i processen det ville være klogt at foretage sig disse elementer.



# Trello



Igennem hele vores produktudvikling har vi brugt Trello til at holde styr på alle de funktioniteter der skulle implementeres fra vores kravspecifikation. Udover at man kan se opgaverne, så kan man også opdatere deres status som enten "Done" eller "in progress". Dette er super godt når man er flere mennesker om at programmere et spil. Man kan også via Trello holde styr på nogle af de bugs og fejl man finder og skal have rettet i løbet af udviklingsprocessen.

På ovenstående billede kan man se et screenshot af vores trello, på et tidspunkt i programmeringsprocessen.

Ved brug af Trello, sørger vi hele tiden for at vi holder os til vores kravspecifikation når vi skal arbejde videre på spillet som produkt. Derudover hjælper det os med ikke at komme i karambolage med hinandens programmering der eventuelt kunne opstå grundet dårlig kommunikation. Trello hjælper også blot med at danne et overblik over hele udviklingsprocessen.



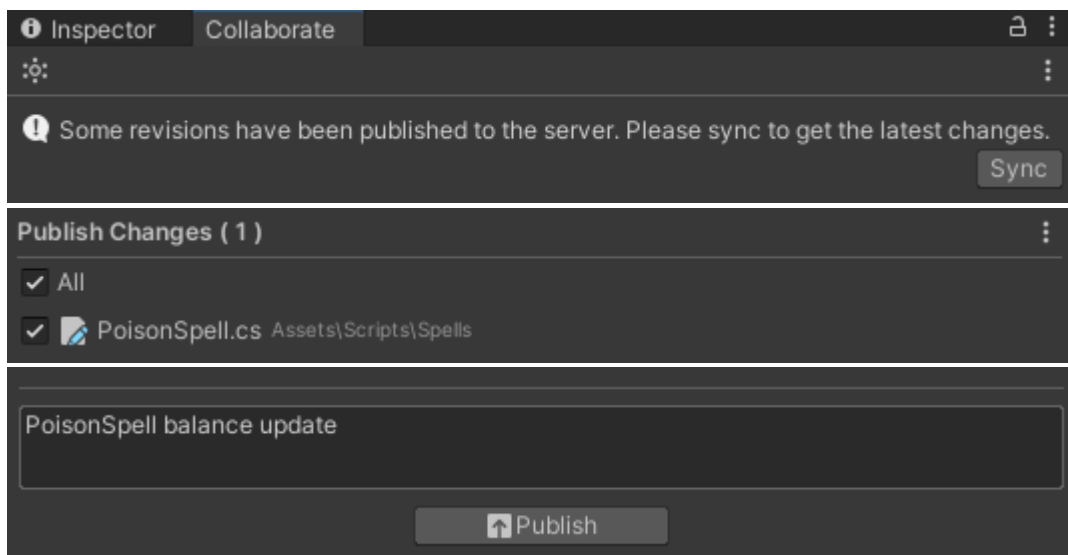
# Programmeringssprog og udviklingsmiljø

Til udvikling af vores spil har vi tænkt os at bruge Unity 2D. Unity er en game-engine, der er til for at gøre det lettere og mere overskueligt at arbejde med og udvikle spil. Derudover gør Unity det også nemmere at arbejde som gruppe på et større samlet projekt.

Unity udbyder to forskellige engines. Unity 2D og Unity 3D. I vores projekt vil vi holde os til at benytte Unity 2D, da vi blot vil fremstille et todimensionelt spil set oppefra, altså et "TopDown-spil". I Unity 2D er der mulighed for nemt at gøre brug af deres mange indbyggede libraries og funktioner. Unity 2D gør det for eksempel nemt at implementere collisions og game-physics. Derudover gør Unity 2D det også nemt at implementere grafik i flere lag og avancerede kollisioner med bane og spilmiljø.

Når man arbejder med Unity, foregår det som regel i C#-sproget. Dette er et oplagt sprog at bruge til spiludvikling, da det er et objektorienteret sprog. Alting i Unity er centreret omkring gameobjects med deres tilhørende funktioner, der eksekveres i forskellige situationer. C#-sproget er også bygget op, så det er nemt at dele vigtige variabler mellem scripts. Dette gør det nemt at få flere gameobject til at interagere med hinanden.

Unity har en funktion der gør det nemt at arbejde på spiludvikling som gruppe. Denne funktion hedder Unity collab. Denne funktion gør det muligt at flere personer på samme tid kan sidde og arbejde på hver deres script. Når man så er færdig uploader man det man har lavet til skyen, hvor ens gruppemedlemmer så kan synkronisere med ens ændringer. Det er dog meget vigtigt at man ikke er to der arbejder i samme script på samme tid. Dette kan skabe problemer hvor man mister fremskridt ved at synkronisere. Denne Collab-funktion er indbygget i Unity, hvis man opretter projektet som et projekt under ens Unity-team. Alle gruppemedlemmer skal så være en del af dette gruppeprojekt.



# Produktudvikling

## Grafiske elementer

### Monstre, spiller, fjender, spells og diverse

**Monster.** I udvikling af monstrene besluttede vi os først for at have fire forskellige typer af monstre. De fire typer vi endte med at beslutte os for var; poison, psychic, ghost og electric. Så da vi skulle begynde på designprocessen af disse monstre var det den idé vi arbejdede ud fra. Monstrene blev først designet i et 64x64 pixel fra front view. Dette gjorde det nemmere at skabe et billede af hvordan vi gerne ville have det til at fremstå. Disse 64x64 frontview tegninger er også de billeder der er brugt når brugeren skal vælge et monster i menuen. Hvert monster blev individuelt designet med henblik på at få dem til at passe ind i deres forskellige typer så brugeren ikke bør være i tvivl om hvilke monstre der er hvad.



**Top view:**



**Player:** playeren blev som top view'et af monstrene, samt alle andre grafiske elementer, tegnet i en 32x32 firkant i "paint.net". Vi skulle have en måde at bestemme playerens retning på så vi lavede en hat og fødder til tydeligt at indikere hvilken retning playeren vendte. Farverne på playeren skulle være nogle der stak ud fra baggrunden hvilket er en af grundene til de meget flotte næsten pastelfarver der er blevet brugt. Da der skulle være en multiplayer funktion valgte vi at implementere farverne, for at gøre det bare lidt nemmere at kende folk fra hinanden, og for at give brugeren en chance for at designe playeren selv. Derfor lavede vi playeren i en række farver der ville være let genkendelige fra hinanden. til animationer af spilleren blev der lavet en række sprites hvor playeren går og caster spells.



**Enemies:** Fjenderne blev lavet på nøjagtig samme måde playeren, dog i stedet for at være let genkendelig og flot i farverne, var disse designet til at skulle fremstå lidt uhyggelige og til at falde lidt ind i baggrunden. de blev også tegnet som 32x32 pixel figurer, men blev i visse tilfælde skaleret op til en større proportion for at virke mere intimiderende.



**NPC Questgiver:** NPC'en er designet på samme måde som playeren i en 32x32 pixel formfaktor, og skulle være en bonde der havde fået angrebet sine marker. Dette var hvad der lå til grunde for denne sprites endelige udseende.



**Spells og Enemy attack:** Spells'ne og enemy attacks blev også tegnet i en 32x32 pixel formfaktor, med den ene guide at det skulle ligne noget der fløj gennem luften og at det var af de typer der var monstre eller enemies.

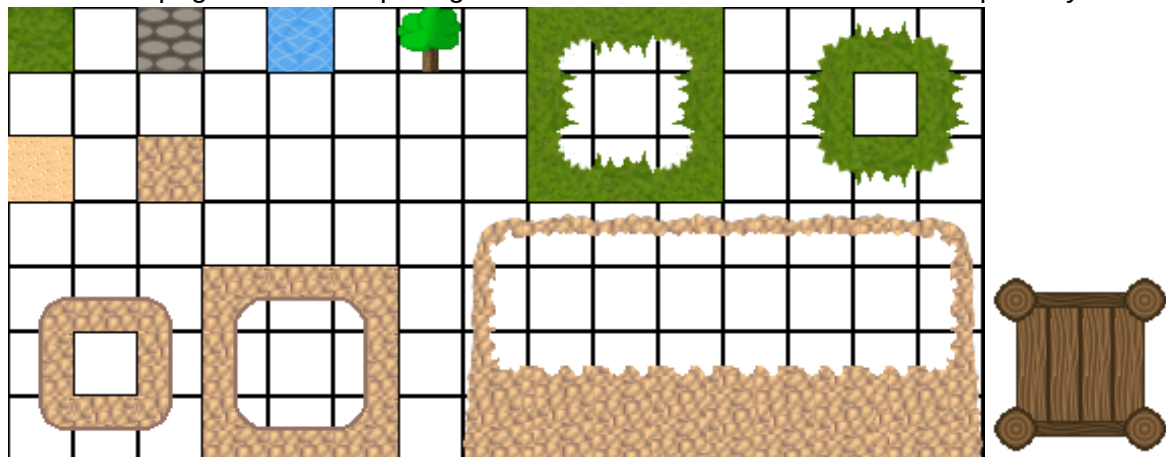


**Diverse:** Diverse er en lille række sprites som blev designet til specifikke UI implementeringer, såsom spell selection slots, questlog, eller respawn crystal.



## Oprettelse af sprites og spritesheet

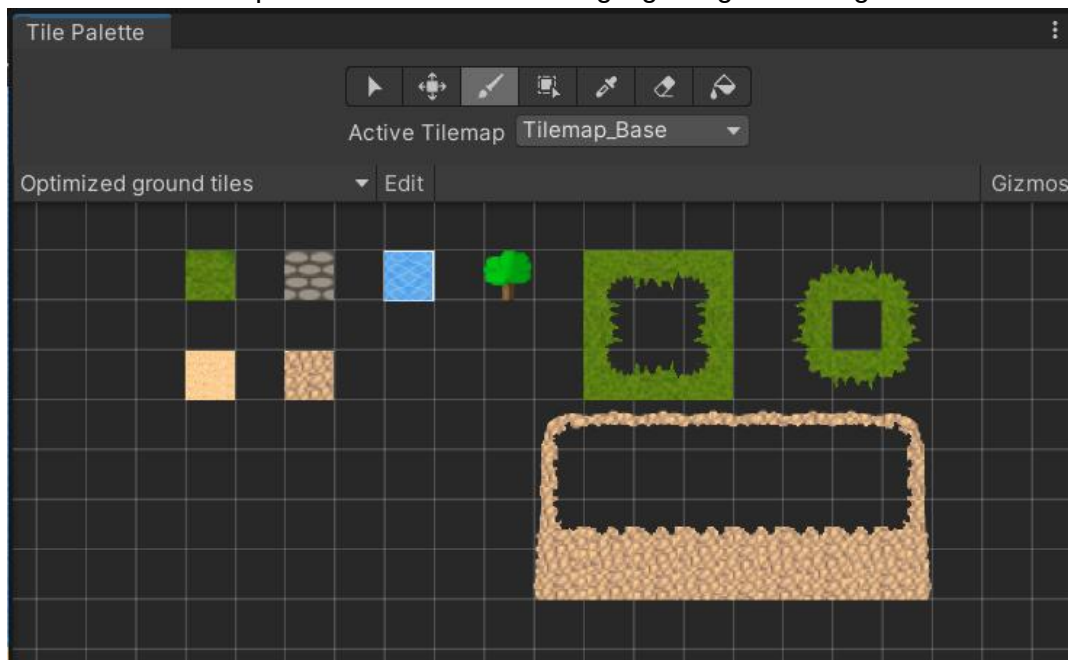
Hele vores map er opbygget af 32x32 pixel sprites som alle kan findes på et sprite sheet. Så den første grafiske opgave i Projektet var at få lavet et spritesheet med alle de forskellige elementer og områder vi ville have i spillet. Dette endte med et mountain biome og forrest, samt en række forskellige roads, og sidst men ikke mindst noget vand. Alt blev samlet og sat ind i en png fil i et 32x32 pixel gitter som senere vil kunne blive skåret op i unity.



## Spritesheet - Baneopbygning<sup>1</sup>

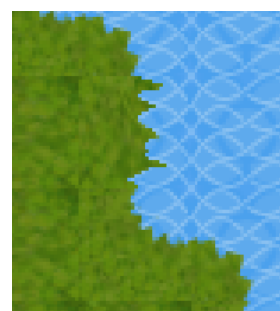
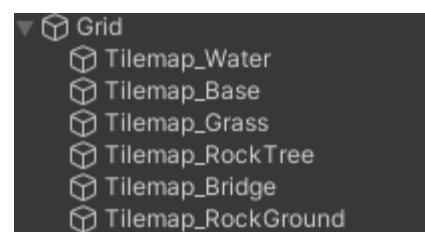
For at tegne størstedelen af vores grafik, bruger vi en funktion der hedder spritesheeting. Denne funktion gør det muligt at tegne store 2D-baner, blot ud fra et par tegninger som bliver skåret op i mindre dele. Ud fra disse dele kan man blot markere dem enkeltvis eller flere af gangen, og derefter indsætte dem på en ternet tegneflade.

Herunder ses det spritesheet som vi hovedsageligt bruger til at tegne vores verden.



Når vi arbejder med spritesheet sættes de ind på de her ternede tegneflader. Disse tegneflader kaldes for tilemaps. Det er som regelt en fordel at opdele ens kreationer i flere forskellige tilemaps. På den måde kan man inddеле dem i strukturer og niveauer. Vi har for eksempel oprettet seks forskellige lag af tilemaps. Dette gør man både fordi at nogle tilemaps skal ligge over nogen andre, og nogen tilemaps skal måske kollidere med andre objekter. Her til højre ses vores forskellige tilemaps brugt til at tegne vores baner. De er her også sat op i forhold til kollisioner og layers. Layering kan blot forstås som når man ligger en række papirer oven på hinanden i en specifik rækkefølge. Vores vand skal for eksempel ligge nederst, da vi skal kunne lægge sprites hen over vandet. Dette kan ses på billedet nederst til højre.

Her har vi lagt et tilemap med græs, hen over et tilemap med vand. Vi har på samme måde så også lavet et lag af går hen over en sti eller noget andet på jorden. På samme måde kan man også tildele enkelte tilemaps specifikke funktioner. Tilemappet til træerne og stenene, selvfølgelig have kollision så man ikke kan gå lig det. Det skal græsset og stierne dog ikke.



græs der

skal  
igennem

<sup>1</sup> Thirslund, Asbjørn, Tilemaps in Unity, 12/3 - 21

# Programmeringselementer

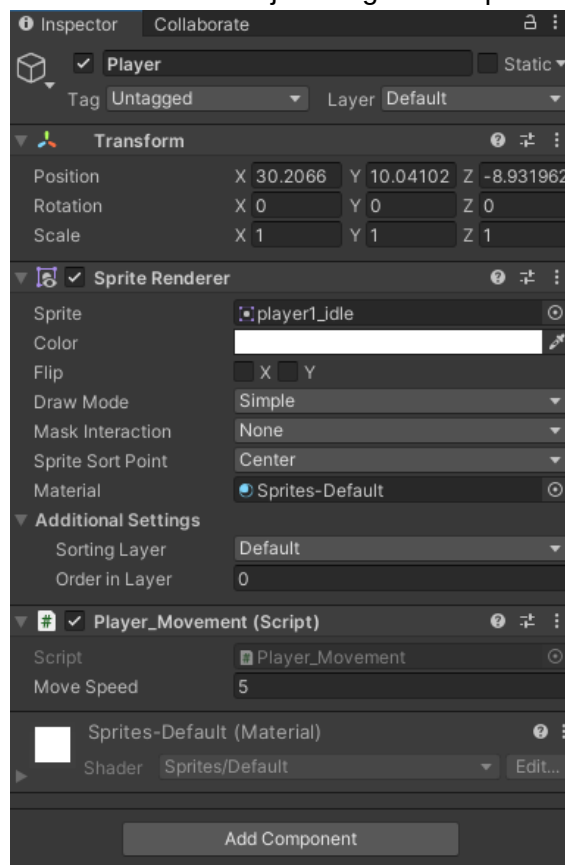
I dette afsnit vil vi komme ind på hvordan vi programmerede de vigtigste dele af vores spil. Da vores spil består af både en singleplayer del og en multiplayer del, har vi opdelt afsnittet i flere sektioner. Derudover vil vi også forklare nogle Unity relaterede emner og begreber, som er vigtige at forstå når man skal programmere spil i Unity.

## Unity-elementer

Unity består af en masse af deres helt egne elementer og funktioner. Hvis ikke man er så bekendt med Unity, så kan det godt lyde uklart og uforståeligt, hvis man ikke har styr på de forskellige begreber der bliver brugt. Vi vil derfor i dette afsnit redegøre for de begreber som vi kommer til at nævne mest igennem forklaringen af vores programmeringselementer

### GameObject

Et GameObject bruges til at repræsentere alt som kan eksisterer i en scene. Det betyder altså at alt som er i spillet er et GameObject. GameObjects er byggestenene til en scene i Unity, og det fungerer som en slags kasse som kan indeholde funktionelle komponenter såsom scripts, som bestemmer hvad GameObjectet kan gøre og hvordan det skal se ud. Et eksempel på et GameObject kunne være en spiller. Det vil sige, at der på GameObjectet højst sandsynlig er et script som får spilleren til at gå. Yderligere så kan man tilføje en sprite, således at GameObjectet ligner en spiller. Et eksempel på netop dette kan ses nedenunder.

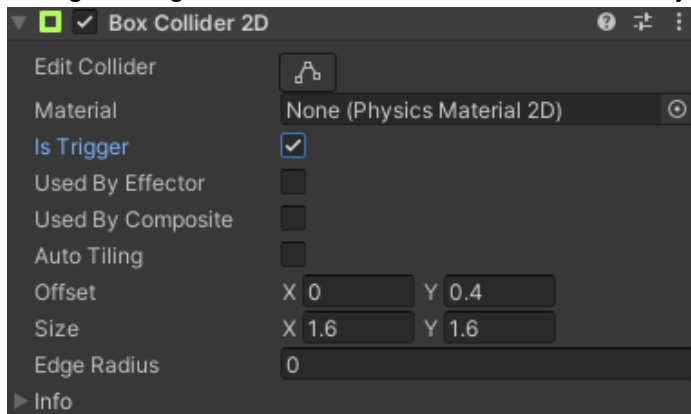


## Collisions 2D

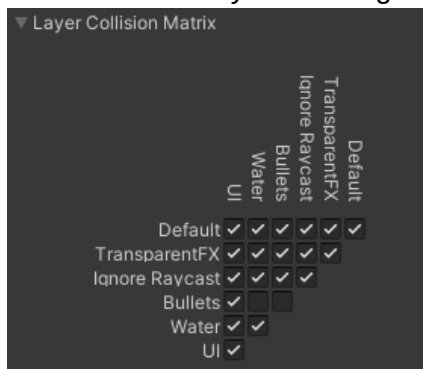
Collisions 2D er et komponent man næsten altid gerne vil have på sine GameObjects. Et collisionkomponent gør at GameObjectet kolliderer med andre GameObjects som også har dette komponent aktivt. Det bruges eksempelvis i vores spil, så spilleren ikke kan gå igennem vægge eller bjerge. Der findes forskellige former indenfor emnet. Hvis man har en cirkel, så vil man helst benytte 'Circle collider' fremfor 'Box collider', da man gerne vil have at kollideringsboksen ligner ens sprite så meget som muligt. Nedenfor ses to billeder af det samme GameObject men med to forskellige colliders. Det ses tydeligt at den firkantede collider passer bedst, da cirklen skaber et for stort mellemrum foran spilleren.



Yderligere kan en collider også udløse en funktion hvis den rammer noget andet. Hvis dette ønskes, så skal man sætte hak i 'Is Trigger'. Dette kan bruges, hvis man skyder et skud som skal gøre noget, hvis det rammer et andet GameObject.



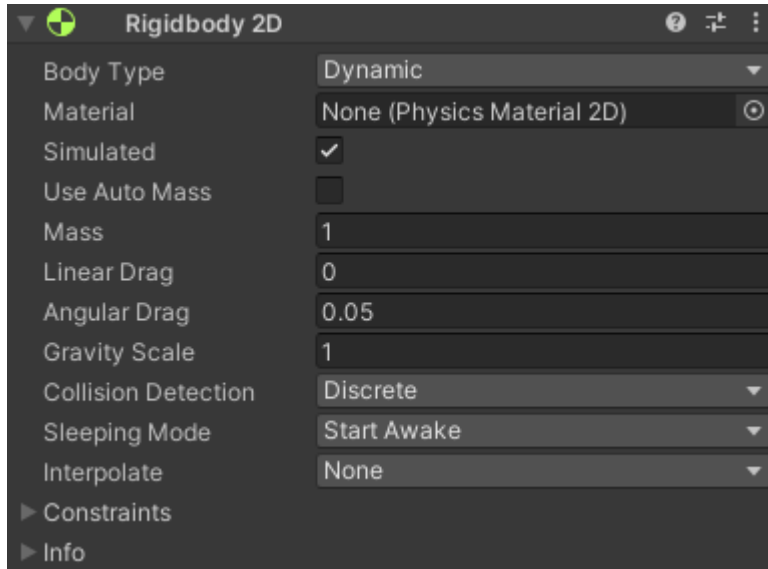
Man kan ydermere inde i Unity's indstillinger vælge hvilke lag som ikke skal kolliderer sammen. Det betyder at de ignorerer hinanden, hvis de kommer til at ramme hinanden.





## Rigidbody 2D

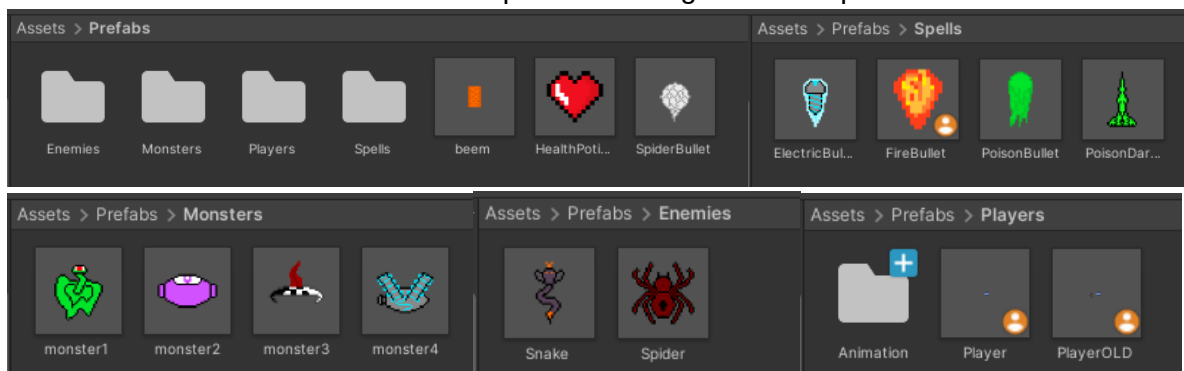
Ligesom Collisions 2D, så er Rigidbody 2D også et komponent som man kan sætte på sit GameObject. En Rigidbody gør at GameObjectet får fysiske egenskaber, såsom tyngdekraft påvirkning, friktion og en masse andet. Hvis der arbejdes med et top down spil, så skal tyngdekraften være slået fra, da man ellers ville falde nedad. Det smarte ved et GameObject som har en Rigidbody 2D, er at man kan påføre dem kræfter i form af vektorer. Dette gør at man kan rykke til GameObjectet, og er faktisk fundamentet for hvordan man får en spiller til at bevæge sig. Nedenfor ses et billede af hvordan komponentet ser ud.



## Prefabs

Unity's prefab-system gør det muligt at kunne gemme GameObjects med alle dets komponenter, egenskaber, værdier og childs (ting som ligger inde under GameObjectet). Denne mulighed laver altså et GameObject om til en slags skabelon der gør det muligt at lave flere instanser af det samme GameObject enten manuelt eller gennem scripts. Dette er et super godt alternativ til bare at duplikerer en masse af det samme GameObject. Det skyldes at hvis man skal lave store ændringer ved det dupligerede GameObject, så skal man gøre det ved alle duplikater. Hvis man derimod har det gemt som et prefab, så skal man kun ændre i prefabet, hvorefter at ændringen vil anvendes ved alle de andre instanser. Måden man gemmer et prefab er ved at tage ens GameObject, og trække det ned i ens Assets.

På billedet nedenunder ses det hvilke prefabs vi bruger i vores spil.



# Singleplayer

## Player & monster valg

Det sted hvor brugeren har mulighed for at gøre spillet mest personligt er ude i spillets menu. Det er her hvor brugeren har mulighed for at vælge navnet på spilleren, farven på spilleren og monsteret til spilleren.

Der bliver kun brugt 1 enkelt script til at holde styr på det hele, og det hedder "MenuController" og sidder på et GameObject med selvsamme navn. Som navnet indikerer, så er det nemlig det som skal kontrollere at det hele i menuen virker. Det er ting såsom knapperne til spillerne og monstrene, input til brugernavn og previewet af spilleren, monsteret og brugernavnet.

```
1 public class MenuController : MonoBehaviour
2 {
3     public TextMeshProUGUI usernamepreview;
4     public TMP_InputField UsernameInput;
5     public Image CurrentCharacter;
6     public Image CurrentMonster;
7
8     public static string CurrentCharacterValue;
9     public static string CurrentMonsterValue;
10
11     public static string UsernameIngame;
```

Til at starte med, så bliver der initialiseret nogle variabler.

usernamepreview bruges til at vise brugernavnet over spilleren i menuen.

UsernameInput bruges til at tage brugerens input til brugernavn.

CurrentCharacter og CurrentMonster bruges til at vise den valgte spiller og det valgte monster.

Den valgte spiller og det valgte monster har ligeledes en statisk streng-værdi, som bruges når spilleren bliver instantieret.

Herefter er der også en statisk streng-værdi som indeholder spillerens brugernavn.

```
1 public Sprite player1;
2 public Sprite player2;
3 public Sprite player3;
4 public Sprite player4;
5 public Sprite player5;
6 public Sprite player6;
7
8 public Sprite monster1;
9 public Sprite monster2;
10 public Sprite monster3;
11 public Sprite monster4;
```

Dernæst så bliver lavet og initialiseret en masse billeder. Disse billeder bruges til at vise hver enkelt farve som spilleren kan have, samt de fire forskellige monstre som brugeren kan vælge.

```
1 void Start()
2 {
3     //Vælger en start character og monster
4     ChangeCharacterToPlayer1();
5     ChangeMonsterToPlayer1();
6 }
```

Det første som bliver kørt er denne Start funktion. Inde i funktionen kører den to funktioner. Disse funktioner vælger spiller 1 og monster 1, da den skal starte med at vise noget i preview skærmen.

```
1 public void ChangeCharacterToPlayer1()
2 {
3     CurrentCharacter.sprite = player1;
4     CurrentCharacterValue = "player1";
5 }
```

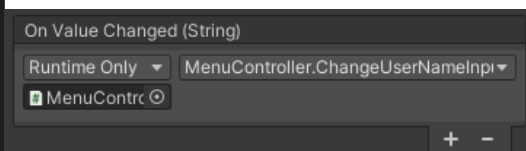
```
1 public void ChangeMonsterToPlayer1()
2 {
3     CurrentMonster.sprite = monster1;
4     CurrentMonsterValue = "monster1";
5 }
```

Måden funktionerne virker på, er udelukkende ved at ændre den spiller og det monster der er i previewet til den spiller og det monster som der er blevet klikket på, og ændre variablerne for den nuværende spiller og det nuværende monster.

```
1 void Update()
2 {
3     usernamepreview.text = UsernameInput.text;
4 }
```

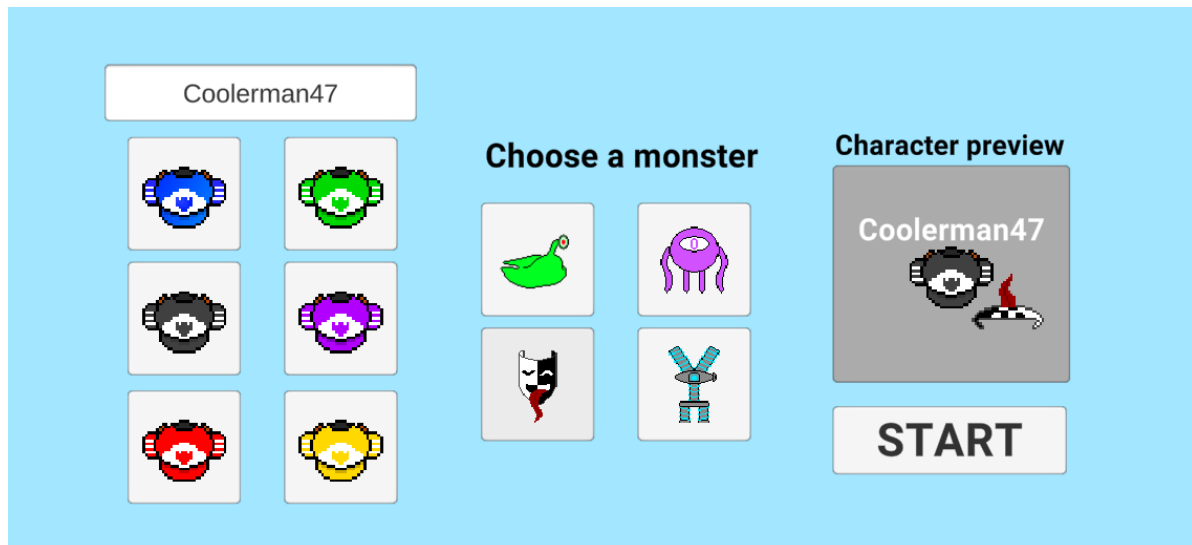
Måden brugernavnet bliver opdateret på i previewet er ved at den ved hver frame "update" tager fat i teksten ved previewet af brugernavnet og erstatter det med det som der er blevet skrevet i inputfeltet.

```
1 public void ChangeUserNameInput()
2 {
3     if(UsernameInput.text.Length >= 5)
4     {
5         StartButton.SetActive(true);
6     }
7     else
8     {
9         StartButton.SetActive(false);
10    }
11 }
```



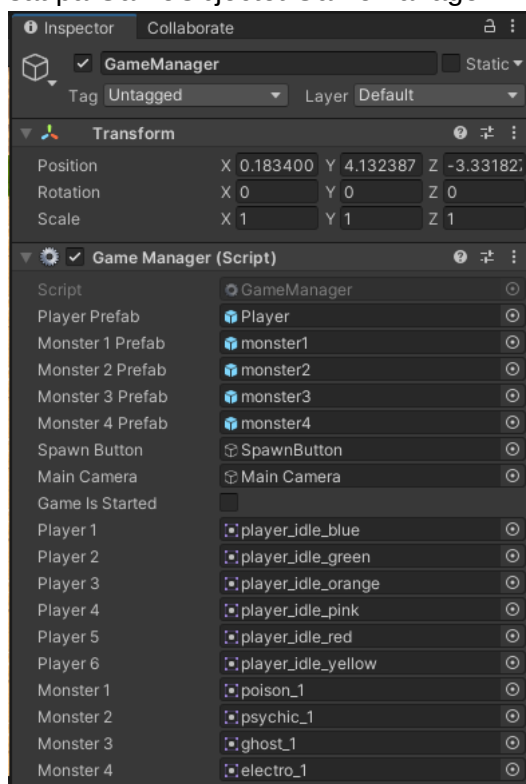
I menuen, så kommer startknappen ikke op før man har skrevet mindst 5 tegn i ens brugernavn. Måden dette fungerer på er, at den kører denne funktion hver gang at der bliver skrevet et tegn i input feltet.

Med et funktionelt script og et færdigt design, endte menuen med at se således ud:



## Player og monster instans

Når spiller-prefabet skal instantieres, så gør vi det i et script ved navn GameManager som er sat på GameObjectet GameManager.



Som det kan ses på det ovenstående billede, så bruger det en masse forskellige variabler. Spillerprefabben bruges til at instantiere spilleren på banen. Herefter bliver prefabs for alle monstre initialiseret, efterfulgt af en spawn knap og er kamera. Til sidst bliver alle sprites for hhv. spilleren og monstre initialiseret.

```
1 public void SpawnPlayer()
2 {
3     GameObject PlayerIns = Instantiate(PlayerPrefab, new Vector3(-2f, 3.9f, 0), Quaternion.identity);
4
5     if (MenuController.CurrentCharacterValue == "player1")
6     {
7         //PlayerIns.GetComponent<SpriteRenderer>().sprite = player1;
8         PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 1);
9     }
10    else if (MenuController.CurrentCharacterValue == "player2")
11    {
12        //PlayerIns.GetComponent<SpriteRenderer>().sprite = player2;
13        PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 2);
14    }
15    else if (MenuController.CurrentCharacterValue == "player3")
```

Efter alle variableerne er blevet lavet i GameManager scriptet, så kommer der en funktion som har til formål at instantiere spilleren. Funktionen bliver kørt når der bliver trykket på spawn-knappen når spillet starter. Når funktionen kører, så laver den en instans af spillerens prefab og gemmer det som en variabel, så vi kan bruge den som reference, når vi senere hen skal tilføje et monster til spilleren. Instantiate() funktionen skal have tre argumenter, i form af et prefab, en position og en rotation. Quaternion.identity betyder at den ikke har nogen rotationer, men at den justeret ud fra banens akser.

Herefter så kommer der et if-statement som skal tjekke for hvilken spiller som brugeren valgte i menuen. Hvis spilleren har valgt player1, så sætter den gang i animationen for player1 hvilket er den blå farve spiller. Førhen vi fik animationer, så brugte vi et stillestående billede. Her indsatte vi bare billedet for den blå spiller i stedet. Dette kan ses på det udkommenteret kode.

```
1 if(MenuController.CurrentMonsterValue == "monster1")
2 {
3     GameObject MonsterIns = Instantiate(Monster1Prefab, new Vector3(-1f, 3.8f, 0), Quaternion.identity);
4     MonsterIns.GetComponent<SpriteRenderer>().sprite = monster1;
5     MonsterIns.transform.parent = PlayerIns.transform;
6 }
7 else if (MenuController.CurrentMonsterValue == "monster2")
```

Det samme sker, når der skal tjekkes for hvilket monster der er valgt. Hvis brugeren valgte monster 1, så laver den en instans af monsteret, med nogle bestemte koordinater således den er bag spilleren. Herefter sætter vi den korrekte sprite på monsteret og gør så det bliver spilleren bliver en parent til monsteret.

Efter dette if-statement, så er spilleren blevet aktiv i scenen sammen med monsteret, og brugeren er nu klar til at bevæge spilleren rundt og skyde.

Nedenfor kan det ses at in-action, hvor der er en startknap, og når det trykkes på, så instantieres spilleren, dets monster og dets GUI.



## Player<sup>2</sup>

Selve spilleren har dets eget script, som skal holde styr på ting såsom liv og skade taget.

I starten af scriptet bliver der initialiseret to lydclip og to audiosources. De to lydclip er af spilleren der tager skade, og to audiosources er komponenter som kan sættes på spilleren. Spilleren får også defineret hvor meget liv den har, og en variabel som skal indeholde de nuværende liv som spilleren har. Defineres spilleren også, en healthbar, brugernavnet, teksten med antal liv samt hele det canvas som indeholder spillerens GUI.

```
1 public class Player : MonoBehaviour
2 {
3     public AudioClip PlayerDamage1;
4     public AudioClip PlayerDamage2;
5     AudioSource audioPlayerDamage1;
6     AudioSource audioPlayerDamage2;
7
8     public float maxHealth = 100f;
9     public float currentHealth;
10    public GameObject player;
11
12    public HealthBar healthBar;
13
14    public TMP_Text usernameText;
15    public TMP_Text HPText;
16
17    public GameObject UserCanvas;
```

```
1 void Start()
2 {
3     audioPlayerDamage1 = AddAudio(false, false, 1f);
4     audioPlayerDamage2 = AddAudio(false, false, 1f);
5
6     currentHealth = maxHealth;
7     healthBar.SetMaxHealth(maxHealth);
8
9     usernameText.text = MenuController.UsernameIngame;
10    HPText.text = currentHealth.ToString("F1") + "/100,0";
11
12    PauseMenu.GameIsPaused = false;
13    CheckForDeath.GameOver = false;
14    CheckForDeathQ1.GameOver = false;
15    UserCanvas.SetActive(true);
16
17 }
```

Inde i start funktionen, bliver der lavet nogle indstillinger til to audiosources. Hvordan de virker bliver forklaret nede i sektionen vedrørende Quests. Herefter bliver spillerens nuværende liv sat til spillerens maksimale antal liv, da spilleren lige er spawnet. Yderligere så fyldes healthbaren op. Dernæst så bliver brugernavnet sat til det som der blev skrevet i menuen, samt antallet af liv bliver også vist. Til sidst, så bliver en masse variabler sat til false, samt spillerens GUI bliver slået til.

---

<sup>2</sup> Thirslund, Asbjørn, 2D Animation in Unity,13/4



```
1 public void TakeDamage(float damage)
2 {
3     currentHealth -= damage;
4     healthBar.SetHealth(currentHealth);
5
6     int SoundRandomizer = Random.Range(1, 100);
7     if (SoundRandomizer <= 50)
8     {
9         audioPlayerDamage1.clip = PlayerDamage1;
10        audioPlayerDamage1.Play();
11    }
12    else if (SoundRandomizer > 50)
13    {
14        audioPlayerDamage2.clip = PlayerDamage2;
15        audioPlayerDamage2.Play();
16    }
17 }
```

Inde i spillerens script, findes denne funktion som gør at spilleren kan tage skade af andre fjender. Denne funktion køres, når spilleren er blevet ramt af et projektil fra en fjende. Den tager ét argument som er mængden af skade som er påført. Herefter så ændrer den på mængden af liv og på healthbaren. Dernæst så finder den et random tal, som skal bruges til at vælge mellem to forskellige lyde, som afspiller en lydclip der lyder som en person der tager skade.

```
1 void Update()
2 {
3     HPText.text = currentHealth.ToString("F1") + "/100,0";
4
5     healthBar.SetHealth(currentHealth);
6
7     if (SpiderBullet.Losthealth == 1)
8     {
9         TakeDamage(Spider.bulletdamage);
10        SpiderBullet.Losthealth = 0;
11    }
12
13    if (SnakeBullet.Losthealth == 1)
14    {
15        Debug.Log("Skudt");
16        TakeDamage(Snake.bulletdamage);
17        SnakeBullet.Losthealth = 0;
18    }
19
20    if (currentHealth <= 0)
21    {
22        Destroy(player);
23    }
```

Inde i update funktionen bliver spillerens liv på UI'et og i healthbaren hele tiden opdateret, således det matcher med hvor mange liv spilleren har.

Herefter så tjekker den om spilleren har taget skade fra en edderkop. Det gør den ved at tjekke efter variablen Losthealth i scriptet for edderkoppens skud. Når edderkoppens skud rammer spilleren, så sættes denne Losthealth variabel til 1. Når den er sat til 1, så køres en funktion hvor spilleren tager skade, hvor argumentet til funktionen er mængden af skade påført. Herefter sættes Losthealth til 0 igen. Det samme sker når den tjekker for om den er blevet ramt af slangen. Herefter tjekker den om spillerens nuværende liv er 0 eller under. Hvis dette er sandt, så skal den fjerne spilleren som instans.

Ydermere inde i update funktionen så tjekker den om hvorvidt spilleren er død. Det gør den inde i et script som hedder CheckForDeath og CheckForDeathQ1. Inde i begge scripts tjekker de for om spilleren stadig eksisterer i scenen. Hvis ikke, så bliver gameover sat til at være sand, og så gør den spillerens UI usynligt. Hvis spilleren ikke er død, så tjekker den i stedet for om spillerne på noget tidspunkt pauser spillet. Hvis spillet er pauset, så skal den også gøre spillerens UI usynlig.

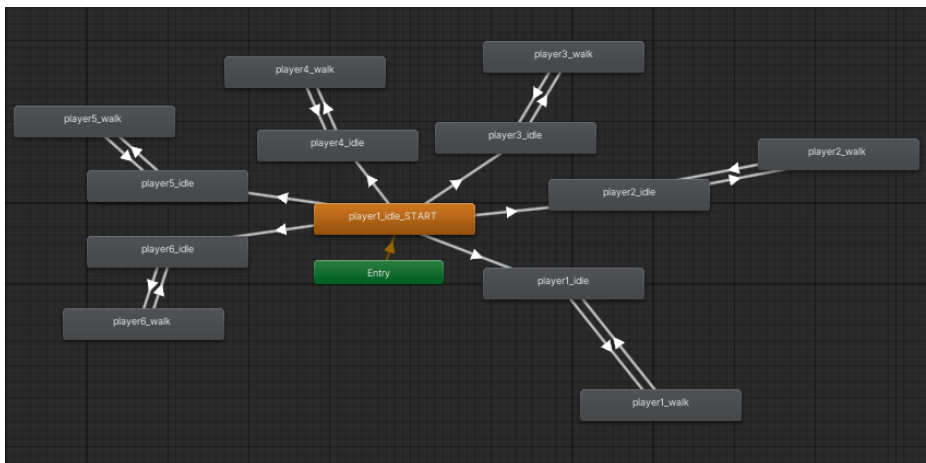
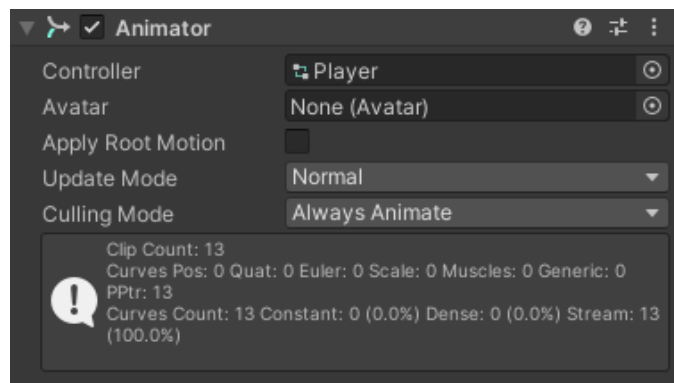
```
1      if (CheckForDeath.GameOver == true)
2      {
3          UserCanvas.SetActive(false);
4      }
5      else if (CheckForDeath.GameOver == false)
6      {
7          if (PauseMenu.GameIsPaused == true)
8          {
9              UserCanvas.SetActive(false);
10         }
11         else if (PauseMenu.GameIsPaused == false)
12         {
13             UserCanvas.SetActive(true);
14         }
15     }
16
17     if (CheckForDeathQ1.GameOver == true)
18     {
19         UserCanvas.SetActive(false);
20     }
21     else if (CheckForDeathQ1.GameOver == false)
22     {
23         if (PauseMenu.GameIsPaused == true)
24         {
25             UserCanvas.SetActive(false);
26         }
27         else if (PauseMenu.GameIsPaused == false)
28         {
29             UserCanvas.SetActive(true);
30         }
31     }
```

Resultatet kan ses på billedet nedenunder hvor det ses at spillerens liv i GUI'en bliver opdateret, samt at spillerens GUI bliver usynligt når spilleren dør eller man pauser spillet.

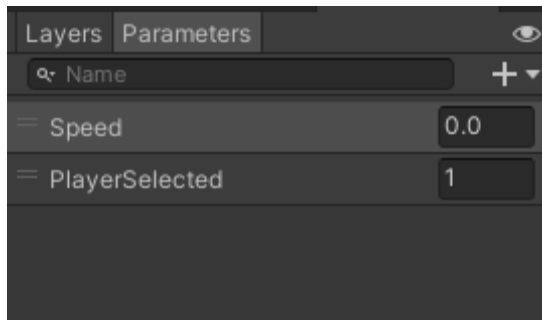


## Player movement & animation

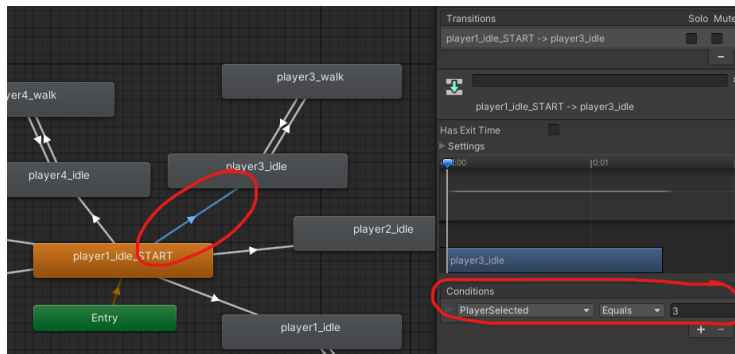
Når man i menuen skal vælge en spiller, så vælges der mellem seks forskellige farver. Hver farve har en idle animation til når spilleren står stille og en run animation til når spilleren bevæger sig. Det betyder, at når spillet skal vælge den rigtige animation til spilleren, så skal spillet først finde ud af hvilken farve spilleren har og derefter om spilleren er i bevægelse.



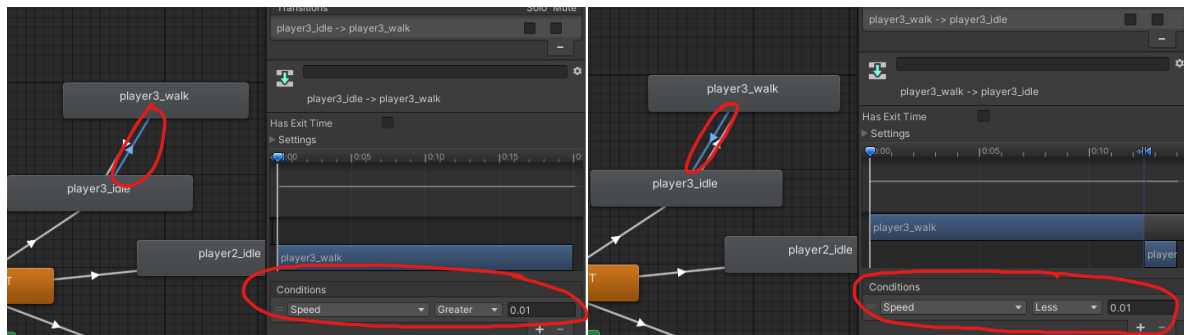
Måden vi har løst dette, er ved at sætte et Animator komponent på vores spiller-prefab. Inde i dette komponent, kan man tilføje en masse animationer som skal afspilles hvis et bestemt kriterium er opfyldt. Animationerne er alle de grå bokse og den ene orange boks. Imellem disse sidder en nogle pile som har bestemte kriterier der skal der opfyldt før de kan afspille den animation den peger hen imod. Når spilleren bliver instantieret, så starter den på den grønne blok ved navn Entry. Herefter skal den pege hen på den orange boks, som kaldes et default state. Det betyder at det er den animation som bliver afspillet først når spilleranimationen starter. Herfra skal programmet afgøre hvilken af de 6 spiller animationer den skal gå ud til. Dette gør den ud fra hvilken spiller der blev valgt i menuen. Når den har valgt en spiller, så skal programmet herefter hele tiden tjekke om spilleren bevæger sig eller står stille. Hvis spilleren bevæger sig, så går den fra animationen playerx\_idle til playerx\_walk. Omvendt, hvis spilleren ikke bevæger sig, så går den fra playerx\_walk til playerx\_idle.



Måden vi styrer hvilken spilleranimation programmet skal gå ud til, er ved at vi laver og benytter parametre som vi har lavet i animatoren. Her har vi lavet float Speed og integer PlayerSelected. Disse skal bruges til at vurdere hvilken animation som der skal afspilles.

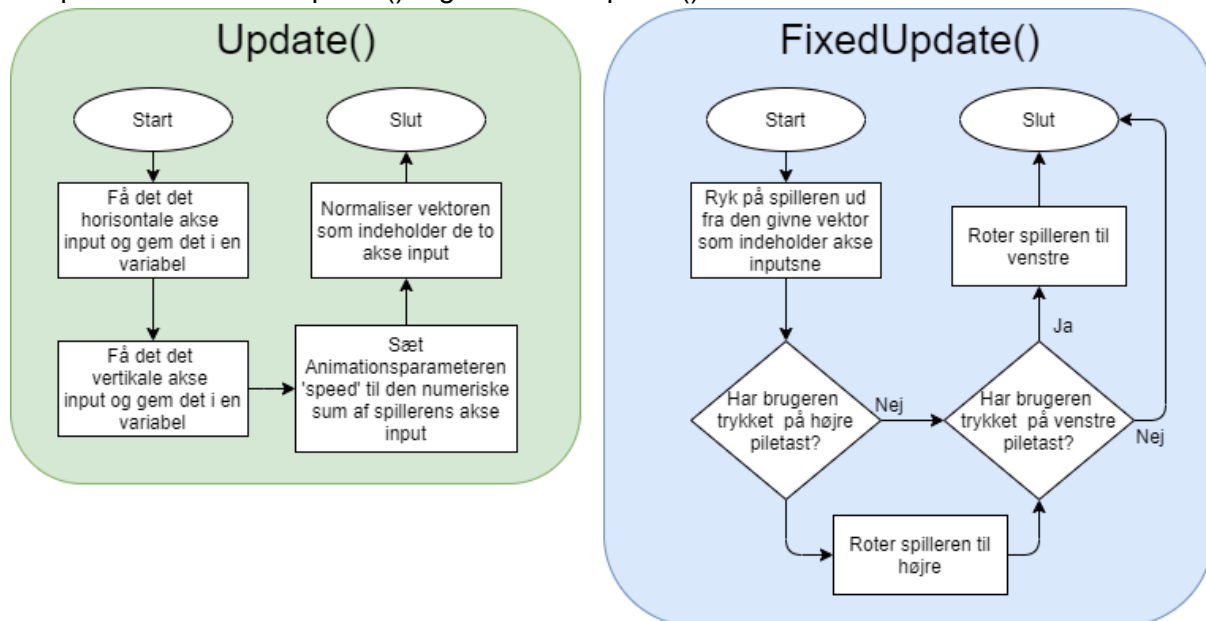


På de første seks pile, som går ud fra den orange default state, er der et kriterium som hedder PlayerSelected. Det vil sige, at for at kunne køre animationerne for player3, så skal PlayerSelected være 3. Dette er gældende for alle seks spilleranimationer. Måden vi sætter denne parameter, er når vi spawner playeren, hvilket vi beskrev i afsnittet *Player og monster instans*. Her bliver parameteren ændret til et tal mellem 1 og 6. Herefter så bliver ét af de seks kriterier opfyldt, og så kan programmet gå i gang med at veksle mellem om spilleren spillere står stille eller bevæger sig.



Når den skal finde ud af, om spilleren går eller bevæger sig, bliver der beregnet en hastighedsværdi, som her bruges til at finde ud af om spilleren bevæger sig eller står stille. Hvis hastigheden er over 0,01, så skifter den til bevægelsesanimationen. Hvis hastigheden er mindre end 0,01, så skifter den til idle animationen. Måden hvorpå hastigheden beregnes bliver forklaret i næste afsnit, hvor der bliver forklaret om spillerens bevægelser.

Når vi skal have spilleren til at bevæge sig, gør vi brug af et script som vi har kaldt PlayerMovement. Før vi gik i gang med at kode scriptet, så lavede vi flowchartet for at kunne danne os et overblik over hvordan vi senere hen skulle programmere scriptet. Vi har dog senere hen opdateret det en smule, så det passede med at spilleren havde animationer. Scriptet består af en "Update()" og en "FixedUpdate()".



I vores "Update()" skal det starte med at få fat i inputtet fra den horisontale og vertikale akse, og få gemt det i en variabel. Herefter skal vi have sat en speed parameter fra animatoren til en numerisk sum af de to inputs. Til sidst skal vektoren som indeholder de to inputs normaliseres, således at man ikke kan gå hurtigere diagonalt. Når scriptet slutter, så starter det forfra, da det er en funktion som kører hvert frame.

I vores "FixedUpdate()" starter vi med at rykke spilleren ud fra den vektor som er blevet normaliseret. Herefter tjekkes der om brugeren trykker på enten højre eller venstre piletast. Hvis der bliver trykket på en af piletasterne, så skal spilleren rotere til enten højre eller venstre. Når scriptet slutter, så starter det forfra, da det er en funktion som kører 50 gange i sekundet.

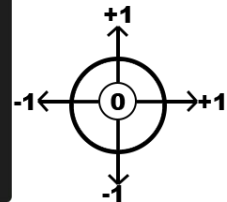


Nu vil vi komme ind på hvordan vi har programmeret "PlayerMovement"-scriptet.<sup>3</sup>

```
1 public class PlayerMovement : MonoBehaviour
2 {
3
4     public float moveSpeed = 5f;
5
6     public Rigidbody2D rb;
7
8
9     public Animator animator;
10
11     Vector2 movement;
```

I starten af scriptet definerer vi hastigheden af vores spiller. Herefter initialiserer vi spillerens Rigidbody 2D komponent samt Animator komponentet. Til sidst initialiserer vi også en vector som skal indeholde spillerens bevægelser.

```
1 void Update()
2 {
3     movement.x = Input.GetAxisRaw("Horizontal");
4     movement.y = Input.GetAxisRaw("Vertical");
5
6     animator.SetFloat("Speed", (Mathf.Abs(movement.x) + Mathf.Abs(movement.y)));
7
8     movement = movement.normalized;
9     print(movement);
10 }
```



Inde i vores update funktion tjekker vi for hver frame update om spilleren bevæger sig enten horisontalt eller vertikalt. Dette giver en værdi som enten er 0, -1 eller +1. Det vil sige at hvis spilleren bevæger sig udelukkende til højre, så bliver værdierne for movement variabelen (+1, 0). Herefter så bliver Animator parameteren 'Speed' sat til at være den numeriske værdi af movemen-tvektoren. Grunden til at vi skal tage den numeriske værdi er fordi vi i Animatoren tjekker for om hastigheden er over 0,01. Derfor nytter det ikke at vi får en hastighedsværdi på -2.

Det næste der sker er at vi normaliserer vores movement vektor. Dette gør vi fordi, at hvis man valgte at gå diagonalt, så ville man bevæge sig omkring 40% hurtigere. Dette ved vi, da længden af en vektor findes ved sige  $\sqrt{movement.x^2 + movement.y^2}$ . Dette betyder at hvis man udelukkende bevæger sig til højre, fås længden af vektoren til at være  $\sqrt{+1 + 0} = 1$ . Hvis man derimod bevæger sig diagonalt, så bliver længden af vektoren  $\sqrt{1 + 1} \approx 1,4$ , hvilket svarer til en hastighedsforøgning på 40%. Dette vil vi ikke have, så derfor benytter vi en af Unity's funktioner som hedder *normalized*. Det som denne funktion gør, er at den justerer movement.x og movement.y således at længden af movement vektoren ikke kommer til at overstige 1. På denne måde undgår vi at spilleren bevæger sig hurtigere diagonalt.

<sup>3</sup> Thirslund, Asbjørn, 2D Movement in Unity, 12/3 - 21

```
1  void FixedUpdate()  
2  {  
3      rb.MovePosition(rb.position + movement * moveSpeed * Time.fixedDeltaTime);  
4  
5      if (Input.GetKey(KeyCode.LeftArrow))  
6      {  
7          rb.rotation += 6.0f;  
8      }  
9      else if (Input.GetKey(KeyCode.RightArrow))  
10     {  
11         rb.rotation -= 6.0f;  
12     }
```

Vi har ligeledes også en FixedUpdate, som skal opdatere spillerens Rigidbody 2D komponent. I en normal Update funktion, bliver funktionen kørt for hver frame. Det vil sige, at hvis en frame tager længere tid om at blive behandlet, så får man et svingende interval for hvornår funktionen bliver kørt. Det er ikke et godt fundament til når man skal bevæge elementer med fysiske egenskaber, da man derved kan risikerer at spilleren bevæger sig langsomt det første sekund, og næste sekund bevæger sig dobbelt så hurtigt. Derfor benytter man sig af en FixedUpdate. Denne funktion bliver kaldt på et bestemt tidspunkt. Det vil sige, at den konsekvent bliver kaldt hvert 0,02 sekund, hvilket svarer til 50 gange i sekundet. Dette er en værdi som kan ændres i projektets indstillinger. Den konsekvente eksekvering af funktionen gør det meget mere optimalt at benytte denne funktion til at bevæge elementer med fysiske egenskaber.

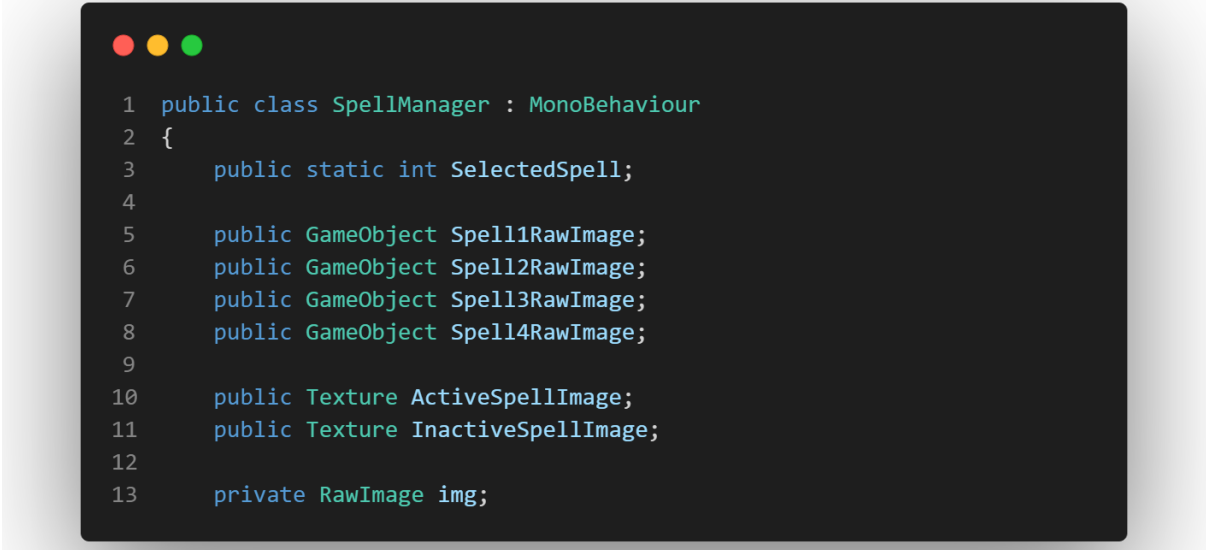
For at rykke på spilleren, så benyttes funktionen MovePosition(), som er en indbygget funktion i Unity. Den skal have ét argument som er en vektor. For at finde den nye vektor, så tages den nuværende position og adderes sammen med den normaliserede movement vektor. Herefter multipliceres det med spillerens hastighed, og til sidst multipliceres det med Time.fixedDeltaTime, da den kun skal opdatere positionen for den ene Fixed update som kører hvert 0,02'ende sekund.

Efter scriptet har rykket på spilleren, så tjekker den om der bliver trykket på venstre eller højre piletast. Hvis der bliver trykket på en af dem, så roterer den spilleren enten 6 grader til højre eller 6 grader til venstre.

## Spells<sup>4</sup>

Til at programmere vores spell-funktion har vi lavet tre vigtige scripts som står for det. Vores første script hedder "SpellManager", og er sat op for at tjekke hvilket spell der er markeret lige nu og hvad der så skal tages stilling til. Vores andet script til spell-funktionen er "Shooting"-scriptet. Dette script for selve "spellet", til at blive skudt af sted ud fra spillerens lokation. Skuddet der bliver sendt af sted, vi i vores spil være oprettet som en prefab der blot skal instantiates. På denne prefab sidder det tredje script. Der vil være fire forskellige prefabs der næsten er identiske. Disse fire prefabs har hver deres script hvor der vil være forskellige værdier, men scriptene vil ellers være ens. Disse fire prefabs er altså de fire forskellige spells man kan skyde afsted.

Herunder vil vi begynde med at beskrive vores "spellmanager"-script:



```
1 public class SpellManager : MonoBehaviour
2 {
3     public static int SelectedSpell;
4
5     public GameObject Spell1RawImage;
6     public GameObject Spell2RawImage;
7     public GameObject Spell3RawImage;
8     public GameObject Spell4RawImage;
9
10    public Texture ActiveSpellImage;
11    public Texture InactiveSpellImage;
12
13    private RawImage img;
```

Vi starter med at definere en lang række vigtige variabler for scriptet. Først skal vi have en variabel der tjekker for hvilket spell, der lige nu er markeret af spilleren. Denne variable skal selvfølgelig være en static float, da andre scripts skal kunne referere til den. Herefter hentes billederne af de fire spells. Vi henter to sprites som skal vise spilleren hvilket spell der er markeret.

---

<sup>4</sup> Thirslund, Asbjørn, Top Down shooting in Unity, 12/3 - 21

```
1 void Spell1Active()
2 {
3     img = (RawImage)Spell1RawImage.GetComponent<RawImage>();
4     img.texture = (Texture)ActiveSpellImage;
5 }
6 void Spell2Active()
```

```
1 void Spell1Inactive()
2 {
3     img = (RawImage)Spell1RawImage.GetComponent<RawImage>();
4     img.texture = (Texture)InactiveSpellImage;
5 }
6 void Spell2Inactive()
```

For at holde styr på de grafiske elementer af spellsne, opretter vi to funktioner. En der sætter det som aktivt og et der sætter det som inaktivt. Disse funktioner kalder vi så på når de bliver relevante.

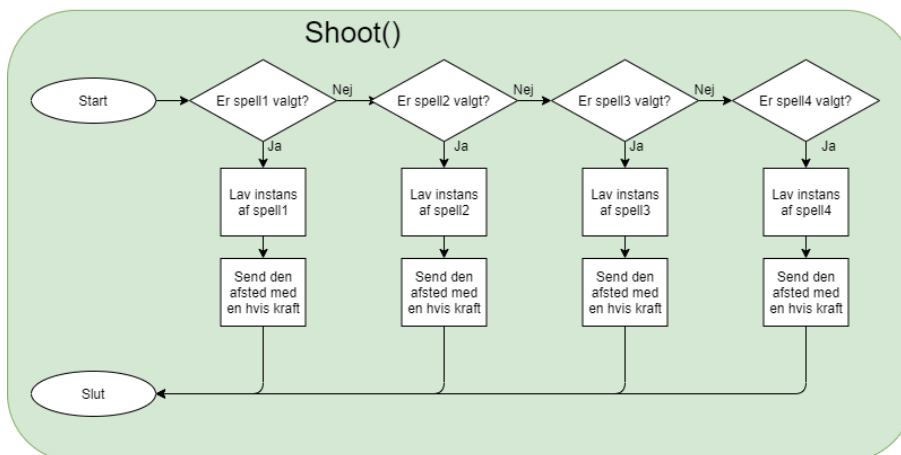
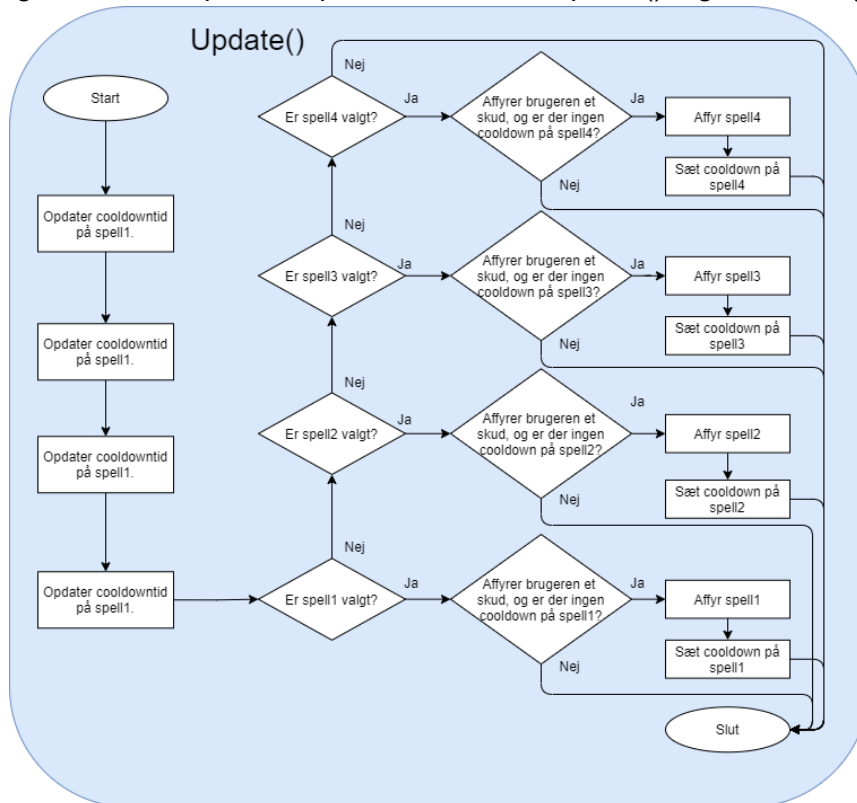
```
1 void Update()
2 {
3     if (Input.GetKey("1"))
4     {
5         SelectedSpell = 1;
6         Spell1Active();
7         Spell2Inactive();
8         Spell3Inactive();
9         Spell4Inactive();
10    }
11    if (Input.GetKey("2"))
```

Inde i vores update funktion, tjekker vi så for hvilket spell der bliver markeret af spilleren. Hver gang spilleren så markere et nyt spell, så sætter den det spell aktivt og sætter de andre spells som inaktive. Derudover angives variabelen der holder styr på hvilket spell der er markeret.

```
1 void Start()
2 {
3     SelectedSpell = 1;
4     Spell1Active();
5 }
```

Til sidst i "Spellmanager"-scriptet, sættes det markerede spell til 1 som standard, når man kører scriptet.

Herefter vil vi forklare "Shooting"-scriptet. Før vi gik i gang med at kode scriptet, så lavede vi et flowchart for at kunne danne os et overblik over hvordan vi senere hen skulle programmere scriptet. Scriptet består af en "Update()" og en "Shoot()".



I Update() opdaterer den tiden på hvert spells cooldown, således spilleren ikke hele tiden kan skyde spells afsted. Herefter tjekker den for hvilket spell der er aktivt. Når den har fundet det aktive spell, så tjekker den for om brugeren affyrer et skud og om spellets cooldown er lig 0. Hvis dette er opfyldt, så affyrer den spillet og sætter en cooldown. Når scriptet slutter, så starter det forfra, da det er en funktion som kører hvert frame.

Shoot() er den funktion som bliver kørt, når der skal affyres et spell. I denne funktion tjekkes der for hvilket spell som er aktivt og herefter laver den en instans af det valgte spell, og giver det en kraft så det bliver skudt afsted. Man kunne godt have lavet 4 separate shoot funktioner til hver spell, og dette ville være optimalt, da et if-statement bruger mere kraft på at skulle vurdere ting, end hvis den bare skulle eksekvere nogle funktioner. Dog er der næsten ingen forskel, så derfor valgte vi ikke at lave det om.

Nu vil vi forklare hvordan vi har programmeret "Shooting"-scriptet.

I dette script laver vi selve funktionen der skal kunne skyde. Først definerer vi en række vigtige variabler. Vi henter de forskellige prefabs for de fire spells. Derudover henter vi et "firepoint", som er der hvor skuddet skal indsættes. Derefter angiver vi en kraft som disse spells kan flyve med. Vi angiver en cooldown(mængde tid hvor spillet ikke kan affyres) til hvert spell. Derefter angiver vi en variabel der hel tiden skal være lig med, hvor lang tid der er til man kan affyre de forskellige spells. Til sidst hentes de fire spell-cooldowns ind som TMP\_text, som er en variabel, der gør at vi kan vise spilleren, hvor lang tid der er til et spell kan blive affyret igen.

```
1 public class Shooting : MonoBehaviour
2 {
3     public Transform firePoint;
4     public GameObject fireBulletPrefab;
5     public GameObject poisonDartBulletPrefab;
6     public GameObject electricBulletPrefab;
7     public GameObject poisonBulletPrefab;
8
9
10    public float autobulletForce = 20f;
11    public float poisonbulletForce = 6f;
12
13    public float cooldownFire = 1f;
14    public float cooldownPoisonDart = 0.2f;
15    public float cooldownElectric = 15f;
16    public float cooldownPoison = 3f;
17    private float Spell1autoNextFireTime = 0f;
18    private float Spell2autoNextFireTime = 0f;
19    private float Spell3autoNextFireTime = 0f;
20    private float Spell4autoNextFireTime = 0f;
21
22    public TMP_Text Spell1Countdown;
23    public TMP_Text Spell2Countdown;
24    public TMP_Text Spell3Countdown;
25    public TMP_Text Spell4Countdown;
```

```
1 void Shoot()
2 {
3     if (SpellManager.SelectedSpell == 1)
4     {
5         GameObject bullet = Instantiate(fireBulletPrefab, firePoint.position,
6         firePoint.rotation);
7         Rigidbody2D rb = bullet.GetComponent<Rigidbody2D>();
8         rb.AddForce(firePoint.up * autobulletForce, ForceMode2D.Impulse);
9     }
10    if (SpellManager.SelectedSpell == 2)
```

Vi opretter nu selve funktionen der gør at vi kan skyde. Først så tjekker vi for hvilket spell spilleren har markeret. Ud fra denne viden, ved vi hvad for et prefab vi skal indsætte. Vi finder prefabs rigidbodycomponent. Når vi har fundet den kan vi kalde på en indbygget kommando der hedder "AddForce", som danner en vector. Vi angiver at skuddet skal startes i punktet "firePount", og der skal så lægges kraft til.



```
1 void Start()
2 {
3     Spell1Cooldown.text = Spell1autoNextFireTime.ToString("F1");
4     Spell2Cooldown.text = Spell2autoNextFireTime.ToString("F1");
5     Spell3Cooldown.text = Spell3autoNextFireTime.ToString("F1");
6     Spell4Cooldown.text = Spell4autoNextFireTime.ToString("F1");
7 }
```

Når vi kører scriptet, sætter vi den nuværende cooldown på de fire spells til at være lig med en ny variabel af med variabeltypen string. Disse fire variabler skal vi bruge til at vise cooldown for de fire spells grafisk i Unity.

```
1 if (SpellManager.SelectedSpell == 1)
2 {
3     if (Input.GetKey(KeyCode.Space) && Time.time > Spell1autoNextFireTime)
4     {
5         Shoot();
6         Spell1autoNextFireTime = Time.time + cooldownFire;
7     }
8 }
9 else if (SpellManager.SelectedSpell == 2)
```

Inde i update-funktionen, tjekker vi så først efter hvad spell er markeret af spilleren, og herefter om spilleren trykker på mellemrumsknappen, som vi har valgt at gøre til vores affyringsknap. Hvis spilleren så trykker på mellemrumsknappen og der ingen cooldown er på det givne spell, så køres shoot-funktionen som instantierer skuddet og sender det afsted. Herefter sætter vi cooldown på, så spilleren ikke kan skyde med det samme igen.

Herunder vil vi forklare det sidste script til spell-funktionen. Dette script er sat på alle spellsne som prefabs.

Vi definere i dette script blot tre variabler. To variabler der tilsammen skal bestemme hvor lang tid skuddet skal flyve før det fjernes. Der defineres så en variabel der beskriver det enkelte spells skade.

```
1 public class FireSpell : MonoBehaviour
2 {
3     public float lifeDuration = 0.4f;
4     public float lifeTimer;
5     public static int BulletDamage = 10;
6 }
```

Når spell-scriptene kører, starter vi med at sætte "lifeTimer" lig med "lifeDuration". Denne variabel skal nemlig bestemme hvor lang tid vores spell kan kunne flyve uden at blive fjernet.

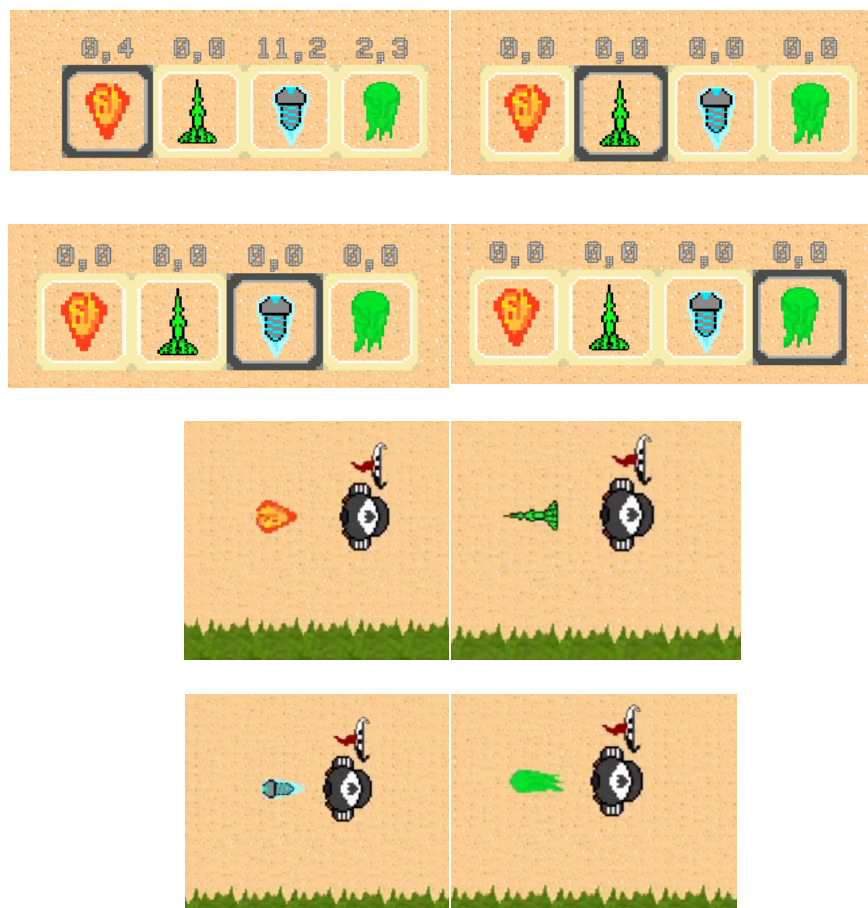
```
1 void Start()
2 {
3     lifeTimer = lifeDuration;
4 }
```

```
1 void Update()
2 {
3     // Tjekker om skuddet skal ødelægges.
4     lifeTimer -= Time.deltaTime;
5     if (lifeTimer <= 0f)
6     {
7         Destroy(gameObject);
8     }
9 }
10
11 void OnCollisionEnter2D(Collision2D collision)
12 {
13     Destroy(gameObject);
14 }
```

I update-funktionen starter vi først en timer, og derefter tjekker vi så for hvornår tiden går i nul eller eventuelt lavere. Hvis tiden rammer nul eller lavere, så bruger vi kommandoen destroy() for at fjerne spillet som gameobject.

Spillet skal dog også fjernes hvis det rammer et vilkårligt object med en collider. For at tjekke for om spillet kolliderer med en vilkårlig collider, bruger vi kommandoen; "OnCollisionEnter2D". Denne kommando tjekker bare om man rammer en anden collider2D.

Resultatet kan ses på billedet nedenunder hvor der spellbaren vises med de forskellige spells, samt et billede af spilleren som affyrer de valgte spells.



## Enemies<sup>5</sup>

Til at starte med definerer vi en lang række variabler, vi skal bruge for at programmere en fjende.

Alle public-variablerne bliver oprettet så vi kan interagere med dem inde i Unity. Dette er vigtigt hvis vi for eksempel som ved "projectile" og "enemy" -variablerne, skal indsætte et gameobject.

Gameobject er en variabel som er knyttet til Unity. Alle variablerne som ellers er knyttet til Unity har fået en grøn farve på billedet til højre.

Variablerne hvor der står static foran, bliver sat til static for at man skal kunne benytte disse variabler i andre scripts.

```
1 public static float bulletdamage = 20f;
2 public float speed;
3 public float stoppingDistance;
4 public float retreatDistance;
5 public float seeDistance = 10;
6
7 private float timeBetweenShots;
8 public float startTimeBetweenShots;
9
10 public GameObject projectile;
11 public Transform player;
12 public GameObject enemy;
13
14 public int maxHealth = 40;
15 public int currentHealth;
16
17 private bool GameIsStarted;
18
19 private float timeLeft = 0;
20
21 public Transform firePoint;
22
23 static AudioSource spiderDamageAudio;
24
25 private bool PlayerFound;
```

Når man laver scripts til et gameobject i Unity, bruger man som regel en "start-funktion".

I denne start-funktion køres der en smule variabelkontrol.

Først sættes fjendens max-liv til dens nuværende liv. Herefter angives tidsrummet mellem hvert skud fra

fjenden. Spilleren er endnu ikke blevet lavet når fjenden er, og derfor sættes variabelen der tjekker for om spilleren er fundet, til "false". Til sidst hentes fjendens lydkomponent.

```
1 void Start()
2 {
3     currentHealth = maxHealth;
4
5     timeBetweenShots = startTimeBetweenShots;
6
7     PlayerFound = false;
8
9     spiderDamageAudio = GetComponent<AudioSource>();
10 }
```

Den næste funktion der køres er en update funktion. Update-funktionen i dette script, er blot sat til at opdatere en gang hver frame. Dette har vi valgt, da tiden ikke spiller en stor rolle i forhold til dette script. ...

---

<sup>5</sup> Calice, Noa, SHOOTING/FOLLOW/RETREAT ENEMY AI, 19/3 - 21

```
1 void Update()
2 {
3
4     if (PlayerFound != true)
5     {
6         player = GameObject.FindGameObjectWithTag("Player").
transform;
7         GameIsStarted = true;
8     }
9
10    if (timeLeft >= 0)
11    {
12        timeLeft -= Time.deltaTime;
13    }
14
```

I starten af update-funktionen, tjekkes der for om spilleren ikke er blevet fundet. Hvis spilleren ikke er blevet fundet af fjende-scriptet, så skal den prøve at søge efter et gameobject med "Player-tagget", som spillerens gameobject besidder. Derudover skal den, når spilleren findes, sætte "GameIsStarted"-variablen lig med "true". Herefter tjekkes det om timeLeft er større end, eller lig med 0. Hvis den er det, så trækkes "Time.deltaTime", fra "timeLeft".

```
1     if (GameIsStarted)
2     {
3         if (GameObject.FindGameObjectWithTag("Player") != null)
4         {
5             if (timeLeft <= 0)
6             {
7                 if (Vector2.Distance(transform.position, player.
position) < seeDistance)
8                 {
```

Nu skal vi til at programmere fjendens angreb. For at gøre dette, tjekkes der først for en række ting der skal være i orden, inden fjenden kan angribe spilleren. Der tjekkes for om spillet er startet, at der findes et gameobject med tagget "Player", at timeLeft er mindre end eller lig med 0 og at spilleren er inden for fjendens angrebsrækkevidde.

```
1 if (Vector2.Distance(transform.position, player.position) > stoppingDistance)
2 {
3     transform.position = Vector2.MoveTowards(transform.position, player.position, speed * Time.deltaTime);
4     RotateTowards(player.position);
5
6 }
```

Hvis spilleren så er for langt væk til at fjenden kan begynde at skyde, vil fjenden så for hver gang vi kører update-funktionen, danne en 2D-vektor der går fra fjendens position til spillerens position. Altså vil fjenden rykke sig hen mod spilleren. Derudover køres en funktion kaldet "RotateTowards", med spillerens position som mål. Denne funktion vil vi gennemgå senere i dette stykke.

```
1      if (timeBetweenShots <= 0)
2      {
3          //Instantiate(projectile, transform.position, Quaternion.identity);
4          Instantiate(projectile, firePoint.position, firePoint.rotation);
5          timeBetweenShots = startTimeBetweenShots;
6      }
7      else
8      {
9          timeBetweenShots -= Time.deltaTime;
10     }
```

Hvis variablen der tjekker for hvornår fjenden skal skyde er under eller lig med nul sendes et skud fra et "firepoint", som er placeret lige foran gameobjectets front. Skuddet bliver blot instantiatet, som betyder at man henter skuddet som et prefab indsætter der i spillets nuværende instans. Herefter sættes tiden tilbage så fjenden ikke skyder med det samme igen.

```
1      private void RotateTowards(Vector2 target)
2      {
3          var offset = -90f;
4          Vector2 direction = target - (Vector2)transform.position;
5          direction.Normalize();
6          float angle = Mathf.Atan2(direction.y, direction.x) * Mathf.Rad2Deg;
7          transform.rotation = Quaternion.Euler(Vector3.forward * (angle + offset));
8      }
```

Først sætter vi et offset til 90 float (grader), herefter laver vi en vector ud fra to koordinater. Fjenden og spillerens position. Herefter benyttes Normalize-funktionen, som får vektoren til at have magnitude af én. Herefter defineres vinklen ved at tage Atan2-funktionen til vektorens x og y -koordinat. "Mathf.Rad2Deg" sørger for at ændre "angle", fra radianer til grader. Til sidst roteres fjenden så i forhold til den fundne vinkel og vores offset.

Fjenden skal også kunne blive angrebet af spilleren. For at gøre så spilleren kan gøre skade på fjenden, oprette vi her funktionen "TakeDamage". Denne funktion trækker blot en variabel fra fjendens nuværende liv og afspiller en lydfil. Funktionen er lavet således at hvis fjenden skulle tage 20 skade, skulle man blot skrive "TakeDamage(20)".

```
1      public void TakeDamage(int damage)
2      {
3          spiderDamageAudio.Play(0);
4          currentHealth -= damage;
5      }
```

```
1 void OnCollisionEnter2D(Collision2D collision)
2 {
3     if (collision.gameObject.name == "FireBullet(Clone)")
4     {
5         TakeDamage(FireSpell.BulletDamage);
6         if (currentHealth <= 0)
7         {
8             Destroy(gameObject);
9         }
10    }
```

For at skaden fra spilleren kan tildeles fjenden, må vi tjekke for hvornår fjenden collider med et af spillerens projektiler (Spells). Vi har i dette script, oprettet 4 if statements der tjekker for kollision med de 4 projektiler, enkeltvis. Hvis fjenden så tager skade, bruger vi blot "TakeDamage"-funktionen med en skade-variabel, der bliver hentet fra et script på projektilet. Herefter tjekkes der for om fjendens nuværende liv har ramt 0 eller lavere. Hvis den har det, så fjernes fjenden som gameobject, med kommandoen "Destroy()".

Resultatet kan ses på billedet nedenunder hvor der er en fjende i form af en edderkop, som har vendt sig mod spilleren og skyder et projektil afsted mod spilleren.





## Quest

I vores spil vil målet være at klare en quest, altså i bund og grund løse en opgave. Derudover, så er brugen af quests typisk brugt indenfor fantasy-genren, som er den æstetiske genre vi prøver at ramme. Vi vil i vores udgave af spillet kun programmere én quest. Denne quest vil både indeholde en start hvor man kan lære at befærde sig i spillet, men den vil samtidigt også indeholder meget svære elementer for at presse achievers.

Spilleren modtager questen fra en bondemand, der har nogle problemer med en af sine plantager. Denne plantage er nemlig blevet overtaget af en masse edderkopper. Bondemanden mener at en stor magisk slange har bragt edderkopperne til plantagen, og sender derfor spilleren ud for at dræbe denne slange og de mange edderkopper.

Det første script er derfor et script vi sætter på denne bondemand-npc(Non-player character).

Til at starte med, definerer vi en del variabler. Først og fremmest laver vi en variabel "q1", og sætter dem til "false" som standard. Denne variabel skal fortælle om questen er startet. Derudover henter vi 4 audioclips samt deres tilhørende audiosources. Vi skal bruge begge for at afspille lyd fra scriptet. Bondemanden skal nemlig sige et eller andet når man kommer hen til ham. Bondemanden får defineret en "seeDistance" der skal bruges til at tjekke for hvornår spilleren er indenfor rækkevidde af bondemanden. Vi henter også spillerens position som variabel. Vi definere et boolsk udtryk, der skal sige om spillet er startet eller ej. Vi henter "gameobjectet" der indeholder det grafiske der skal poppe frem når bondemanden skal tildele spilleren sin quest. Til sidst defineres et boolsk udtryk der fortæller om spilleren er tæt nok på bondemanden til at tage imod questen.

```
1 public class NPCQ1 : MonoBehaviour
2 {
3     public static bool q1 = false;
4
5     public AudioClip Farmer1;
6     public AudioClip Farmer2;
7     public AudioClip Farmer3;
8     public AudioClip Farmer4;
9     AudioSource audioData1;
10    AudioSource audioData2;
11    AudioSource audioData3;
12    AudioSource audioData4;
13
14    private int seeDistance = 5;
15    public Transform player;
16    private bool GameIsStarted;
17    public GameObject Q1Wrapper;
18
19    public bool PlayerCloseEnough;
```

```
1 void Update()
2 {
3     if (GameIsStarted)
4     {
5         if (GameObject.FindGameObjectWithTag("Player") != null)
6         {
7             if (Vector2.Distance(transform.position, player.position) < seeDistance)
8             {
9                 if (PlayerCloseEnough == true)
```

Inde i vores update funktion tjekker vi for en del ting inden vi kan vise spilleren de grafiske elementer af questen. Først og fremmest så skal spillet være startet, der skal eksistere et gameobject med tagget "Player", distancen fra bondemand til spilleren skal være mindre end "seeDistance" og så skal variablen "PlayerCloseEnough" være sand.

```
1 public void GameStart()
2 {
3     player = GameObject.
FindGameObjectWithTag("Player").transform;
4     GameIsStarted = true;
5 }
```

```
1 if (PlayerCloseEnough == false)
2 {
3     Q1Wrapper.SetActive(false);
4     PlayerCloseEnough = true;
5 }
```

Variablen "GameIsStarted" bliver sat lig med true, når spillet starter. Variablen "PlayerCloseEnough" bliver sat lig med true hvis den stadig ikke er sand men man er indenfor rækkevidde af bondemanden.

Hvis alle disse ting så er sande kører nedenstående kode:

```
1 Q1Wrapper.SetActive(true);
```

Her sættes "Q1Wrapper"-gameobjektet aktivt. Dette betyder at grafikken fra questen indtil nu har været slået fra, men den sættes nu aktivt under disse forhold.

På denne grafik der kommer frem er der en knap der skal starte questen. Når man trykker på den knap kører følgende to funktioner:

```
1 public void SetQ1Active()
2 {
3     q1 = true;
4 }
5
```

```
1 public void GotoQ1()
2 {
3     SceneManager.LoadScene("Quest1");
4 }
```

Den ene funktion, sætter variablen "q1" til at være sand, og den anden sender spilleren over til en anden scene hvor questen udfolder.

```
1 void Start()
2 {
3     audioData1 = AddAudio(false, false, 1f);
4     audioData2 = AddAudio(false, false, 1f);
5     audioData3 = AddAudio(false, false, 1f);
6     audioData4 = AddAudio(false, false, 1f);
7 }
```

Når scriptet for bondemanden starter henter vi også fire audiofiler. En af disse audiofiler skal afspilles når man går hen til ham.

```
1 public void PlayFarmer1()
2 {
3     audioData1.clip = Farmer1;
4     audioData1.Play();
5 }
6 public void PlayFarmer2()
```

Vi definere også fire funktioner der når de bliver kaldt på, skal afspille deres tilsvarende lydclip.

Inde i samme "if"-statement hvor vi satte grafikken til sand kører vi så følgende kode:

Vi laver her en randomizer der finder et tal mellem 1 og 100. Herefter spiller vi så lydfil 1, hvis randomizeren er under eller lig med 30, hvis den er højere end mellem 30 og 61 spiller den lydfil 2, hvis den er mellem 60 og 91 spiller den lydfil 3 og hvis den så er over 90 spiller den lydfil 4.

Dette gør altså at vores bondemand vil sige en ud af fire ting når man går hen til ham for at modtage hans quest.

```
1 int SoundRandomizer = Random.Range(1, 100);
2 if (SoundRandomizer <= 30)
3 {
4     PlayFarmer1();
5 }
6 else if (SoundRandomizer > 30 && SoundRandomizer <= 60)
7 {
8     PlayFarmer2();
9 }
10 else if (SoundRandomizer > 60 && SoundRandomizer <= 90)
11 {
12     PlayFarmer3();
13 }
14 else if (SoundRandomizer > 90)
15 {
16     PlayFarmer4();
17 }
```

Resultatet kan ses på billederne nedenunder som viser at farmeren kommer med questen i det øverste højre hjørne.



## Multiplayer

### Hvad er Photon PUN?

PUN står for Photon Unity Network, og er en Unity package, lavet af firmaet Photon, som gør det muligt at udvikle flexible multiplayer-spil i Unity. Ved at oprette en bruger hos Photon, får man mulighed for at benytte deres servere helt gratis, hvilket gør at en bruger ikke skal stå for at holde en server kørende. Dette er smart, således at andre ikke er afhængig af én specifik spiller om holder serveren kørende. PUN er opbygget således, at spillere tilslutter sig lobbies/rum, når de skal spille sammen med andre. Dette gør at man kan have flere forskellige 'servere' oppe at køre, så alle ikke absolut skal spille sammen. Dette ses eksempelvis i spilklassikeren World of Warcraft, hvor du kan vælge forskellige verdener at spille i. Når man indlæser PUN i Unity, så medfølger en masse script, som indeholder alle de funktioner som udviklerne skal bruge til at udvikle multiplayer-spillet. Dette gør at udviklerne ikke selv skal starte helt fra bunden og sørge for den mindste detalje.<sup>6</sup>

### Hvorfor valgte vi dette?

Vi valgte at benytte PUN, da ingen af os havde erfaring med at udvikle spil som skulle sende og modtage data igennem en server. Da vores ambitioner med selve spillet var store, så frygtede vi at det ville tage for lang tid at skulle implementere multiplayer uden garanti for at det ville fungere. Vi havde yderligere fået anbefalet PUN fra andre bekendte, og vi fik læst os frem til at det var en intuitivt og godt værktøj til at få implementeret multiplayer i spillet. Derudover, så gav det også mulighed for at der ikke var en spiller som hele tiden skulle enten have en dedikeret server åben, eller at alle andre var afhængig af én spiller have spillet åbent.<sup>7</sup>

### Start eller tilslut et online session-room

Når brugeren tilslutter sig spillet, så vil vi gerne have at personen ikke har mulighed for at vælge en specifik lobby. Grunden til dette er, at vi gerne vil have samlet alle spillere i én samlet verden. Dette er selvfølgelig i taget i forhold til at der ikke er særlig mange aktive brugere i startfasen af spillet. Hvis antallet af spillere stiger, kunne der sagtens blive udvidet med mulighed for at vælge en bestemt server.

Det første vi gør, når vi skal have sat det op, er at lave et tomt gameobject i Unity. Dette gameobject skal ikke indeholde noget udover det script som skal lave lobbyen og få spilleren til at tilslutte sig lobbyen. Vi har valgt at give det tomme gameobject navnet Photon, da det indeholder det essentielle script med Photon forbindelsen. Nedenfor ses gameobjectet samt hvad det indeholder.

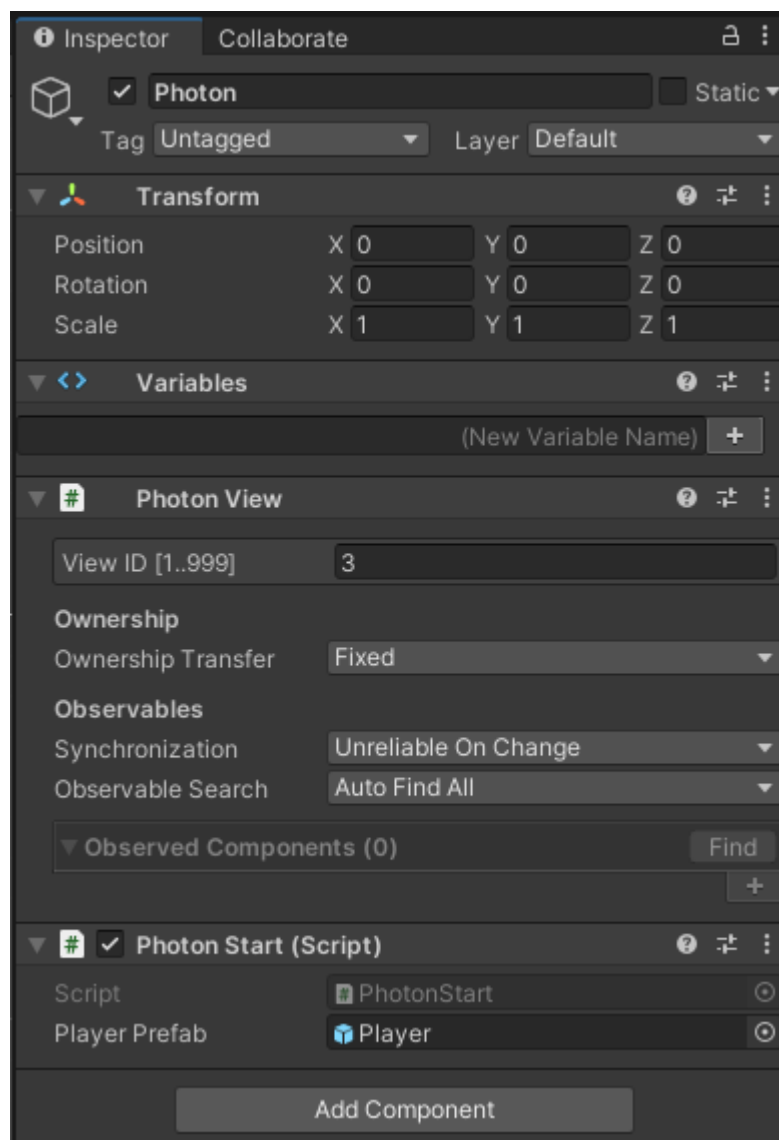
---

<sup>6</sup> Photon INC, Introduction to PUN/ Photon Unity.

<sup>7</sup> Photon INC, PUN.

Photon

Dette er sådan GameObjectet ser ud i scenen.

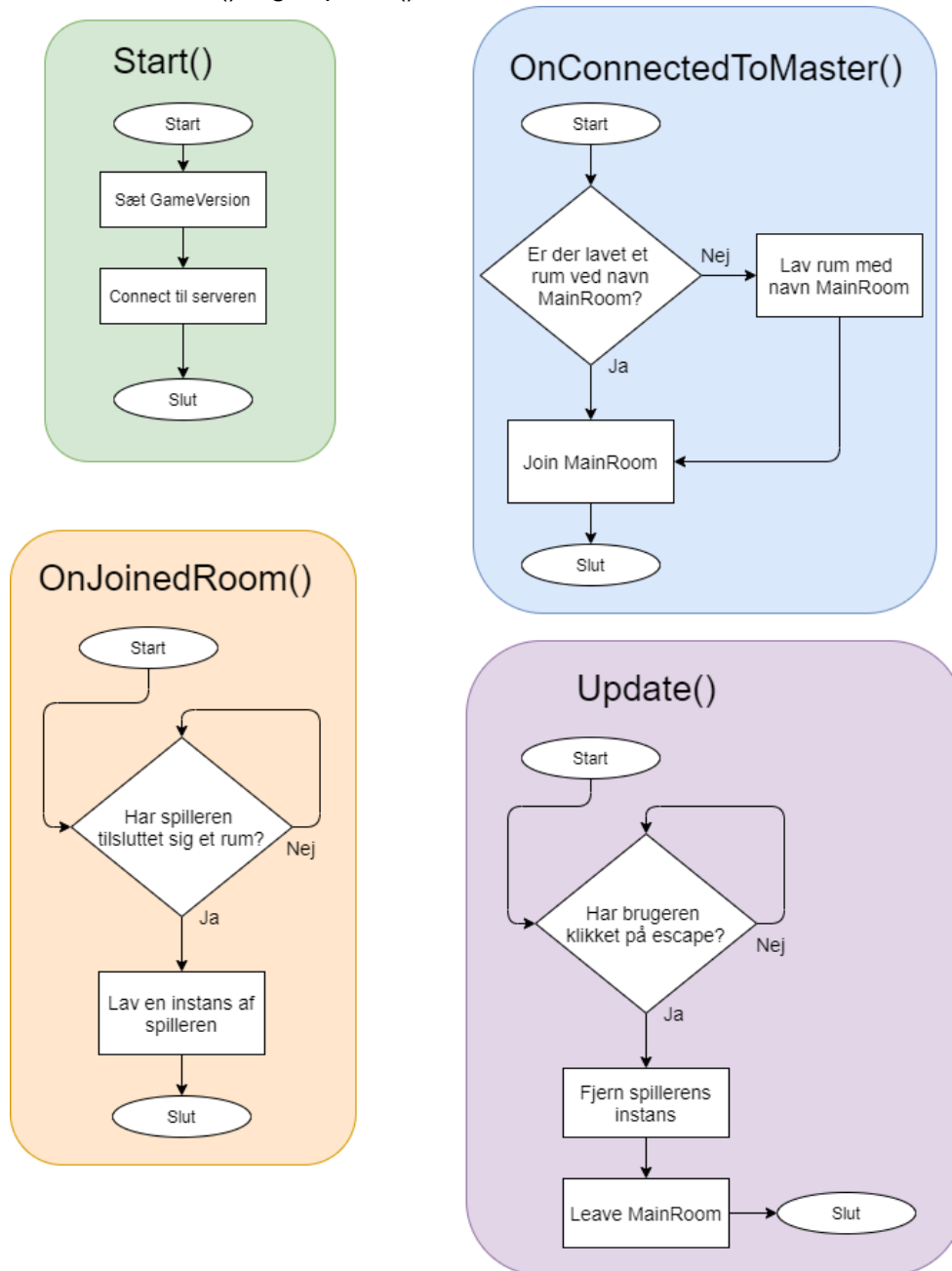


På det billedet til venstre, ses det at gameobjectet består af et PhotonView script og et Photon Start script.

Førstnævnte script er udviklet af Photon selv, og bruges til at holde styr på de elementer som skal bruges for hver spiller instans. Sagt på en anden måde, så er PhotonView det som forbinder de forskellige instanser på hver computer og definere hvilke komponenter som der skal observeres og hvordan de skal observeres. Inde i dette script, skal der hertil gives et id, som er unikt for hvert komponent dette script er på.

Det næste script er Photon Start, og er det script som indeholder selve forbindelsen og oprettelsen af lobbies. Her ses det at der bliver brugt et prefab fra spilleren. Hvordan det bruges bliver forklaret nedenunder, hvor der bliver beskrevet om opbygningen af scriptet.

Lige inden vi går i gang med at forklare koden, så har vi også lavet et flowchart til “PhotonStart”-scriptet. Scriptet består af “Start()”, “OnConnectedToMaster()”, “OnJoinedRoom()” og “Update()”.



Inde i “Start()” sættes spillets aktuelle version, hvorefter den tilslutter sig til serveren. Når spilleren er tilsluttet serveren, så kører funktionen “OnConnectedToMaster()”. Inde i denne funktion tjekker den om rummet ‘MainRoom’ eksisterer. Hvis den gør dette, så tilslutter den brugeren til dette rum. Hvis rummet ikke eksisterer, så laver den i stedet rummet hvorefter den så tilslutter brugeren til rummet. Når brugeren har tilsluttet sig et rum så kører den næste funktion “OnJoinedRoom()”. I denne funktion tjekker den først om spilleren har tilsluttet sig et rum. Hvis brugeren har det, så laver den en instans af spilleren. I mellemtiden kører “Update()” hele tiden. Inde i denne funktion tjekkes der hele tiden for om spilleren har trykket på escape. Hvis brugeren har det, så fjerner den spillerens instans og kobler sig fra rummet.



Nu vil vi forklare hvordan vi har programmeret "PhotonStart"-scriptet.

```
1 using Photon.Pun;  
2 using Photon.Realtime;
```

Som det absolut første, så fortæller vi i scriptet, at vi skal til at benytte os af nogle funktioner som er blevet importeret fra Photon. Photon.Realtime bruges til at forbinde til Photon serveren, imens at Photon.Pun bruges til lave eller joine en lobby samt lave spilleren når personen tilslutter sig en lobby.

```
1 public class PhotonStart : MonoBehaviour  
2 {  
3  
4     public GameObject playerPrefab;
```

Herefter laver vi et public gameobject med navnet playerPrefab, så vi kan tilgå det fra inspector menuen. Dette betyder at vi kan trække et gameobject over på variabelen i stedet for manuelt at søge efter det.

```
1 void Start()  
2 {  
3     PhotonNetwork.GameVersion = "1.0";  
4     PhotonNetwork.ConnectUsingSettings();  
5 }
```

Herefter så bliver der kørt en start funktion som initialiserer en spilversionen. Denne variabel er ret smart, da den gør at en gammel version af spillet ikke kan forbinde til en lobby med en nyere version af spillet. Dette kan eksempelvis bruges, hvis man har fundet en fejl i sit spil som man får rettet hurtigt. Så kan man opdatere versionen, således dem med dem med fejlen i spillet ikke kan spille sammen med dem som har opdateret spillet. Linjen nedenunder forbinder dernæst til serveren.

```
1 public void OnConnectedToMaster()  
2 {  
3     PhotonNetwork.JoinOrCreateRoom("MainRoom", new RoomOptions() {MaxPlayers = 5 }, null);  
4 }
```

Herefter laver vi en funktion som kører når brugeren er blevet connected til serveren, hvilket også passer fint til navnet af funktionen. Inde i denne funktion bliver det benyttet en funktion som tilhører Photon. Denne funktion gør at man laver en lobby ved navn MainRoom, som

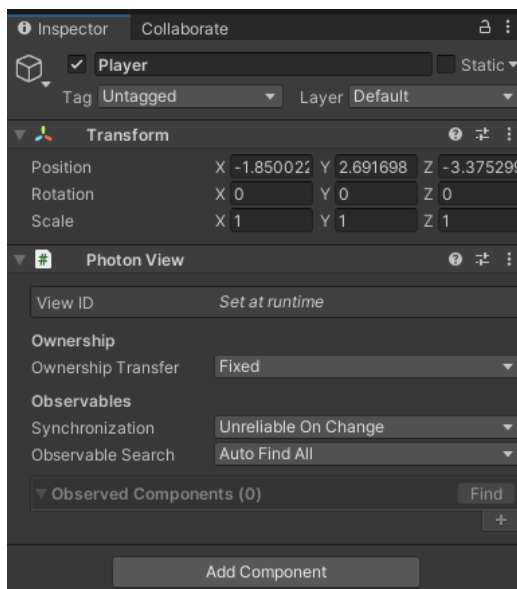
har 5 spillere som maksimum. Herefter så tilslutter den spilleren til den lobby som lige er blevet lavet. Da vi benytter deres gratis udgave, så tillader de også kun op til 5 spillere pr. lobby.

Hvis der allerede findes en lobby ved navn MainRoom, så tilslutter den spilleren til denne lobby i stedet for at lave to lobbies med samme navn.

```
1 public void OnJoinedRoom()
2 {
3     Debug.Log("joined room");
4     PhotonNetwork.Instantiate(playerPrefab.name, new Vector3(0f,5f,0f), Quaternion.identity, 0);
5 }
```

Den næste funktion tjekker for om brugeren har joinet et rum. Det vil altså sige, at den kører lige efter ovenstående funktion, hvor man har tilsluttet sig en lobby.

Inde i funktionen har vi en debug linje, hvor vi har testet om scriptet kørte denne funktion, når spilleren tilsluttede sig et rum. Herefter så opretter den spilleren. Koden er næsten den samme som hvis man lavede spilleren i singleplayer. Den eneste forskel er at PhotonNetwork er sat foran, da spilleren også skal laves på de andres skærme.



Da spilleren jo derfor skal vises på de andres skærme, så skal Photon View også indsættes på spillerens prefab.

```
1 public void Update()
2 {
3     if (Input.GetKeyDown(KeyCode.Escape))
4     {
5         Destroy(playerPrefab);
6         PhotonNetwork.LeaveRoom();
7     }
8 }
```

Til sidst så tjekkes det om brugeren klikker på escape. Dette bliver tjekket for hvert frame, da det er inde i en update funktion. Hvis brugeren klikker på escape, så fjerner den spilleren og

kobler af lobbyen. Det er vigtigt at brugeren kobler af lobbyen, da der ellers kan ske den fejl, at den samme bruger forbinder til serveren 2 gange. Vi har senere hen fundet ud af, at hvis man lukker spillet manuelt, så kan Photon godt selv finde ud af at koble spilleren fra. Dog så er det en god idé stadig at have, hvis brugeren nu kun vil ud i menuen for at ændre farve, monster eller brugernavn.

## Styring af spillerens bevægelser

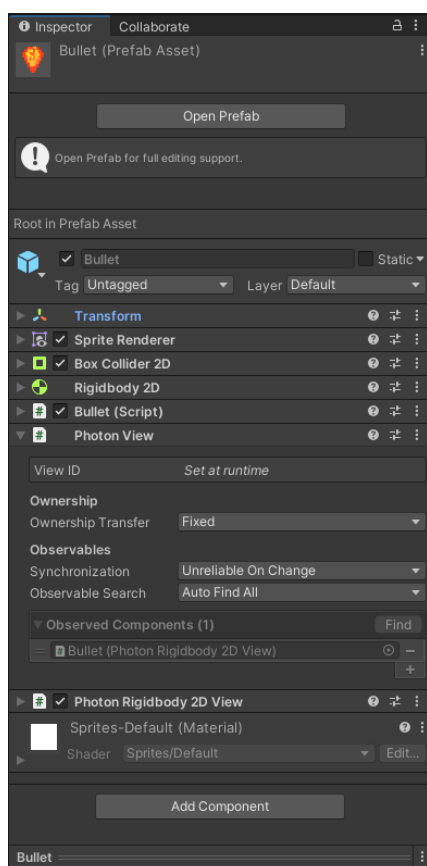
Når man arbejder med multiplayer, så opstår der nogle kritiske aspekter når man skal bevæge en spiller. Dette skyldes at det samme prefab af spilleren bliver lavet for alle brugere, men hver bruger skal kun styre deres egen spiller. For at løse dette problem, skal vi altså finde en måde hvorpå vi fortæller programmet, at brugeren kun skal bevæge den spiller som personen har valgt. Til dette bruges et stykke kode fra Photon som hedder "IsMine". Navnet er ikke tilfældigt, fordi denne boolean variabel tjekker for om spilleren er ejet/lavet af brugeren.

```
1  void Update()
2  {
3      if (photonView.IsMine)
4      {
5          movement.x = Input.GetAxisRaw("Horizontal");
6          movement.y = Input.GetAxisRaw("Vertical");
7
8
9          movement = movement.normalized;
10
11         //mousePos = cam.ScreenToWorldPoint(Input.mousePosition);
12     }
13 }
```

Måden vi har benyttet denne variabel, er ved at tjekke for om brugeren er ejet/lavet af brugeren, og hvis den er det, så får scriptet lov til at udfører det givne kode. Dette er præcist sådan vi gør når vi bevæger spilleren og når vi skal skyde vores ildkugle afsted. I scriptet ovenfor ses det at vi bruger photonView.IsMine, når vi henter noget input data til vores player movement script.

```
1  void Update()
2  {
3      if (photonView.IsMine)
4      {
5          if (Input.GetKey(KeyCode.Space) && Time.time > autoNextFireTime)
6          {
7              Shoot();
8              autoNextFireTime = Time.time + autoCooldownTime;
9          }
10     }
11 }
```

Det kan ligeledes ses i vores script hvor vi tjekker for om spilleren skyder. Her bliver der også tjekket for om spilleren er ejet af brugeren. Hvis dette udsagn er sandt, så tjekkes der for om der bliver trykket på skydeknappen.



Da der bliver skudt et skud afsted, så er det også vigtigt, at skuddet som prefab har et PhotonView script på, da skuddet skal vises for alle brugere, og ikke kun brugeren som skyder skuddet afsted. Dette kan ses på det nedenstående billede. Det vil sige, at hver gang der bliver affyret et skud, så får det skud tildelt et specifikt id som er ens for alle brugere. Det vil sige at det første skud får id'et 1, andet skud får id'et 2 og så videre. Id'erne bliver ikke nulstillet før at alle fra lobbyen er koblet af.

## Var PUN det rigtige valg?

Da vi skulle finde ud af hvordan vi skulle implementere en form for multiplayer, synes vi at Photon PUN var en rigtig god og oplagt mulighed, da man nemt kunne komme i gang uden den store erfaring indenfor multiplayer udvikling. Derfor satte vi os også for at benytte dette med det samme, da andre bekendte havde fortalt os at de ligeledes brugte det og synes det var godt. Vores oplevelse med det var også meget god til at starte med. Vi fandt dog meget hurtigt ud af, at der faktisk var en masse begrænsninger og ulemper som fulgte med ved at bruge det. Da vores spil afviger fra det eksempel som er i dokumentationen for funktionerne og lignende, så blev det hurtigt svært at skulle finde frem til en løsning til de problemer vi fik. Eksempelvis, så fik vi problemer, da vi skulle vise spillernes forskellige farver som brugerne havde valgt ude i menuen. Da vi prøvede at finde løsninger på internettet, så var hjælpen minimal, da der ikke er særlig mange der har spurgt om lignende problemer. Derfor var vi efterladt meget til os selv, og brugte timevis på at læse forskellige dokumentationer og prøve forskellige løsninger. Efter at have brugt utallige timer på at finde løsninger, gik det også op

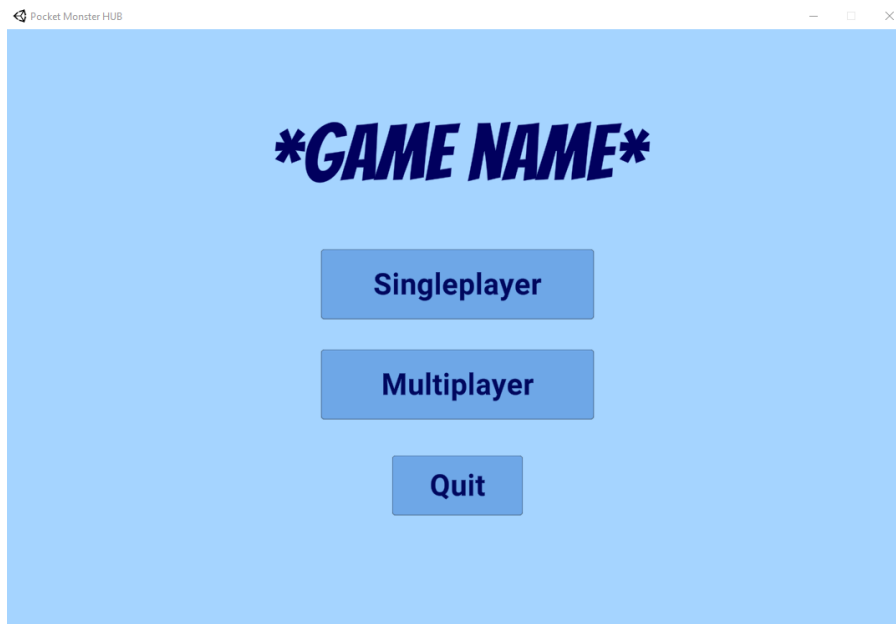
for os, at vi ikke rigtig forstår hvad der sker inde i de funktioner vi bruger. Vi ved godt hvordan vi skal bruge dem, men vi ved ikke specifikt hvordan den forbinder til Photon servere, eller hvordan den tjekker om hvorvidt spilleren er ejet af den rigtige bruger. Ting som dette er der heller ej dokumentation på. Dette var også grunden til at vi valgte at stoppe på den udviklingen af multiplayerdelen, og fokuserer mere på singleplayer, hvor vi kunne udvikle alle de funktionaliteter som vi gerne vil have med. Når vi ser tilbage på det nu, så skulle vi have brugt meget mere tid på at have set på andre alternativer og have undersøgt disse nærmere. Eksempelvis kunne vi have brugt socket i Unity. Dette er en smule mere low-level programmering, men det havde også resulteret i at vi ville have mere styr på hvordan man lavede en server og sendte data frem og tilbage.

For vores vedkommende, så var PUN ikke det optimale valg til at lave et multiplayer spil. Vi fik godt nok lavet et spil hvori man kunne tilslutte sig, bevæge sig og skyde skud sammen med andre. Hvis tidshorisonten havde været på vores side, så ville vi muligvis haft mulighed for at kunne løse de problemer vi stødte på. Dog så synes vi stadig at PUN var for overfladisk til at vi kunne forklare hvordan det low-level hang sammen. Netop dette satte også begrænsningerne for hvor langt vi kunne komme. Dog fik vi stadig lavet et multiplayer spil som vi ved hvordan fungerer, og det fungerer godt når man bevæger sig rundt og skyder.

# Produktfremvisning

I dette afsnit vil vi gerne præsentere vores endelige produkt. Det endelige produkt består af 3 dele. En multiplayer version af spillet, en singleplayer version af spillet og en Game hub menu, hvor brugeren kan vælge mellem at spille multiplayer eller singleplayer.

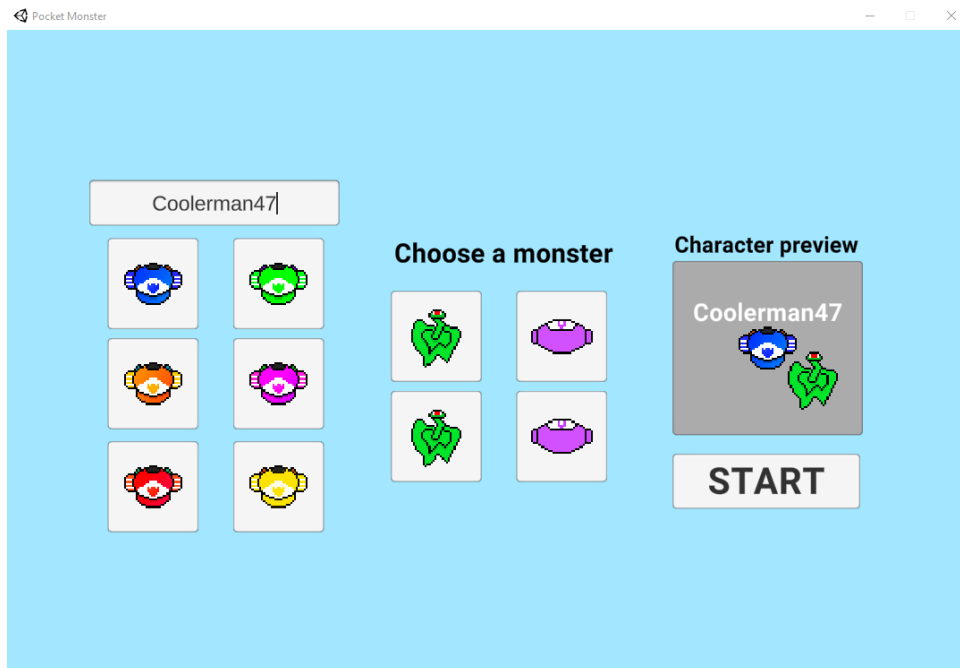
## Game hub



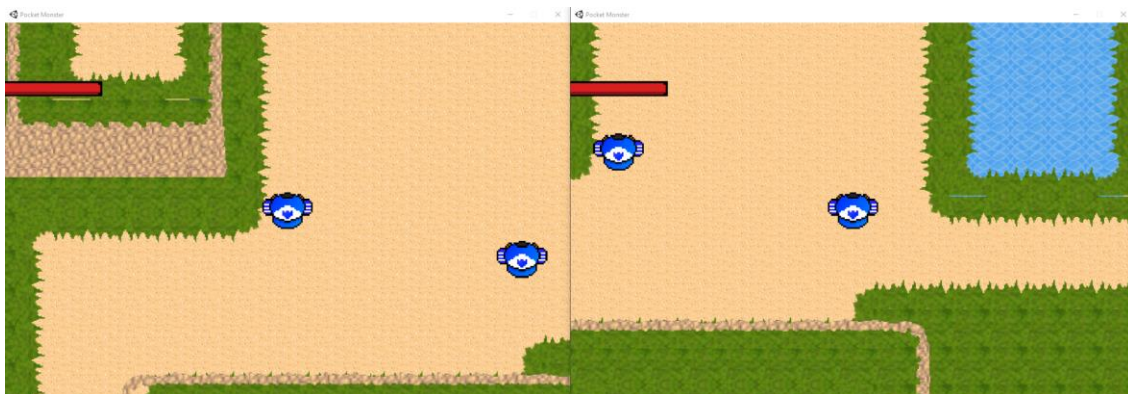
Når man starter vores spil, så bliver man mødt af denne menu. Denne menu er ikke en direkte aktiv del af vores spil, hvilket også er grunden til at vi ikke har dokumenteret udviklingen denne. Den bruges til at give brugere end mere flydende oplevelse, hvis de vil skifte imellem singleplayer versionen og multiplayer versionen. Som knapperne hentyder, så starter hubben den version af spillet der er blevet valgt. Det vil sige at singleplayer-knappen åbner singleplayerspillet, og multiplayer-knappen åbner multiplayerspillet. Quit-knappen lukker hubben ned. Da vores kreativitet ikke rækker til et ordentligt navn til spillet, er \*game name\* blevet sat som et form for placeholder, for at vise at spillets navn skulle være der.



# Multiplayer



Den første scene som loades er selektionsmenuen, hvor man kan tilpasse sin bruger.

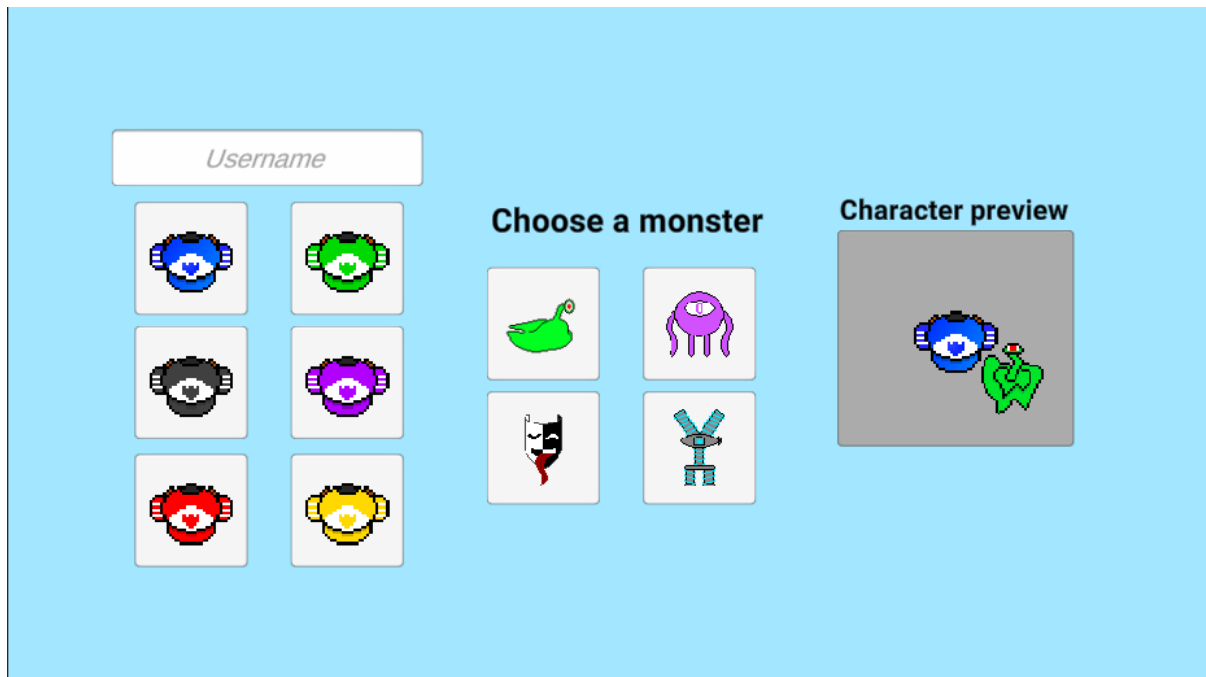


Når man klikker start, forbinder man til serveren og lobbyen, og herefter kan man se de eventuelle andre spillere. Her kan man gå rundt blandt de andre spillere.



Man kan ligeledes også skyde skud fra sig, som de andre spillere kan se. Dette er såmænd alt der lige pt. er muligt i vores multiplayer del af spillet.

## Singleplayer



Den scene der loades, når man starter singleplayer-versionen af vores spil, er denne selektionsmenu. Her kan man vælge navn, farve og monster.



Starter man spillet kommer man herefter ind til en startbane, hvor der er et skilt der guider spilleren til at komme igang



Går man ned og tager questen, kommer man til questbanen:



Her bliver man mødt af to skilte der forklare en omkring kampsystemet og hvordan man kan miste og får liv. Derudover henviser den også til man skal kigge i sin spellbook for at lære mere omkring sine spells.



Spellbooks ser ud således og indeholder en række information om de forskellige spells. Den kan åbnes og lukkes på knappen (p)

For at tjekke ens egentlige mål, altså questens mål, så tjekker man blot sin questlog:



Her vil der være et nemt og overskueligt overblik over hvad spilleren skal foretage sig, for at komme gennem questen.



Nu er spilleren klar til at udforske den quest-verden og kæmpe sig til sit mål.





Den første bane man loader in på, ser ud som på ovenstående billede. Det er blot et hyggeligt lille landskab med et skilt der byder dig velkommen og en bondemand, der gerne vil sende spilleren ud på en quest.



Questbanen i sin fulde form, ser ud som på ovenstående billede. Målet her, er blot at komme ned og dræbe slangen i højre hjørne.

# Spilanalyse, med MDA framework og Bartles spillertyper

Testen tager udgangspunkt i både vores singleplayerversion og multiplayerversion, da de begge har vigtige elementer i forhold til spilanalysen.

Med denne analyse vil vi komme ind på vores målsætninger for hvilke Aestheticselementer vi havde planlagt for vores spil. Vi vil med hjælp af strukturen fra MDA framework udarbejde hvilke af de 8 punkter af Aesthetics, opgivet af Robin Hunicke, Marc LeBlanc og Robert Zubek i udskriften "MDA: A Formal Approach to Game Design and Game Research", vores spil udfylder. Derefter uddybe til hvilke Dynamics og Mechanics der har ledt til disse Aesthetics, og til sidst lave et estimat af hvilke af Bartles spillertyper de appellerer til.

De 8 punkter af Aesthetics præsenteret i "MDA: A Formal Approach to Game Design and Game Research" er;

1. Sensation: Game as sense-pleasure,
2. Fantasy: Game as make-believe,
3. Narrative: Game as drama,
4. Challenge: Game as obstacle course,
5. Fellowship: Game as social framework,
6. Discovery: Game as uncharted territory,
7. Expression: Game as self-discovery,
8. Submission: Game as pastime,

Vi har vurderet vores spil til tildeles at udfylde 6 af de ovenstående elementer i en større eller mindre grad.

Det første Aesthetics element, Sensation, appellerer til behagelig følelsesmæssige forbindelser til spillet. Denne Aesthetic bliver i vores spil, blandt andet, tilvejebragt af en række Dynamics, som den behagelig stimulerende musik der i sammenspil med de lyse farvetoner af banen gør det til en rolig og behagelig oplevelse. Som kort nævnt bliver disse Dynamics skabt i samspillet mellem de to Mechanics, 'audio' og 'visual output', fra spillet. Disse valg af Mechanics består af; at spilleren og monstrene er tegnet med pastelfarver, at banen hovedsagelig består af lyse toner og farver, og at musikken i baggrunden er rolig og stimulerende. Vi, som udviklere, forsøger at lave en Dynamic mellem de to sanseinput der vil resultere i en Sensation Aesthetic, der gør det til en behagelig oplevelse for brugeren og giver dem lyst til at fortsætter eller vende tilbage.

Den næste Aesthetics præsenteret i "MDA: A Formal Approach to Game Design and Game Research" vi har prøvet tildeles at udfylde er Fantasy. Fantasy bygger på at spillet foregår i en imaginær verden hvor brugeren bliver interesseret og suget ind i denne nye verden. Trods at vi ikke når helt til ende med at præsentere brugeren for den fuldt udviklede lore for denne nye verden, formår vi dog stadig at implementere de mest basale Mechanics der lægger bunden for brugerens indsættelse i vores fantasyverden. Vi implementerede den Mechanic at playeren ikke kunne spawnes uden sit monster som i loreen er basen for spillerens magi, ydermere gav vi spilleren evnen til at bruger spells og magi. Dynamikken mellem disse Mechanics er sammen med det faktum at det hele foregår i en verden hvor



dine modstander er dyr vi kender fra vores egen verden men på størrelse med lastbiler og med evnen til at bruge deres egen magi er med til at indsætte brugeren i den imaginære verden der bliver skabt i spillet, resulterende i at spillet for en Fantasy Aesthetic. Denne Aesthetic opfordre brugeren til undersøg verdenen og lore for at finde yderligere information og appellere på den måde meget til Bartles spillertype 'Explorers'.

Den tredje Aesthetic vi har prøvet at opnå, fra "MDA: A Formal Approach to Game Design and Game Research", er 'Narrative'. Dette er endnu en af de Aesthetics vi allerede i spillets nuværende stadie har opnået til dels, men som der er planen at udvide i endnu større grad i en potentiel videreudvikling. Narrative skabes når brugeren bliver sat ind i, eller udlever en allerede forfattet historie eller lore. Dette kan gøre spilleren interesseret i spillet, med et ønske om at blive ved med at spille, eller at komme tilbage til spillet, for at komme videre i historien. Dette element af spillet prøver vi at få inkorporeret gennem vores quest system. Vi har tilføjet den Mechanic til vores questgiver 'Farmerdude'. Når han giver den første quest, gives ikke kun objektet for playerens quest, men tilkobler en indlevende historie. Dette er den første implementeringen af Mechanics med henblik på Narrative. Denne meget simple Mechanic i sammensætning med questens objekt giver en Dynamic hvor brugeren indlever, og udlever en fortælling eller historie. Den Dynamic giver brugeren en Narrative Aesthetic, som ved fremtidige opdateringer er lagt op til at kunne gøres større ved at implementere flere quests og sætte dem ind i en større og mere kompleks historie.

Det fjerde Aesthetic element fra "MDA: A Formal Approach to Game Design and Game Research" vi har tænkt ind i vores spil er Discovery. Discovery er blot muligheden spilleren har for at udforske uopdaget territorier og områder. Vi opsætter to Mechanics der er endeligt hovedansvarlige for at for denne Aesthetic. Den ene Mechanic den frie bevægelse, vores spil er til dels et open world spil der giver brugeren mulighed for at gå hvorhen de lyster uanset hvilke quests. Den anden er mere fravalget af en Mechanic. Vi har ikke implementeret noget map i spillet hvilke opfordre, og til dels tvinger, brugeren til at gå på opdagelse rundt på mappet, om ikke andet så for at løse sin quest. Så valget omkring disse Mechanics giver en Dynamic hvor brugeren har mulighed og formål ved at udforske mappet, givende den Aesthetic Discovery. Denne Aesthetic opfordre og tvinger brugeren til at udforske spillet og appellere dermed kraftigt til Bartles spillertyper der hedder 'Explorers'.

Nu er vi så kommet til en af de lidt mindre fremtrædende Aesthetics, expression. Expression er brugerens mulighed for at kunne tilpasse sit udstyr eller udseende. Den eneste Mechanic vi har tiltænkt denne Aesthetic er i oprettelsen af playeren, hvor brugeren får mulighed for selv at vælge farve på playerens skin, samt hvilket monster man vil have som sin ledsager. Dette giver en Dynamic hvor brugerne kan udtrykke sig i de farver og former man kunne ønske sig. Dette giver os den Aesthetic fra "MDA: A Formal Approach to Game Design and Game Research" Expression.

En anden Aesthetic som på nuværende tidspunkt ikke er så fremtrædende er, Fellowship. Selv om denne Aesthetic ikke er så markant endnu, er det bestemt i planerne for de fremtidige opdateringer af spillet at det sociale aspekt vil vokse med den videre implementering af multiplayer. Fellowship handler om at få brugeren til at se spillet som en social ting. Dette opnår vi til dels gennem vores multiplayer Mechanic, som trods dens begrænsninger tillader flere bruger at spille sammen samt skabe en Dynamic hvor man som bruger kan samles med sine venner og i fremtiden kunne komme til at løse sine quests



sammen. Dette samlet vil give den ønskede Aesthetic Fellowship. Med alle disse sociale aspekter er det tydeligt at denne Aesthetic, hovedsageligt appellere til Bartles spillertype Socializers.

Challenge er den sidste af de Aesthetics fra "MDA: A Formal Approach to Game Design and Game Research" vi har opfyldt i vores spil. Til gengæld er det også et af de mest essentielle elementer i spillet da det hovedsageligt er det der på nuværende tidspunkt får brugeren til at blive ved med at spille, til de som har klaret deres quest. Challenge er at fremstille spillet som en bane med forhindringer som skal gøre det udfordrende for spilleren at nå til banens ende, eller i dette tilfælde klare sin quest. Der er en lang række Mechanics der er med til at underbygge denne Aesthetic. De meste essentielle af dem er Mechanics såsom, Enemies med deres evne til at kæmpe igen mod playeren, at playeren har forskellige spells til at håndtere Enemies, Quest systemet der skal sætte et mål, og et endeligt mål, på spillerens færd gennem mappet. Alle disse Mechanics, og flere til, er med til at give spillet et blancheret Gameplay og den Dynamic der leder det til en Challenger Aesthetic. Denne kamp men Enemies i målet om at løse sin quest, kan appellere to af Bartles spillertyper; killers og achievers. Dette aspekt appellerer tildeles til killers da de får muligheden for at slagte sig vej gennem enemies for at nå til målet. Vi mener dog at det i endnu højere grad appellerer til achievers da vi på nuværende tidspunkt har en enkelt quest, og den er ikke nem at løse hvilket vil give achievers mulighed for at prøve igen og igen på at klarer bossen indtil det endelig lykkes og de kan få muligheden for at prale med at de klarede den før deres venner.

Dette betyder at vores spil indeholder en lang række af forskellige Aesthetics, som resultat af de Mechanics, og dermed Dynamics, vi har lavet. Hvad vi har kunne se ud fra vores analyse, appellerer spillet hovedsageligt til Explores og Achievers fra Bartles spillertyper. Selvom at man godt kan argumentere for at der også er aspekter der vil appellere til typer såsom killers, er de to førnævnte langt mere fremtrædende, og vil i en eventuel markedsførings situation være dem der var tiltænkt som den primære målgruppe<sup>8</sup>.

---

<sup>8</sup> Hunicke Robin, LeBlanc Marc og Zubek Robert, "MDA: A Formal Approach to Game Design and Game Research", <http://www.cs.northwestern.edu/~hunicke/pubs/MDA.pdf>

# Tænke-højt test & spørgeskema

Testen tager udgangspunkt i vores singleplayer version, da denne version er tættest på vores intentioner med det endelige produkt.

Når vi har udviklet vores spil, har vi gjort det efter hvad vi synes gav mening. Selvfølgelig giver spillet mening for os, da vi har udviklet det, men det er også helt essentielt at vores målgruppe har en forståelse for spillet og hvordan det gribes an. Det er derfor nødvendigt at lave en test som fortæller os om vi har lavet spillet således, at vores målgruppe forstår at spille spillet korrekt uden at skulle have hjælp fra andre. Hvis vi ikke har ramt vores målgruppe korrekt, så resulterer det i at interessen for spillet forsvinder, og at det dermed ikke er nogen succes.

Da vores spil primært egner sig til vores målgruppe, så har vi ligeledes fundet testpersoner som vi mener passer ind i vores målgruppe. Da vi skulle beskrive vores produkt og vision med spillet, så fandt vi frem til at vores spils målgruppe primært er achievers og explores. Dette blev også bekræftet oppe i vores spilanalyse. Vi har derfor fundet nogle bekendte som vi mener er én af disse to. Vi kan dog aldrig være 100% sikre på vores valg af testpersoner, så dette er selvfølgelig taget i forbehold.

Vores test er opdelt i tre dele som testpersonerne skal igennem. Igennem alle deltests, har vi taget noter af de observationer vi gjorde os imens vi så testpersonen spille spillet. I den første del skal testpersonen afprøve spillet uden at få noget af vide fra udviklerne af spillet. Dette gøres for at se hvor intuitivt spillet fremstår for en person der aldrig har prøvet eller set spillet. Imens testpersonen prøver sig frem med spillet, bliver der noteret hvor godt en person klarer sig igennem spillet.

| Fremgangsmåde                             | Hvorfor skal det testes                                                                |
|-------------------------------------------|----------------------------------------------------------------------------------------|
| Lav en spiller og start spillet.          | Det er essentielt at skulle lave en bruger for at starte spillet.                      |
| Finde ud af hvordan man styrer spilleren. | For at kunne gennemføre spillet, så skal man kunne bevæge spilleren igennem terrænet.  |
| Finde ud af hvordan man starter en quest. | Quests er essentielle i spillet, det er dem som gør at man kan komme videre i spillet. |
| Finde ud af hvordan man bruger spells.    | For at kunne dræbe fjender, og klare quests, så skal man kunne bruge spells.           |

|                                         |                                                                                                          |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------|
| Finde ud af hvordan man klarer questen. | For at kunne komme videre i spillet, er det essentielt for brugeren at vide hvordan questen gennemføres. |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------|

Til den anden del af testen, stiller vi testpersonen en række opgaver som skal løses af testperson selv. Opgaverne vil være at finde frem til forskellige funktioner og bruge dem i spillet. Opgaverne kan ses herunder, med et eksempel på hvordan vi har noteret.

| Fremgangsmåde.                                  | Tjekbox                        |
|-------------------------------------------------|--------------------------------|
| Lav en vilkårlig spiller.                       | Tjek                           |
| Åben SpellBook.                                 | Tjek                           |
| Åben QuestLog.                                  | Tjek                           |
| Få din Quest Lov til at indeholde den 1. Quest. | Svært ved at finde Quest start |
| Benyt alle Spells.                              | ...                            |
| Åben pausemenuen.                               |                                |
| Lav en ny bruger og start spillet.              |                                |
| Luk spillet.                                    |                                |

Deltest 2 fortæller os om testperson forstår hvordan spillets funktionalitet fungerer, og om de er nemme nok til at de kan huskes ud fra at have spillet det én gang.

Til deltest 3 gav vi testpersonen et spørgeskema med en række relevante spørgsmål. Disse spørgsmål er opbygget således at de kan give os en god kvalitativ feedback som gør at vi kan evaluere på vores spil. Yderligere så giver det også mulighed for testpersonen at skrive sin egen mening, og eventuelt komme med nogle kommentarer som vi ikke har kunnet observere i deltest 1 eller deltest 2. Spørgsmålene til deltest 3 står herunder:

1. Hvordan var sværhedsgraden?
2. Hvad var svært?
3. Hvordan var styringen af spilleren?
4. Var det nemt at finde rundt?
5. Underholdningsværdi (1/10)?
6. Hvordan var den generelle oplevelse?
7. Fandt du nogle fejl, hvis ja hvilke?

# Resultater på testen

## Deltest 1 & 2

Ud fra svarene deltest 1 og deltest 2, kan vi konkludere at vores spil var meget nemt at forstå. Alle vores testpersoner fik tjek som resultat i alle kriterier, hvilket betyder at de klarede kriteriet uden problemer. Vores testpersoner havde altså nemt ved at finde og benytte de vigtigste funktioner i vores spil. Der var blot én undtagelse. En af vores testpersoner havde svært ved at finde ud af hvordan man skulle benytte de forskellige spells. Denne testperson valgte ikke at læse skiltene med information omkring hvordan man skulle vælge og affyre sine spells.

Resultatet for testpersonen kan ses herunder:

| Fremgangsmåde.                         | Tjekbox                                   |
|----------------------------------------|-------------------------------------------|
| Finde ud af hvordan man bruger spells. | Skiltene blev ikke læst... så vidste ikke |

## Deltest 3

Deltest 3 viste sig at være en super god idé, da det var denne deltest som gav os mere detaljeret information om hvordan testpersonerne oplevede spillet. Vi har valgt at sortere i vores svar, så vi har de mest relevante stående tilbage. Det betyder, at hvis der er flere som har skrevet eller ment det samme, har vi valgt den mest uddybende kommentar. Da vi ikke har spurgt om tilladelse til brug af deres navne, så fremstår testpersonerne anonyme med et tilfældigt navn.

### Hvordan var sværhedsgraden?

Anonym 1: "Spillet var svært, dog hjalp hjerterne på mappet til at holde min health bar oppe."

Anonym 2: "Medium-svær. Man kan sagtens lære spillet men hvis man ikke har meget skill klarer man det nok ikke første gang"

Vi kan her se at begge personer synes spillet er svært, og her synes Anonym 1 yderligere at det var godt med hjerterne som gjorde at han kunne fortsætte spillet. Vores intentioner med hjerterne, som giver mere liv, var også at spilleren skulle få noget af det tabte liv tilbage, så man ikke skulle være ekstrem forsigtig med at kæmpe.

### Hvad var svært?

Anonym 3: At rotere og skyde samtidig, hvilket gør det sværere at undvige edderkopperne.

### Hvordan var styringen af spilleren?

Anonym 4: "Fin, jeg tror det ville være mere intuitivt at bruge mus til at dreje"

Anonym 5: "Styringen af spilleren var meget nem at lære."

Anonym 6: "Det var ret intuitivt. Det var på en måde meget fedt at man ikke behøvede behøvede at bruge musen."

Da vi spurgte ind til hvordan styringen til spilleren var, så var der lidt delte meninger. Vores intention med styringen var, at det ikke skulle være alt for let at ramme fjenderne, så vi valgte en alternativ måde at styre på som var en del sværere at være præcis med. Anonym 3 bekræfter således også, at det var styringen som gjorde spillet svært. Når vi tager et kig på Anonym 5 og Anonym 6's svar, så kan de godt lide styringen, og synes faktisk at den var nem at lære, modsat Anonym 4 som hellere ville have brugt muse. Styringen kommer ned til personlig præference, og det er ikke noget man kan undgå om sådan.

### **Var det nemt at finde rundt?**

Anonym 7: "Da der ikke var nogen decideret sti at løbe rundt på blev det lidt svært at finde rundt."

Anonym 8: "Stien man følger gør det nemt at forstå, men har prøvet at komme længere i spillet før, hvor det har været sværere at finde rundt pga. manglende sti og kun græs."

Anonym 9: "Min første tanke da jeg kom ud på den store grønne mark var, 'hvor fuck skal jeg nu hen'. Jeg tror jeg fandt den rigtige vej lidt af held men der kunne måske godt have været en sti eller i hvert fald mulighed for at se floden lidt tidligere."

Meningen bag dette spørgsmål var at finde ud af om den bane vi havde lavet var intuitiv nok til at brugeren kunne finde rundt. Her mente alle, at det var nemt at følge indtil man kom ud til et grønt areal hvor stien ophørte. Vores intentioner var at man selv skulle udforske området i stedet for at blive fulgt gennem hele banen.

### **Generelle oplevelse?**

Anonym 10: "Jeg synes det ville være mere intuitivt hvis startknappen var synlig, men sådan 'greyed out' så det var nemmere at finde ud af at man skal have 5 karakterer i sit navn."

Anonym 11: "Det var svært, men man havde virkelig lyst til at fortsætte spillet og komme videre og længere."

Dette spørgsmål lagde op til at testperson selv havde mulighed for at skrive nogle uddybende kommentarer. Dette var også tilfældet, da Anonym 10 synes at start knappen i menuen skulle synliggøres, da den kun kommer frem når man har skrevet et brugernavn med 5+ tegn. Dette er god feedback, som vi slet ikke selv havde i tankerne at undersøge.

## **Vigtige observationer**

Under deltest 1 lavede vi os nogle observationer for hver testperson. Disse observationer bygger både på citater og håndteringen af spillet.

### Anonym 12

Har lidt svært ved movement.

Åbnet spellbook "Ej hvor er det sejt".

Troede at quit var quit til menu.

### Anonym 13

"Styringen virker i starten besværlig og svær at bruge men man vender sig ret hurtig til det og så fungerer det faktisk ret godt."

“Underligt at man kan se slangen gennem væggen i starten man kunne måske spawn den når man går ind i grotten.”

#### Anonym 14

I tvivl om man også skulle dræbes spiders for at løse opgaven.

#### Anonym 15

Gik baglæns, fordi han ikke vidste kasketten var sat på omvendt.

Går langs stierne.

“Det var fucking freaking svært, jeg skal altså lige prøve igen.”

Disse observationer er mindst lige så vigtige som de tre andre deltests, og kan også bruges til at forbedre vores spil, så vores målgruppe får en endnu bedre oplevelse af at spille vore spil.

## Hvordan arbejder vi videre herfra?

På baggrund af disse tests har vi optaget en stor mængde feedback som på flere måder kan forbedre vores spil.

I deltest 1 og deltest 2 kunne vi observere at der var en som ikke læste skiltet som forklaret om spells. Her mener vi at vi måske kunne nedsætte mængden af tekst på skiltene, eller på anden vis gøre skiltet mere attraktivt, så kommende spillere ikke springer den vigtige information over.

I deltest 3 fik vi bekræftet at det var godt at vi havde lavet hjerter som kunne give spilleren mere liv. Dette er god feedback, da vi derved også kan justere sværhedsgraden ved at placere hhv. Flere eller færre hjerter på en bane. Vi blev ydermere bekræftet i at vores valg af spillerstyringen er god og gør spillet mere udfordrende. Dog så er der enkelte som hellere ville bruge musen, og for at opfylde deres behov, kunne vi lave en indstilling som gør at man kan vælge mellem tastatur eller mus som styring. Vi skal derudover have kigget på den måde vi laver vores baner på. I stedet for at have områder uden nogen sti, så kunne vi i stedet lave et område hvor stien deler sig i to, så man skulle træffe et valg. Til sidst så skal vi have gjort start knappen i menuen mere synlig, således at man synligt kan se at man mangler noget før man kan starte spillet.

Nogle af de observationer vi gjorde os under deltest 1 gav os også ting at reflektere over i forbindelse med at gøre spillet bedre.

Da en person blev bedt om at gå til menuen blev der trykket “quit”, fordi testpersonen troede at det var “quit to menu”. Derfor kunne det give god mening at ændre det fra “quit” til “quit game”. Yderligere skal vi have forklaret questen lidt bedre, således at brugeren ved præcis hvad man skal for at klarer questen. Der var en af testpersonerne som gik baglæns under en stor del af spillet, og fandt først ud af det da der skulle skydes. Dette kunne godt betyde at vi skal lave et bedre udseende til spillerne, så man ikke er i tvivl om hvad der fremad.



Der er ingen tvivl om at vores brugertest har gjort os klogere på vores eget spil. Vi blev bekræftet i at nogle af vores beslutninger var gode. Vi har også fået en masse nyttigt feedback som vi kan bruge til at optimere vores spil endnu mere. Hvis vi havde nået at få forbedret spillet, så havde vi lavet endnu en brugertest indtil at vi ikke havde flere ting som kunne gøres bedre.

## Projektevaluering

I dette stykke vil vi gennemgå hvordan vi kom igennem de vigtigste mål vi satte os for i starten af projektet. Først og fremmest så fik vi fulgt vores Scrum of kravspecifikation hele vejen gennem projektet. Dette gjorde det nemt og overskueligt at arbejde gennem hele processen. Når det er sagt, så kom vi aldrig helt i mål med vores kravspecifikation. Vi manglede her til slut tre punkter i could have-sektionen.

Unity 2D som game engine var helt klart det rigtige valg af game engine. På grund af Unity's mange indbyggede funktioner, nåede vi både at komme i mål med en simpel multiplayer prototype, og et næsten færdigt singleplayer-spil. Da vi begyndte at udvikle på vores spil havde vi en idé om at vi skulle lave alle funktioner i spillet kompatibelt med multiplayer-funktionen "PUN photon". Dette valgte vi dog efter at have implementeret de vigtigste del i multiplayer at droppe. Dette valg blev taget da vi hellere ville komme videre med den rene tekniske spiludvikling som var nemmere at foretage i et singleplayer-format. Vi kan dog se på vores testresultater, at det var en god beslutning at fokusere på at lave et godt singleplayer-spil. På denne måde har vi altså både vist at vi kan opstille en connection mellem to computere og skabe et multiplayer-univers, samtidigt med at vi har vist at vi kan udvikle avancerede spil-funktioner i singleplayer. Af dette har vi lært, at det som udgangspunkt er smart at opbygge sit spil til at håndtere multiplayer funktioner, lige fra starten. Derudover er det vigtigt at kende sit library ud og ind for at være istand til, hurtigt og effektive kunne fikse fejl.

I forhold til de grafiske elementer, endte vi også op med at producere alt grafikken selv. Dette var på ingen måde et must have, men det betyder nu at vi har et produkt der ikke har nogle kommercielle blokader.

I vores spilanalyse finder vi frem til at vores færdigproducerede spil, lever op til de krav vi satte til målgruppen. Spillet appellerer til "Achievers", "Explorers" og "Killers". Dette betyder at vores estimat af målgruppe blev opnået. Derudover valgte vi genremæssigt at bygge spillet op efter fantasy-genren. Dette lever vores færdige produkt også op til ifølge vores spilanalyse.

# Konklusion

Videospil er meget mere end bare tidsfordriv. Det giver mulighed for socialt samvær og nye bekendtskaber rundt omkring på kloden. Vi har med vores videospil prøvet at stræbe efter at give spilleren disse kvaliteter. I vores spil har brugeren mulighed for at spille sammen med andre, eller nyde spillets æstetik i eget selskab.

Vi har ved hjælp af MoSCoW-modellen fået opstillet en nem og overskueligt kravspecifikation som lægger op til til vores arbejdsmetode 'Scrum'. Her har vi benyttet os af et Gantt-kort som holder styr på hvornår vi skal afslutte et sprint og gå i gang med et nyt. Vi har yderligere ved hjælp af Scrum's rollefordeling haft uddelegeret ansvarsområder, således vi havde ansvar for hver vores del af projektet.

Da vi skulle udvikle vores spil i Unity, valgte vi at lave et singleplayerspil og et multiplayerspil. Dette gjorde vi på baggrund af, at vi følte os begrænset af den måde spillets multiplayerfunktionalitet skulle sættes op. Derfor valgte vi at fortsætte i singleplayer, hvor vi fik opsat et avanceret kampsystem med spells, samt en hel quest som man kan gennemføre. Vi har lavet spilleren og dens monster således at det kan tilpasses af brugeren selv, så man kan opbygge en karakter som brugeren selv synes er passende.

Vi har benyttet Photon PUN til at implementerer multiplayerfunktionalitet i vores spil. Dette er et asset, som har kunnet downloades fra Unity's Asset Store. Med PUN fik vi lavet et funktionelt multiplayerspil, som kører over Photons gratis servere, hvilket betyder at brugeren selv ikke behøves at hoste en server. Dog har vi haft problemer med yderligere udvikling af multiplayerspillet, da vores grundlæggende forståelse for PUN ikke var nok til at kunne tilpasse det vores spil perfekt.

Vi har udarbejdet en spilanalyse af vores endelige singleplayer produkt, da dette var det tættere på vores intention af det færdige spil. Denne spilanalyse tager udgangspunkt i strukturen fra MDA framework, hvor vi analyserer spillets målsætninger, dynamics og mechanics for at finde spillets centrale aesthetics. I spilanalysen fandt vi frem at vores spil primært appellerer til spillertyperne achievers og explores.

Da vores spil var lavet efter vores intentioner, så var det vigtigt med en fyldestgørende brugertest, som skulle sikre os, at vores målgruppe kunne finde ud af spillet. Ud fra vores resultater, fandt vi frem til at brugerne generelt ikke havde problemer med hvordan spillets essentielle dele såsom movement, spells, quests og fjender fungerede. Vi har også på baggrund af testobservationerne optaget en stor mængde feedback som kan bruges til at forbedre vores spil i fremtiden.

Samlet set har vi i et tilstrækkeligt omfang har skabt et spil, inden for de forudbestemte og fastsatte tidsrammer, der lever op til, ikke blot de overordnede krav sat for dette projekt, men også de specificerede krav vi satte for os selv forud for udviklingsprocessen. Vi kan ligeledes se på vores test at vi har ramt vores valgte målgruppe godt, hvilket skaber et grundlag for en udvikling i den rigtige retning.

# Litteraturliste

Komplett INC, Positive aspekter ved gaming for børn, 25/4 - 21,  
<https://www.komplett.dk/article/88166/fordele-ved-spil-for-boern>

Unity INC, Update and fixed update, 11/4 - 21,  
<https://www.youtube.com/watch?v=u42aWzAlAqg>

Thirslund, Asbjørn, Top Down shooting in Unity, 12/3 - 21,  
<https://www.youtube.com/watch?v=LNLVOjbrQj4>

Calice, Noa, SHOOTING/FOLLOW/RETREAT ENEMY AI, 19/3 - 21,  
[https://www.youtube.com/watch?v=\\_Z1t7MNk0c41](https://www.youtube.com/watch?v=_Z1t7MNk0c41)

Diving Squid, How to make an online multiplayer game, 16/3 - 21,  
<https://www.youtube.com/watch?v=nmPukdOsYQA>

Thirslund, Asbjørn, 2D Animation in Unity, 13/4,  
<https://www.youtube.com/watch?v=hkaysu1Z-N8>,

Thirslund, Asbjørn, PAUSE MENU in Unity, 16/4 - 21,  
<https://www.youtube.com/watch?v=JivuXdrIHK0&t=570s>

Photon INC, PUN, 16/3 - 21,  
<https://www.photonengine.com/pun>

Photon INC, Introduction to PUN/ Photon Unity, 16/3 - 21,  
<https://doc.photonengine.com/en-us/pun/current/getting-started/pun-intro>

Photon INC, Introduction, 6/4 - 21,  
<https://doc.photonengine.com/en-us/pun/current/demos-and-tutorials/pun-basics-tutorial/intro>

Photon INC, Lobby, 6/4 - 21,  
<https://doc.photonengine.com/en-us/pun/current/demos-and-tutorials/pun-basics-tutorial/lobby>

Photon INC, Player Networking, 6/4 - 21,  
<https://doc.photonengine.com/en-us/pun/current/demos-and-tutorials/pun-basics-tutorial/player-networking>

Photon INC, Player Instantiation, 6/4 - 21,

<https://doc.photonengine.com/en-us/pun/current/demos-and-tutorials/pun-basics-tutorial/player-instantiation>

Thirslund, Asbjørn, Tilemaps in Unity, 12/3 - 21,  
[https://www.youtube.com/watch?v=ryISV\\_nH8qw&t=379s](https://www.youtube.com/watch?v=ryISV_nH8qw&t=379s)

Thirslund, Asbjørn, 2DMovement in Unity, 12/3 - 21,  
<https://www.youtube.com/watch?v=dwcT-Dch0bA&t=980s>

Hunicke Robin, LeBlanc Marc og Zubek Robert, "MDA: A Formal Approach to Game Design and Game Research", 25/4 - 21,  
<http://www.cs.northwestern.edu/~hunicke/pubs/MDA.pdf>

Thirslund, Asbjørn, How to make a Health bar in Unity, 16/4 - 21,  
[https://www.youtube.com/watch?v=BLfNP4Sc\\_iA&t=596s](https://www.youtube.com/watch?v=BLfNP4Sc_iA&t=596s)

# Bilag

## Bilag 1 - Tro og love erklæring

"Jeg bekræfter herved med min underskrift, at opgavebesvarelsen er udarbejdet af mig. Jeg har ikke anvendt tidligere bedømt arbejde uden henvisning hertil, og opgavebesvarelsen er udfærdiget uden anvendelse af uretmæssig hjælp og uden brug af hjælpemidler, der ikke har været tilladt under prøven."

27/04-2021 *Gustav Alexander Lilleskov Kensø*

---

Dato og underskrift

27/04-2021 *Rasmus Alexander Christiansen*

---

Dato og underskrift

27/04-2021 *Simon Saxild Danielsen*

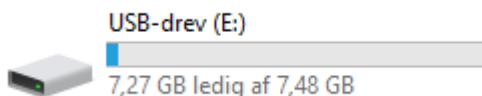
---

Dato og underskrift

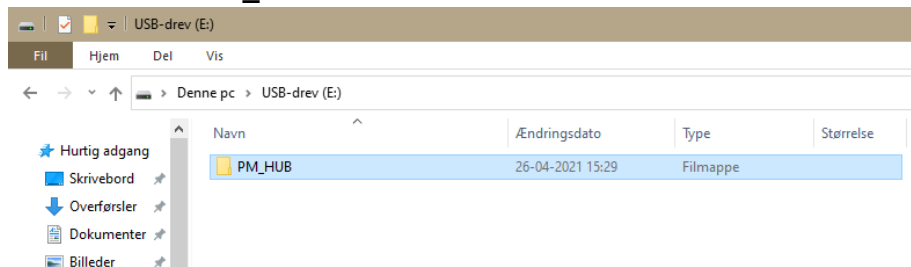
## Bilag 2 - Installation af programmet fra USB

Her vil vi trinvis fortælle hvordan man kører vores spil fra USB-stikket i Windows 10.

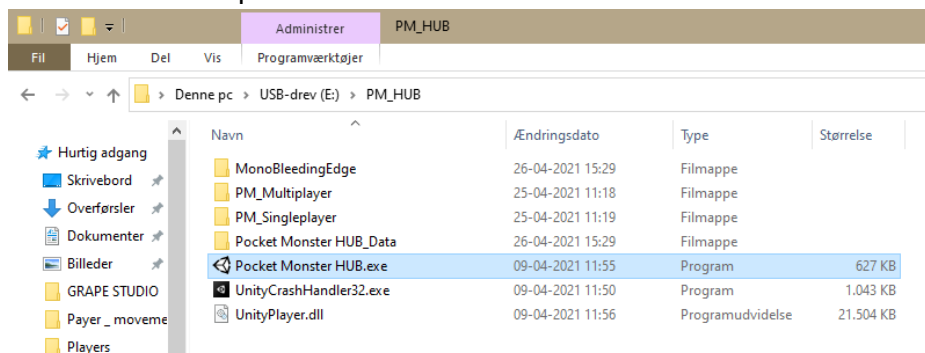
1. Indsæt USB-stikket i computeren.
2. Åben stifinderen og dobbeltklik på USB-drev.



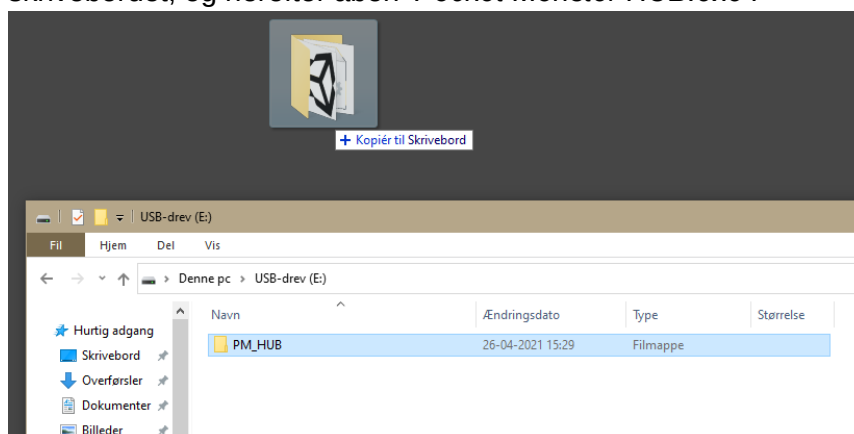
3. Åben herefter 'PM\_HUB'.



4. Herfra kan du åbne spillet direkte fra USB-stikket, eller hente det ned på computeren.
  - a. Hvis du vil åbne spillet fra USB-stikket kan 'Pocket Monster HUB.exe' åbnes.



- b. Hvis du vil hente det ned på computeren, så flyt mappen 'PM\_HUB' ud på skrivebordet, og herefter åben 'Pocket Monster HUB.exe'.

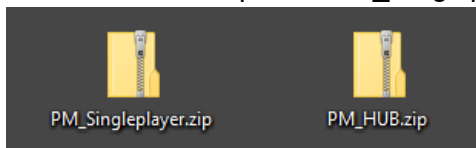




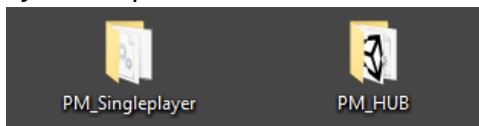
## Bilag 3 - Installation af programmet fra Lectio

Hvis spillet hentes fra Lectio, består det af to dele. Disse dele skal sættes sammen for at spillet virker korrekt. Følg instruktionen nedenunder

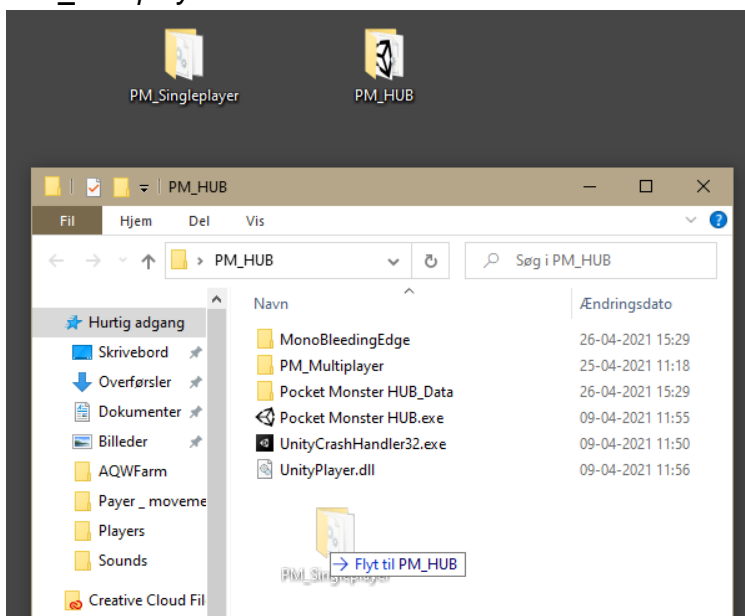
1. Download de to zip-filer *"PM\_Singleplayer.zip"* og *"PM\_HUB.zip"*.



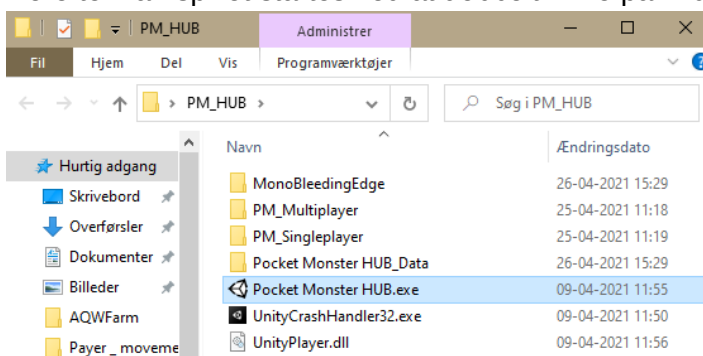
2. Herefter åbnes begge zip-filer, hvor mapperne *"PM\_Singleplayer"* og *"PM\_HUB"* flyttes ud på skrivebordet.



3. Nu åbnes mappen *"PM\_HUB"*. Dernæst flyttes mappen *"PM\_Singleplayer"* ind i mappen *"PM\_HUB"*, således at den er i samme mappe som mappen *"PM\_Multiplayer"*.



4. Herefter kan spillet startes ved at dobbeltklikke på *"Pocket Monster HUB"*.



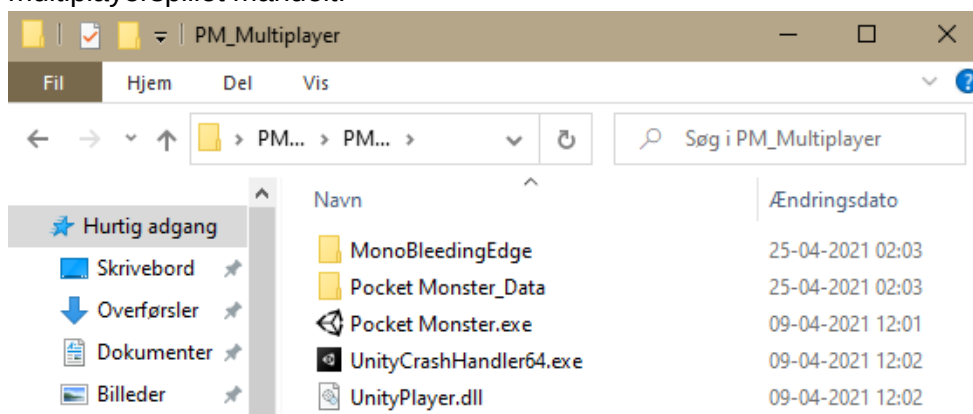
## Bilag 4 - Hvis singleplayer eller multiplayer ikke starter i hubben

Dette er en alternativ løsning til at åbne singleplayerversionen eller multiplayerversionen, hvis ikke de starter i hubben.

Vær obs. på at der godt kan gå noget tid før spillene starter fra hubben. Vent i minimum 25 sekunder, hvis spillet køres fra USB-stikket

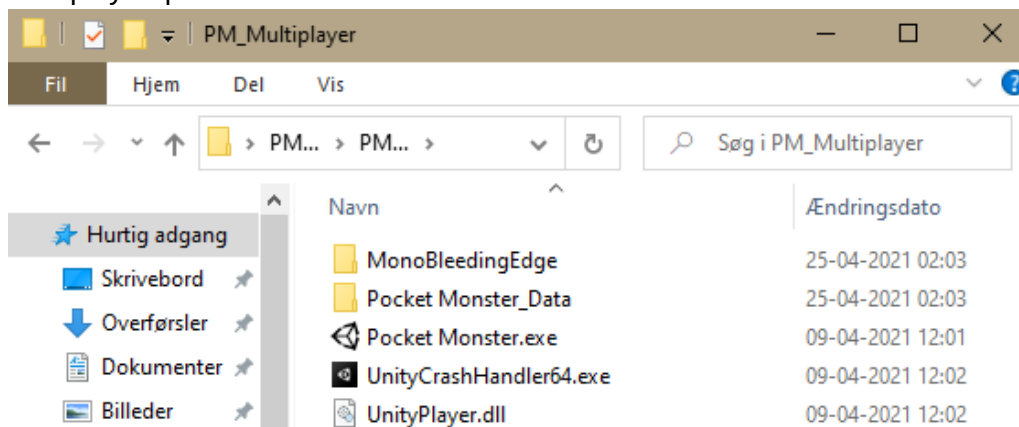
### USB-stik

1. Udfør bilag 2 fra trin 1-4.
2. Herefter åben mappen "PM\_Singleplayer" eller mappen "PM\_Multiplayer".
3. Herunder ligger der en fil som hedder "Pocket Monster.exe", som er ens for både multiplayer og singleplayer. Denne fil åbnes for at starte singleplayerspillet eller multiplayerspillet manuelt.



### Lectio

1. Udfør bilag 3 fra trin 1 til 3
2. Herefter åben mappen "PM\_Singleplayer" eller mappen "PM\_Multiplayer".
3. Herunder ligger der en fil som hedder "Pocket Monster.exe", som er ens for både multiplayer og singleplayer. Denne fil åbnes for at starte singleplayerspillet eller multiplayerspillet manuelt.



## Bilag 5 - Tænke-højt test beskrivelse

# Tænke-højt test af læringsprogrammet

### Planlægning af test

- Hvad skal testes og hvordan?
- Hvem skal teste? Kom så tæt på målgruppen som muligt
- Hvilket udstyr og hvor skal det foregå?
- Hvordan skal brugeren briefes inden testen?
- Hvilke opgaver skal konkret løses?

Dokumentation, f.eks. videotest: skærmoptagelse, youtube, del, sæt link til filmen i dokumentationen

### Refleksioner !

- Hvordan gik testen?
- Hvilke observationer gjorde test-personerne?
- Hvordan vil I arbejde videre her fra?
  - Hvad vil I ændre? Hvad vil I bevare? Hvad ville I gøre, hvis I havde tid? Og hvorfor?

## Bilag 6 - Test

Målgruppe: Achievers og Adventure.

Testen skal give et indblik i om udviklerne har lavet spillet således at brugeren kan komme i gang uden at skulle spørge udviklerne om hvordan man skal gøre.

Briefing til tester:

Observationer:

Opgaver som skal løses:

| Fremgangsmåde                             | Hvorfor skal det testes                                                                                  |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Lav en spiller og start spillet.          | Det er essentielt at skulle lave en bruger for at starte spillet.                                        |
| Finde ud af hvordan man styrer spilleren. | For at kunne gennemføre spillet, så skal man kunne bevæge spilleren igennem terrænet.                    |
| Finde ud af hvordan man starter en quest. | Quests er essentielle i spillet, det er dem som gør at man kan komme videre i spillet.                   |
| Finde ud af hvordan man bruger spells.    | For at kunne dræbe fjender, og klare quests, så skal man kunne bruge spells.                             |
| Finde ud af hvordan man klarer questen.   | For at kunne komme videre i spillet, er det essentielt for brugeren at vide hvordan questen gennemføres. |

Andenrunde af opgaver som skal løses:

|                                                 |         |
|-------------------------------------------------|---------|
| Fremgangsmåde.                                  | Tjekbox |
| Lav en vilkårlig spiller.                       | Tjek    |
| Åben SpellBook.                                 | Tjek    |
| Åben QuestLog.                                  | ...     |
| Få din Quest Lov til at indeholde den 1. Quest. |         |
| Benyt alle Spells.                              |         |
| Åben pausemenuen.                               |         |
| Lav en ny bruger og start spillet.              |         |
| Luk spillet.                                    |         |

### Uddybende spørgsmål:

Hvordan var sværhedsgraden?

Hvad var svært?

Hvordan var styringen af spilleren?

Var det nemt at finde rundt?

Underholdningsværdi?

Generelle oplevelse?

Fandt du nogle fejl, hvis ja hvilke?

## Bilag 7 - Spiller animationssprites



## Bilag 8 - Yderligere sprites

## Bilag 9 - Script Player

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using TMPro;

public class Player : MonoBehaviour
{
    public AudioClip PlayerDamage1;
    public AudioClip PlayerDamage2;
    AudioSource audioPlayerDamage1;
    AudioSource audioPlayerDamage2;

    public float maxHealth = 100f;
    public float currentHealth;
    public GameObject player;

    public HealthBar healthBar;

    public TMP_Text usernameText;
    public TMP_Text HPText;

    public GameObject UserCanvas;
```

```
// Start is called before the first frame update
void Start()
{
    audioPlayerDamage1 = AddAudio(false, false, 1f);
    audioPlayerDamage2 = AddAudio(false, false, 1f);

    currentHealth = maxHealth;
    healthBar.SetMaxHealth(maxHealth);

    usernameText.text = MenuController.UsernameIngame;
    HPText.text = currentHealth.ToString("F1") + "/100,0";

    PauseMenu.GameIsPaused = false;
    CheckForDeath.GameOver = false;
    CheckForDeathQ1.GameOver = false;
    UserCanvas.SetActive(true);
}

// Update is called once per frame
void Update()
{
    // Endrer spillerens liv i gui'en.
    HPText.text = currentHealth.ToString("F1") + "/100,0";

    // Sætter spillerens liv i healthbaren til nuværende liv.
    healthBar.SetHealth(currentHealth);

    // Hvis en edderkop har ramt fjenden.
    if (SpiderBullet.Losthealth == 1)
    {
        // Tag skade fra edderkoppen.
        TakeDamage(Spider.bulletdamage);
        SpiderBullet.Losthealth = 0;
    }

    // Hvis en slange har ramt fjenden.
    if (SnakeBullet.Losthealth == 1)
    {
        // Tag skade fra slangen.
        TakeDamage(Snake.bulletdamage);
        SnakeBullet.Losthealth = 0;
    }
}
```



```
}

// Hvis spilleren ikke har mere liv.
if (currentHealth <= 0)
{
    // Fjern spilleren.
    Destroy(player) ;
}

// Hvis der er GameOver
if (CheckForDeath.GameOver == true)
{
    // Sæt spillerens GUI til at være usynligt.
    UserCanvas.SetActive(false) ;
}

// Ellers hvis der ikke er GameOver.
else if (CheckForDeath.GameOver == false)
{
    // Hvis spillet i stedet er pauset.
    if (PauseMenu.GameIsPaused == true)
    {
        UserCanvas.SetActive(false) ;
    }
    // Hvis spillet heller ikke er pauset.
    else if (PauseMenu.GameIsPaused == false)
    {
        UserCanvas.SetActive(true) ;
    }
}

// Det samme GameOver princip som oven over, men bare i Quest
1.
if (CheckForDeathQ1.GameOver == true)
{
    UserCanvas.SetActive(false) ;
}
else if (CheckForDeathQ1.GameOver == false)
{
    if (PauseMenu.GameIsPaused == true)
    {
        UserCanvas.SetActive(false) ;
    }
}
```

```
        else if (PauseMenu.GameIsPaused == false)
        {
            UserCanvas.SetActive(true);
        }
    }
}

// Funktion hvor spilleren tager skade.
public void TakeDamage(float damage)
{
    // Endre spillerens nuværende liv.
    currentHealth -= damage;
    healthBar.SetHealth(currentHealth);

    // Vælg en tilfældig player damage lyd og afspil den.
    int SoundRandomizer = Random.Range(1, 100);
    if (SoundRandomizer <= 50)
    {
        audioPlayerDamage1.clip = PlayerDamage1;
        audioPlayerDamage1.Play();
    }
    else if (SoundRandomizer > 50)
    {
        audioPlayerDamage2.clip = PlayerDamage2;
        audioPlayerDamage2.Play();
    }
}

// Funktion til at sætte indstillinger på AudioSources.
public AudioSource AddAudio(bool loop, bool playAwake, float vol)
{
    AudioSource newAudio = gameObject.AddComponent<AudioSource>();
    newAudio.loop = loop;
    newAudio.playOnAwake = playAwake;
    newAudio.volume = vol;
    return newAudio;
}
}
```

## Bilag 10 - script Payer movement

```
using System.Collections;
```

```
using System.Collections.Generic;
using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    public float moveSpeed = 5f;

    public Rigidbody2D rb;

    public Animator animator;

    Vector2 movement;

    void Update()
    {
        // Tjekker for horisontale og vertikale inputs.
        movement.x = Input.GetAxisRaw("Horizontal");
        movement.y = Input.GetAxisRaw("Vertical");

        // Sætter 'Speed' i Animator parameteren til den numeriske sum
        af movement.x og movement.y.
        animator.SetFloat("Speed", (Mathf.Abs(movement.x) +
        Mathf.Abs(movement.y)));

        // Normaliser movement vektoren, så man ikke går 40% hurtigere
        diagonal.
        movement = movement.normalized;
    }

    void FixedUpdate()
    {
        // Rykker spillerens position
        rb.MovePosition(rb.position + movement * moveSpeed *
        Time.fixedDeltaTime);

        // Hvis venstre piletast er trykket ned, så skal spilleren
        dreje til venstre.
        if (Input.GetKey(KeyCode.LeftArrow))
        {
            rb.rotation += 6.0f;
        }
    }
}
```

```
// Hvis højre piletast er trykket ned, så skal spilleren dreje  
til højre.  
else if (Input.GetKey(KeyCode.RightArrow))  
{  
    rb.rotation -= 6.0f;  
}  
}  
}
```

## bilag 11- script Snake

```
using System.Collections;  
using System.Collections.Generic;  
using System.Collections.Specialized;  
using System.Security.Cryptography;  
using System.Threading;  
using UnityEngine;  
  
public class Snake : MonoBehaviour  
{  
    public static float bulletdamage = 0.5f;  
    public float speed;  
    public float stoppingDistance;  
    public float retreatDistance;  
    public float seeDistance = 20;  
  
    private float timeBetweenShots;  
    public float startTimeBetweenShots;  
  
    public GameObject projectile;  
    public Transform player;  
    public GameObject enemy;  
  
    public GameObject gem;  
  
    public int maxHealth = 350;  
    public int currentHealth;  
  
    private bool GameIsStarted;  
  
    private float timeLeft = 0;
```

```
public Transform firePoint;

private bool PlayerFound;

void Start()
{
    // Sætter fjendens liv til det maksimale liv
    currentHealth = maxHealth;

    timeBetweenShots = startTimeBetweenShots;

    PlayerFound = false;
}

void Update()
{
    // Leder efter spilleren indtil spilleren er fundet.
    if (PlayerFound != true)
    {
        player =
GameObject.FindGameObjectWithTag("Player").transform;
        GameIsStarted = true;
    }

    // Får variablen til at gå imod 0.
    if (timeLeft >= 0)
    {
        timeLeft -= Time.deltaTime;
    }

    if (GameIsStarted)
    {
        if (GameObject.FindGameObjectWithTag("Player") != null)
        {
            if (timeLeft <= 0)
            {
                // Hvis spilleren er indenfor den bestemte afstand
                if (Vector2.Distance(transform.position,
player.position) < seeDistance)
                {

```

```
// Gaa efter spiller
if (Vector2.Distance(transform.position,
player.position) > stoppingDistance)
{
    transform.position =
Vector2.MoveTowards(transform.position, player.position, speed *
Time.deltaTime);

    RotateTowards(player.position);
}

// Hvis spillerne kommer for tæt på, skal den
tage afstand.
else if (Vector2.Distance(transform.position,
player.position) < stoppingDistance &&
Vector2.Distance(transform.position, player.position) >
retreatDistance)
{
    transform.position =
this.transform.position;
}

// Hvis spilleren kommer for tæt på, skal den
rottere mod spilleren.
else if (Vector2.Distance(transform.position,
player.position) < retreatDistance)
{
    transform.position =
Vector2.MoveTowards(transform.position, player.position, -speed *
Time.deltaTime);

    RotateTowards(player.position);
}

// Hvis der ikke er noget cooldown
if (timeBetweenShots <= 0)
{
    // Affyr et projektil mod spilleren
    Instantiate(projectile, firePoint.position,
firePoint.rotation);

    timeBetweenShots = startTimeBetweenShots;
}
```



```
        // Hvis der er cooldown
        else
        {
            // Får cooldown til at gå ned til 0.
            timeBetweenShots -= Time.deltaTime;
        }
    }
}

}

}

}

}

// Funktion som får fjende til at kigge imod taget (spilleren).
private void RotateTowards(Vector2 target)
{
    var offset = -90f;
    Vector2 direction = target - (Vector2)transform.position;
    direction.Normalize();
    float angle = Mathf.Atan2(direction.y, direction.x) *
    Mathf.Rad2Deg;
    transform.rotation = Quaternion.Euler(Vector3.forward * (angle
+ offset));
}

// Funktion hvor fjenden tager skade.
public void TakeDamage(int damage)
{
    currentHealth -= damage;
}

void OnCollisionEnter2D(Collision2D collision)
{
    // Hvis fjenden rammer en FireBullet
    if (collision.gameObject.name == "FireBullet(Clone)")
    {
        // Tag skade
        TakeDamage(FireSpell.BulletDamage);

        // Hvis fjendens liv er død.
        if (currentHealth <= 0)
        {

```

```
        // Sætter diamantens position til fjendens position og
        // gør den synlig.
        gem.transform.position = new
Vector2(enemy.transform.position.x, enemy.transform.position.y);
        gem.SetActive(true);
        Destroy(gameObject);
    }
}

// Hvis fjenden rammer en PoisonBullet
else if (collision.gameObject.name == "PoisonBullet(Clone)")
{
    TakeDamage(PoisonSpell.BulletDamage);

    // Hvis fjendens liv er død.
    if (currentHealth <= 0)
    {
        gem.transform.position = new
Vector2(enemy.transform.position.x, enemy.transform.position.y);
        gem.SetActive(true);
        Destroy(gameObject);
    }
}

// Hvis fjenden rammer en ElectricBullet
else if (collision.gameObject.name == "ElectricBullet(Clone)")
{
    TakeDamage(ElectricSpell.BulletDamage);
    timeLeft = 5f;

    // Hvis fjendens liv er død.
    if (currentHealth <= 0)
    {
        gem.transform.position = new
Vector2(enemy.transform.position.x, enemy.transform.position.y);
        gem.SetActive(true);
        Destroy(gameObject);
    }
}

// Hvis fjenden rammer en PoisonDartBullet
else if (collision.gameObject.name ==
"PoisonDartBullet(Clone)")
```

```
{  
    TakeDamage(PoisonDartSpell.BulletDamage) ;  
  
    // Hvis fjendens liv er død.  
    if (currentHealth <= 0)  
    {  
        gem.transform.position = new  
Vector2(enemy.transform.position.x, enemy.transform.position.y) ;  
        gem.SetActive(true) ;  
        Destroy(gameObject) ;  
    }  
}  
}
```

## bilag 12- script Spider

```
using System.Collections;  
using System.Collections.Generic;  
using System.Collections.Specialized;  
using System.Security.Cryptography;  
using System.Threading;  
using UnityEngine;  
  
public class Spider : MonoBehaviour  
{  
    public static float bulletdamage = 20f;  
    public float speed;  
    public float stoppingDistance;  
    public float retreatDistance;  
    public float seeDistance = 10;  
  
    private float timeBetweenShots;  
    public float startTimeBetweenShots;  
  
    public GameObject projectile;  
    public Transform player;  
    public GameObject enemy;  
  
    public int maxHealth = 40;  
    public int currentHealth;
```

```
private bool GameIsStarted;

private float timeLeft = 0;

public Transform firePoint;

static AudioSource spiderDamageAudio;

private bool PlayerFound;

void Start()
{
    // Sætter fjendens liv til det maksimale liv
    currentHealth = maxHealth;

    timeBetweenShots = startTimeBetweenShots;

    PlayerFound = false;

    spiderDamageAudio = GetComponent<AudioSource>();
}

void Update()
{
    // Leder efter spilleren indtil spilleren er fundet.
    if (PlayerFound != true)
    {
        player =
GameObject.FindGameObjectWithTag("Player").transform;
        GameIsStarted = true;
    }

    // Får variablen til at gå imod 0.
    if (timeLeft >= 0)
    {
        timeLeft -= Time.deltaTime;
    }

    if (GameIsStarted)
    {

```

```
        if (GameObject.FindGameObjectWithTag("Player") != null)
        {
            if (timeLeft <= 0)
            {

                // Hvis spilleren er indenfor den bestemte afstand
                if (Vector2.Distance(transform.position,
player.position) < seeDistance)
                {
                    // Gaa efter spiller
                    if (Vector2.Distance(transform.position,
player.position) > stoppingDistance)
                    {
                        transform.position =
Vector2.MoveTowards(transform.position, player.position, speed *
Time.deltaTime);

                        RotateTowards(player.position);

                    }

                    // Hvis spillerne kommer for tæt på, skal den
tage afstand.
                    else if (Vector2.Distance(transform.position,
player.position) < stoppingDistance &&
Vector2.Distance(transform.position, player.position) >
retreatDistance)
                    {
                        transform.position =
this.transform.position;

                    }

                    // Hvis spilleren kommer for tæt på, skal den
rotere mod spilleren.
                    else if (Vector2.Distance(transform.position,
player.position) < retreatDistance)
                    {
                        transform.position =
Vector2.MoveTowards(transform.position, player.position, -speed *
Time.deltaTime);

                        RotateTowards(player.position);

                    }
                }
            }
        }
    }
}
```

```
// Hvis der ikke er noget cooldown
if (timeBetweenShots <= 0)
{
    // Affyr et projektil mod spilleren
    Instantiate(projectile, firePoint.position,
firePoint.rotation);

    timeBetweenShots = startTimeBetweenShots;
}

// Hvis der er cooldown
else
{
    // Får cooldown til at gå ned til 0.
    timeBetweenShots -= Time.deltaTime;
}
}
}
}

// Funktion som får fjende til at kigge imod taget (spilleren).
private void RotateTowards(Vector2 target)
{
    var offset = -90f;
    Vector2 direction = target - (Vector2)transform.position;
    direction.Normalize();
    float angle = Mathf.Atan2(direction.y, direction.x) *
Mathf.Rad2Deg;
    transform.rotation = Quaternion.Euler(Vector3.forward * (angle
+ offset));
}

// Funktion hvor fjenden tager skade.
public void TakeDamage(int damage)
{
    spiderDamageAudio.Play(0);
    currentHealth -= damage;
}
```



```
void OnCollisionEnter2D(Collision2D collision)
{
    // Hvis fjenden rammer en FireBullet
    if (collision.gameObject.name == "FireBullet(Clone)")
    {
        // Tag skade
        TakeDamage(FireSpell.BulletDamage);

        // Hvis fjendens liv er død.
        if (currentHealth <= 0)
        {
            Destroy(gameObject);
        }
    }

    // Hvis fjenden rammer en PoisonBullet
    else if (collision.gameObject.name == "PoisonBullet(Clone)")
    {
        TakeDamage(PoisonSpell.BulletDamage);
        if (currentHealth <= 0)
        {
            Destroy(gameObject);
        }
    }

    // Hvis fjenden rammer en ElectricBullet
    else if (collision.gameObject.name == "ElectricBullet(Clone)")
    {
        TakeDamage(ElectricSpell.BulletDamage);
        timeLeft = 5f;
        if (currentHealth <= 0)
        {
            Destroy(gameObject);
        }
    }

    // Hvis fjenden rammer en PoisonDartBullet
    else if (collision.gameObject.name ==
"PoisonDartBullet(Clone)")
    {
        TakeDamage(PoisonDartSpell.BulletDamage);
        if (currentHealth <= 0)
```

```
        {  
            Destroy(gameObject);  
        }  
    }  
}
```

## bilag 13 - script SnakeBullet

```
using System;  
using System.Collections;  
using System.Collections.Generic;  
using System.Diagnostics;  
using UnityEngine;  
  
public class SnakeBullet : MonoBehaviour  
{  
    private float speed = 10;  
  
    public float lifeTimer;  
    public float lifeDuration = 0.5f;  
  
    private Transform player;  
    private Vector2 target;  
  
    public static int Losthealth = 0;  
  
    void Start()  
    {  
        // Find spilleren og gør den til et target  
        player = GameObject.FindGameObjectWithTag("Player").transform;  
        target = new Vector2(player.position.x, player.position.y);  
        lifeTimer = lifeDuration;  
    }  
  
    void Update()  
    {  
        // Rykker projektilet  
        transform.position = Vector2.MoveTowards(transform.position,  
target, speed * Time.deltaTime);
```

```
        lifeTimer -= Time.deltaTime;

        // Hvis dens lifetimer løber ud, så skal projektilet fjernes.
        if (lifeTimer <= 0f)
        {
            DestroyProjectile();
        }

    }

    void OnTriggerEnter2D(Collider2D main)
    {
        // Hvis det rammer en spiller skal det ødelægge projektilet og
        sætte Losthealth til 1.
        if (main.CompareTag("Player"))
        {
            Destroy(gameObject);
            SnakeBullet.Losthealth = 1;
        }

        // Hvis det rammer en kant skal det ødelægge projektilet.
        else if (main.CompareTag("RockTree"))
        {
            Destroy(gameObject); //DestroyProjectile();
        }
    }

    void OnCollisionEnter2D(Collision2D collision)
    {
        Destroy(gameObject);
    }

    void DestroyProjectile()
    {
        Destroy(gameObject);
    }
}
```

## bilag 14 - script SpiderBullet

```
using System;
using System.Collections;
using System.Collections.Generic;
using System.Diagnostics;
using UnityEngine;

public class SpiderBullet : MonoBehaviour
{
    private float speed = 10;

    private Transform player;
    private Vector2 target;

    public static int Losthealth = 0;

    void Start()
    {
        // Find spilleren og gør den til et target
        player = GameObject.FindGameObjectWithTag("Player").transform;
        target = new Vector2(player.position.x, player.position.y);
    }

    void Update()
    {
        // Rykker projektilet
        transform.position = Vector2.MoveTowards(transform.position,
target, speed * Time.deltaTime);

        // Hvis dens lifetimer løber ud, så skal projektilet fjernes.
        if (transform.position.x == target.x && transform.position.y ==
target.y)
        {
            DestroyProjectile();
        }
    }

    void OnTriggerEnter2D(Collider2D main)
    {

```

```
// Hvis det rammer en spiller skal det ødelægge projektilet og
sætte Losthealth til 1.
    if (main.CompareTag("Player"))
    {
        Destroy(gameObject);
        SpiderBullet.Losthealth = 1;
    }
    // Hvis det rammer en kant skal det ødelægge projektilet.
    else if (main.CompareTag("RockTree"))
    {
        Destroy(gameObject);
    }
}

void OnCollisionEnter2D(Collision2D collision)
{
    Destroy(gameObject);
}

void DestroyProjectile()
{
    Destroy(gameObject);
}
}
```

## bilag 15 - script FireSpell

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class FireSpell : MonoBehaviour
{
    public float lifeDuration = 0.4f;
    public float lifeTimer;
    public static int BulletDamage = 10;

    void Start()
    {
        lifeTimer = lifeDuration;
    }
}
```

```
void Update()
{
    // Tjekker om skuddet skal ødelægges.
    lifeTimer -= Time.deltaTime;
    if (lifeTimer <= 0f)
    {
        Destroy(gameObject);
    }
}

// Hvis skuddet rammer noget, skal det ødelægges.
void OnCollisionEnter2D(Collision2D collision)
{
    Destroy(gameObject);
}
}
```

## bilag 16 - script PoisonDartSpell

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class PoisonDartSpell : MonoBehaviour
{
    public float lifeDuration = 0.4f;
    public float lifeTimer;
    public static int BulletDamage = 5;

    void Start()
    {
        lifeTimer = lifeDuration;
    }

    void Update()
    {
        // Tjekker om skuddet skal ødelægges.
        lifeTimer -= Time.deltaTime;
        if (lifeTimer <= 0f)
```



```
{  
    Destroy(gameObject);  
}  
  
// Hvis skuddet rammer noget, skal det ødelægges.  
void OnCollisionEnter2D(Collision2D collision)  
{  
    Destroy(gameObject);  
}  
}
```

## bilag 17 - script ElectricSpell

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
  
public class ElectricSpell : MonoBehaviour  
{  
    public float lifeDuration = 0.4f;  
    public float lifeTimer;  
    public static int BulletDamage = 0;  
  
    void Start()  
    {  
        lifeTimer = lifeDuration;  
    }  
  
    void Update()  
    {  
        // Tjekker om skuddet skal ødelægges.  
        lifeTimer -= Time.deltaTime;  
        if (lifeTimer <= 0f)  
        {  
            Destroy(gameObject);  
        }  
    }  
  
    // Hvis skuddet rammer noget, skal det ødelægges.  
    void OnCollisionEnter2D(Collision2D collision)
```

```
{  
    Destroy(gameObject);  
}  
}
```

## bilag 18 - script PoisonSpell

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
  
public class PoisonSpell : MonoBehaviour  
{  
    public float lifeDuration = 1f;  
    public float lifeTimer;  
    public static int BulletDamage = 40;  
  
    void Start()  
    {  
        lifeTimer = lifeDuration;  
    }  
  
    void Update()  
    {  
        // Tjekker om skuddet skal ødelægges.  
        lifeTimer -= Time.deltaTime;  
        if (lifeTimer <= 0f)  
        {  
            Destroy(gameObject);  
        }  
    }  
  
    // Hvis skuddet rammer noget, skal det ødelægges.  
    void OnCollisionEnter2D(Collision2D collision)  
    {  
        Destroy(gameObject);  
    }  
}
```

## bilag 19 - script Sign

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class Sign : MonoBehaviour
{
    public int seeDistance;
    private bool PlayerCloseEnough;
    private bool GameIsStarted;
    public Transform player;
    public GameObject SignUI;

    // Start is called before the first frame update
    void Start()
    {
        PlayerCloseEnough = true;
    }

    public void GameStart()
    {
        // Find spilleren
        player = GameObject.FindGameObjectWithTag("Player").transform;
        GameIsStarted = true;
    }

    // Update is called once per frame
    void Update()
    {
        if (GameIsStarted)
        {
            if (GameObject.FindGameObjectWithTag("Player") != null)
            {
                // Når spilleren er tæt nok på, så skal skiltet vises.
                if (Vector2.Distance(transform.position,
player.position) < seeDistance && PauseMenu.GameIsPaused == false)
                {
                    if (PlayerCloseEnough == true)
                    {
                        SignUI.SetActive(true);
                        PlayerCloseEnough = false;
                    }
                }
            }
        }
    }
}
```

```
    }

    // Når spilleren ikke er tæt nok på, så skal skiltet
ikke vises.
    else
    {
        if (PlayerCloseEnough == false)
        {
            SignUI.SetActive(false);
            PlayerCloseEnough = true;
        }
    }
}
}
```

## bilag 20 - script CombatSign

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class CombatSign : MonoBehaviour
{
    public int seeDistance;
    private bool PlayerCloseEnough;
    private bool FoundPlayer;
    private bool GameIsStarted;
    public Transform player;
    public GameObject SignUI;

    // Start is called before the first frame update
    void Start()
    {
        PlayerCloseEnough = true;
        FoundPlayer = false;
    }

    // Update is called once per frame
    void Update()
```

```
{  
    // Find spilleren  
    if (FoundPlayer == false)  
    {  
        player =  
GameObject.FindGameObjectWithTag("Player").transform;  
        FoundPlayer = true;  
        GameIsStarted = true;  
    }  
  
    player = GameObject.FindGameObjectWithTag("Player").transform;  
  
    if (GameIsStarted)  
    {  
        if (GameObject.FindGameObjectWithTag("Player") != null)  
        {  
            // Når spilleren er tæt nok på, så skal skiltet vises.  
            if (Vector2.Distance(transform.position,  
player.position) < seeDistance && PauseMenu.GameIsPaused == false)  
            {  
                if (PlayerCloseEnough == true)  
                {  
                    SignUI.SetActive(true);  
                    PlayerCloseEnough = false;  
                }  
            }  
            // Når spilleren ikke er tæt nok på, så skal skiltet  
ikke vises.  
            else  
            {  
                if (PlayerCloseEnough == false)  
                {  
                    SignUI.SetActive(false);  
                    PlayerCloseEnough = true;  
                }  
            }  
        }  
    }  
}
```

## bilag 21 - script HealthSign

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class HealthSign : MonoBehaviour
{
    public int seeDistance;
    private bool PlayerCloseEnough;
    private bool FoundPlayer;
    private bool GameIsStarted;
    public Transform player;
    public GameObject SignUI;

    // Start is called before the first frame update
    void Start()
    {
        PlayerCloseEnough = true;
        FoundPlayer = false;
    }

    // Update is called once per frame
    void Update()
    {
        // Find spilleren
        if (FoundPlayer == false)
        {
            player =
GameObject.FindGameObjectWithTag("Player").transform;
            FoundPlayer = true;
            GameIsStarted = true;
        }

        player = GameObject.FindGameObjectWithTag("Player").transform;

        if (GameIsStarted)
        {
            if (GameObject.FindGameObjectWithTag("Player") != null)
            {
                // Når spilleren er tæt nok på, så skal skiltet vises.
            }
        }
    }
}
```

```
        if (Vector2.Distance(transform.position,
player.position) < seeDistance && PauseMenu.GameIsPaused == false)
        {
            if (PlayerCloseEnough == true)
            {
                SignUI.SetActive(true);
                PlayerCloseEnough = false;
            }
        }
        // Når spilleren ikke er tæt nok på, så skal skiltet
ikke vises.
        else
        {
            if (PlayerCloseEnough == false)
            {
                SignUI.SetActive(false);
                PlayerCloseEnough = true;
            }
        }
    }
}
```

## bilag 22 - script SpellManager

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

public class SpellManager : MonoBehaviour
{
    public static int SelectedSpell;

    public GameObject Spell1RawImage;
    public GameObject Spell2RawImage;
    public GameObject Spell3RawImage;
    public GameObject Spell4RawImage;

    public Texture ActiveSpellImage;
```



```
public Texture InactiveSpellImage;

private RawImage img;

// Start is called before the first frame update
void Start()
{
    // Starter ud med at have valgt et default spell
    SelectedSpell = 1;
    Spell1Active();
}

// Update is called once per frame
void Update()
{
    // Hvis man klikker på 1, så sættes spell 1 aktivt.
    if (Input.GetKey("1"))
    {
        SelectedSpell = 1;
        Spell1Active();
        Spell2Inactive();
        Spell3Inactive();
        Spell4Inactive();
    }

    // Hvis man klikker på 2, så sættes spell 2 aktivt.
    if (Input.GetKey("2"))
    {
        SelectedSpell = 2;

        Spell2Active();
        Spell1Inactive();
        Spell3Inactive();
        Spell4Inactive();
    }

    // Hvis man klikker på 3, så sættes spell 3 aktivt.
    else if (Input.GetKey("3"))
    {
        SelectedSpell = 3;

        Spell3Active();
        Spell1Inactive();
        Spell2Inactive();
    }
}
```

```
        Spell4Inactive();
    }
    // Hvis man klikker på 4, så sættes spell 4 aktivt.
    else if (Input.GetKey("4"))
    {
        SelectedSpell = 4;

        Spell4Active();
        Spell1Inactive();
        Spell2Inactive();
        Spell3Inactive();
    }
}

// Spell 1 sættes aktivt ved at ændre på firkanten rundt omkring
spell 1.
void Spell1Active()
{
    img = (RawImage) Spell1RawImage.GetComponent<RawImage>();
    img.texture = (Texture) ActiveSpellImage;
}
void Spell2Active()
{
    img = (RawImage) Spell2RawImage.GetComponent<RawImage>();
    img.texture = (Texture) ActiveSpellImage;
}
void Spell3Active()
{
    img = (RawImage) Spell3RawImage.GetComponent<RawImage>();
    img.texture = (Texture) ActiveSpellImage;
}
void Spell4Active()
{
    img = (RawImage) Spell4RawImage.GetComponent<RawImage>();
    img.texture = (Texture) ActiveSpellImage;
}

// Spell 1 sættes inaktivt ved at ændre på firkanten rundt omkring
spell 1.
void Spell1Inactive()
{
    img = (RawImage) Spell1RawImage.GetComponent<RawImage>();
    img.texture = (Texture) InactiveSpellImage;
```

```
}  
  
void Spell2Inactive()  
{  
    img = (RawImage) Spell2RawImage.GetComponent<RawImage>();  
    img.texture = (Texture) InactiveSpellImage;  
}  
  
void Spell3Inactive()  
{  
    img = (RawImage) Spell3RawImage.GetComponent<RawImage>();  
    img.texture = (Texture) InactiveSpellImage;  
}  
  
void Spell4Inactive()  
{  
    img = (RawImage) Spell4RawImage.GetComponent<RawImage>();  
    img.texture = (Texture) InactiveSpellImage;  
}  
}
```

## bilag 24 - script MenuController

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
using UnityEngine.UI;  
using TMPro;  
using UnityEngine.SceneManagement;  
  
public class MenuController : MonoBehaviour  
{  
    public TextMeshProUGUI usernamepreview;  
    public TMP_InputField UsernameInput;  
    public Image CurrentCharacter;  
    public Image CurrentMonster;  
  
    public static string CurrentCharacterValue;  
    public static string CurrentMonsterValue;  
  
    public static string UsernameIngame;  
  
    public Sprite player1;  
    public Sprite player2;
```

```
public Sprite player3;
public Sprite player4;
public Sprite player5;
public Sprite player6;

public Sprite monster1;
public Sprite monster2;
public Sprite monster3;
public Sprite monster4;

[SerializeField] private GameObject StartButton;

void Start()
{
    // Vælger en start character og monster
    ChangeCharacterToPlayer1();
    ChangeMonsterToPlayer1();
}

void Update()
{
    // Opdaterer brugernavnet i previewet.
    usernamepreview.text = UsernameInput.text;
}

public void ChangeUserNameInput()
{
    // Hvis spillerens navn er 5 eller længere, så kan man se
    startknappen.
    if(UsernameInput.text.Length >= 5)
    {
        StartButton.SetActive(true);
    }
    // Hvis spillerens navn er 4 eller mindre, så kan man ikke se
    startknappen.
    else
    {
        StartButton.SetActive(false);
    }
}

// Når der bliver klikket på start knappen så indlæser den næste
scene.
```

```
public void StartGame ()
{
    UsernameIngame = UsernameInput.text;
    SceneManager.LoadScene (sceneBuildIndex:1) ;
}

// Endre spillerens preview sprite til det valgte.
public void ChangeCharacterToPlayer1 ()
{
    CurrentCharacter.sprite = player1;
    CurrentCharacterValue = "player1";
}
public void ChangeCharacterToPlayer2 ()
{
    CurrentCharacter.sprite = player2;
    CurrentCharacterValue = "player2";
}
public void ChangeCharacterToPlayer3 ()
{
    CurrentCharacter.sprite = player3;
    CurrentCharacterValue = "player3";
}
public void ChangeCharacterToPlayer4 ()
{
    CurrentCharacter.sprite = player4;
    CurrentCharacterValue = "player4";
}
public void ChangeCharacterToPlayer5 ()
{
    CurrentCharacter.sprite = player5;
    CurrentCharacterValue = "player5";
}
public void ChangeCharacterToPlayer6 ()
{
    CurrentCharacter.sprite = player6;
    CurrentCharacterValue = "player6";
}

// Endre monsterets preview sprite til det valgte.
public void ChangeMonsterToPlayer1 ()
{
    CurrentMonster.sprite = monster1;
    CurrentMonsterValue = "monster1";
}
```

```
}  
  
public void ChangeMonsterToPlayer2()  
{  
    CurrentMonster.sprite = monster2;  
    CurrentMonsterValue = "monster2";  
}  
  
public void ChangeMonsterToPlayer3()  
{  
    CurrentMonster.sprite = monster3;  
    CurrentMonsterValue = "monster3";  
}  
  
public void ChangeMonsterToPlayer4()  
{  
    CurrentMonster.sprite = monster4;  
    CurrentMonsterValue = "monster4";  
}  
  
}
```

## bilag 25 - script GameManager

```
using System;  
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
  
public class GameManager : MonoBehaviour  
{  
    //Instantiate prefab  
    public GameObject PlayerPrefab;  
    public GameObject Monster1Prefab;  
    public GameObject Monster2Prefab;  
    public GameObject Monster3Prefab;  
    public GameObject Monster4Prefab;  
    public GameObject SpawnButton;  
    public GameObject MainCamera;  
  
    public bool GameIsStarted = false;  
  
    private GameObject player;  
    private GameObject MonsterIns;
```

```
public Sprite player1;
public Sprite player2;
public Sprite player3;
public Sprite player4;
public Sprite player5;
public Sprite player6;

public Sprite monster1;
public Sprite monster2;
public Sprite monster3;
public Sprite monster4;

public void Start()
{
    NPCQ1.q1 = false;
}

// Bliver kørt når der klikkes på Spawn knappen.
public void SpawnPlayer()
{
    // Laver en instans af spilleren.
    GameObject PlayerIns = Instantiate(PlayerPrefab, new Vector3(-
2f, 3.9f, 0), Quaternion.identity);

    // Hvis brugeren har valgt spiller 1 i menuen
    if (MenuController.CurrentCharacterValue == "player1")
    {
        // Så skal den sætte PlayerSelected Animator Parameters til
1.

PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 1);
    }
    else if (MenuController.CurrentCharacterValue == "player2")
    {

PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 2);
    }
    else if (MenuController.CurrentCharacterValue == "player3")
    {

PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 3);
    }
```



```
        else if (MenuController.CurrentCharacterValue == "player4")
        {

PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 4);
        }
        else if (MenuController.CurrentCharacterValue == "player5")
        {

PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 5);
        }
        else if (MenuController.CurrentCharacterValue == "player6")
        {

PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 6);
        }

        // Hvis brugeren har valgt monster 1 i menuen.
        if (MenuController.CurrentMonsterValue == "monster1")
        {
            // Laver en instans af monster 1, og gør den til et child
            af spilleren.
            GameObject MonsterIns = Instantiate(Monster1Prefab, new
Vector3(-1f, 3.8f, 0), Quaternion.identity);
            MonsterIns.GetComponent<SpriteRenderer>().sprite =
monster1;
            MonsterIns.transform.parent = PlayerIns.transform;
        }
        else if (MenuController.CurrentMonsterValue == "monster2")
        {
            GameObject MonsterIns = Instantiate(Monster2Prefab, new
Vector3(-1f, 3.8f, 0), Quaternion.identity);
            MonsterIns.GetComponent<SpriteRenderer>().sprite =
monster2;
            MonsterIns.transform.parent = PlayerIns.transform;
        }
        else if (MenuController.CurrentMonsterValue == "monster3")
        {
            GameObject MonsterIns = Instantiate(Monster3Prefab, new
Vector3(-1f, 3.8f, 0), Quaternion.identity);
            MonsterIns.GetComponent<SpriteRenderer>().sprite =
monster3;
            MonsterIns.transform.parent = PlayerIns.transform;
        }
    }
```

```
        else if (MenuController.CurrentMonsterValue == "monster4")
        {
            GameObject MonsterIns = Instantiate(Monster4Prefab, new
Vector3(-1f, 3.8f, 0), Quaternion.identity);
            MonsterIns.GetComponent<SpriteRenderer>().sprite =
monster4;
            MonsterIns.transform.parent = PlayerIns.transform;
        }

        // Spawn knappen sættes herefter til usynlig.
        SpawnButton.SetActive(false);
        GameIsStarted = true;
    }
}
```

## bilag 26 - script CheckForDeath

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class CheckForDeath : MonoBehaviour
{
    public GameObject player;
    public bool GameIsStarted;
    public GameObject GameOverMenuUI;
    public static bool GameOver;

    public void GameStart()
    {
        // Find spiller
        player = GameObject.FindGameObjectWithTag("Player");
        GameIsStarted = true;
        GameOver = false;
    }

    // Update is called once per frame
    void Update()
```

```
{  
    if (GameIsStarted)  
    {  
        // Hvis spilleren ikke er død, og ikke eksisterer i scenen,  
        så skal der vises GameOver på en GUI.  
        if (GameObject.FindGameObjectWithTag("Player") == null)  
        {  
            // Vis respawn menu  
            GameOver = true;  
            GameOverMenuUI.SetActive(true);  
        }  
    }  
}  
}
```

## bilag 27 - script CheckForDeathQ1

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
  
public class CheckForDeathQ1 : MonoBehaviour  
{  
    public GameObject player;  
    public GameObject GameOverMenuUI;  
    public bool playerFound;  
    public static bool GameOver;  
  
    public void Start()  
    {  
        playerFound = false;  
        GameOver = false;  
    }  
  
    // Update is called once per frame  
    void Update()  
    {  
        // Find spiller  
        if (playerFound == false)
```

```
{  
    player = GameObject.FindGameObjectWithTag("Player");  
    playerFound = true;  
}  
  
    // Hvis spilleren ikke er død, og ikke eksisterer i scenen, så  
    skal der vises GameOver på en GUI.  
    if (GameObject.FindGameObjectWithTag("Player") == null)  
    {  
        // Vis respawn menu  
        GameOver = true;  
        GameOverMenuUI.SetActive(true);  
    }  
}
```

## bilag 28 - script NPCQ1

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
using UnityEngine.SceneManagement;  
  
public class NPCQ1 : MonoBehaviour  
{  
    public static bool q1 = false;  
  
    public AudioClip Farmer1;  
    public AudioClip Farmer2;  
    public AudioClip Farmer3;  
    public AudioClip Farmer4;  
    AudioSource audioData1;  
    AudioSource audioData2;  
    AudioSource audioData3;  
    AudioSource audioData4;  
  
    private int seeDistance = 5;  
    public Transform player;  
    private bool GameIsStarted;  
    public GameObject Q1Wrapper;
```

```
public bool PlayerCloseEnough;

void Start()
{
    PlayerCloseEnough = true;

    audioData1 = AddAudio(false, false, 1f);
    audioData2 = AddAudio(false, false, 1f);
    audioData3 = AddAudio(false, false, 1f);
    audioData4 = AddAudio(false, false, 1f);
}

public void GameStart()
{
    // Finder spiller
    player = GameObject.FindGameObjectWithTag("Player").transform;
    GameIsStarted = true;
}

void Update()
{
    if (GameIsStarted)
    {
        if (GameObject.FindGameObjectWithTag("Player") != null)
        {
            // Hvis spilleren er tæt nok på så viser den quest gui.
            if (Vector2.Distance(transform.position,
player.position) < seeDistance)
            {
                if (PlayerCloseEnough == true)
                {
                    QlWrapper.SetActive(true);
                    //audioData = GetComponent<AudioSource>();
                    //audioData.Play(0);
                    int SoundRandomizer = Random.Range(1, 100);
                    if (SoundRandomizer <= 30)
                    {
                        PlayFarmer1();
                    }
                    else if (SoundRandomizer > 30 &&
SoundRandomizer <=60)
                    {
                        PlayFarmer2();
                    }
                }
            }
        }
    }
}
```

```
        }
        else if (SoundRandomizer > 60 &&
SoundRandomizer <= 90)
        {
            PlayFarmer3();
        }
        else if (SoundRandomizer > 90)
        {
            PlayFarmer4();
        }
        PlayerCloseEnough = false;
    }
}
// Hvis spilleren ikke er tæt nok på så viser den ikke
quest gui.
else
{
    if (PlayerCloseEnough == false)
    {
        QlWrapper.SetActive(false);
        PlayerCloseEnough = true;
    }
}
}
}

// Når man klikker på start quest
public void GotoQ1()
{
    // Indlæser quest 1 bane.
    SceneManager.LoadScene("Quest1");
}

// Funktion til at tilføje indstillinger til AudioSource.
public AudioSource AddAudio(bool loop, bool playAwake, float vol)
{
    AudioSource newAudio = gameObject.AddComponent<AudioSource>();
    newAudio.loop = loop;
    newAudio.playOnAwake = playAwake;
    newAudio.volume = vol;
    return newAudio;
}
```

```
}

// Spil farmer lyd.
public void PlayFarmer1()
{
    audioData1.clip = Farmer1;
    audioData1.Play();
}
public void PlayFarmer2()
{
    audioData2.clip = Farmer2;
    audioData2.Play();
}
public void PlayFarmer3()
{
    audioData3.clip = Farmer3;
    audioData3.Play();
}
public void PlayFarmer4()
{
    audioData4.clip = Farmer4;
    audioData4.Play();
}

// Sætter questen til at være aktiv.
public void SetQ1Active()
{
    q1 = true;
}
}
```

## bilag 29 - script Shooting

```
using System.Collections;
using System.Collections.Generic;
using System.Security.Cryptography;
using UnityEngine;
using TMPro;

public class Shooting : MonoBehaviour
```



```
{  
    public Transform firePoint;  
    public GameObject fireBulletPrefab;  
    public GameObject poisonsdartBulletPrefab;  
    public GameObject electricBulletPrefab;  
    public GameObject poisonBulletPrefab;  
  
    public float autobulletForce = 20f;  
    public float poisonbulletForce = 6f;  
  
    public float cooldownFire = 1f;  
    public float cooldownPoisonDart = 0.2f;  
    public float cooldownElectric = 15f;  
    public float cooldownPoison = 3f;  
    private float Spell1autoNextFireTime = 0f;  
    private float Spell2autoNextFireTime = 0f;  
    private float Spell3autoNextFireTime = 0f;  
    private float Spell4autoNextFireTime = 0f;  
  
    public TMP_Text Spell1Countdown;  
    public TMP_Text Spell2Countdown;  
    public TMP_Text Spell3Countdown;  
    public TMP_Text Spell4Countdown;  
  
    void Start()  
    {  
        Spell1Countdown.text = Spell1autoNextFireTime.ToString("F1");  
        Spell2Countdown.text = Spell2autoNextFireTime.ToString("F1");  
        Spell3Countdown.text = Spell3autoNextFireTime.ToString("F1");  
        Spell4Countdown.text = Spell4autoNextFireTime.ToString("F1");  
    }  
  
    void Update()  
    {  
        // Opdaterer cooldown på spells.  
        // Hvis den næste fire time er større end den nuværende tid, så  
        // skal den opdaterer cooldown time.  
        if (Spell1autoNextFireTime > Time.time)  
        {  
            Spell1Countdown.text = (Spell1autoNextFireTime -  
Time.time).ToString("F1");  
        }  
    }  
}
```

```
        if (Spell12autoNextFireTime > Time.time)
        {
            Spell12Countdown.text = (Spell12autoNextFireTime -
Time.time).ToString("F1");
        }
        if (Spell13autoNextFireTime > Time.time)
        {
            Spell13Countdown.text = (Spell13autoNextFireTime -
Time.time).ToString("F1");
        }
        if (Spell14autoNextFireTime > Time.time)
        {
            Spell14Countdown.text = (Spell14autoNextFireTime -
Time.time).ToString("F1");
        }

        // Tjekker hvilket spell som der er aktivt.
        // Hvis spell 1 er aktivt.
        if (SpellManager.SelectedSpell == 1)
        {
            // Hvis brugeren skyder afsted og der ikke er cooldown på
spell 1.
            if (Input.GetKey(KeyCode.Space) && Time.time >
Spell11autoNextFireTime)
            {
                // Skyd et skud og giv spell 1 cooldown.
                Shoot();
                Spell11autoNextFireTime = Time.time + cooldownFire;
            }
        }
        else if (SpellManager.SelectedSpell == 2)
        {
            if (Input.GetKey(KeyCode.Space) && Time.time >
Spell12autoNextFireTime)
            {
                Shoot();
                Spell12autoNextFireTime = Time.time +
cooldownPoisonDart;
            }
        }
        else if (SpellManager.SelectedSpell == 3)
```

```
{
    if (Input.GetKey(KeyCode.Space) && Time.time >
Spell3autoNextFireTime)
    {
        Shoot();
        Spell3autoNextFireTime = Time.time + cooldownElectric;
    }
}
else if (SpellManager.SelectedSpell == 4)
{
    if (Input.GetKey(KeyCode.Space) && Time.time >
Spell4autoNextFireTime)
    {
        Shoot();
        Spell4autoNextFireTime = Time.time + cooldownPoison;
    }
}

}

// Bliver kaldt, når der skal skydes et spell.
void Shoot()
{
    // Hvis det valgte spell er 1.
    if (SpellManager.SelectedSpell == 1)
    {
        // Lav en instans af skuddet og affyr det afsted med en
valgt kraft.
        GameObject bullet = Instantiate(fireBulletPrefab,
firePoint.position, firePoint.rotation);
        Rigidbody2D rb = bullet.GetComponent<Rigidbody2D>();
        rb.AddForce(firePoint.up * autobulletForce,
ForceMode2D.Impulse);
    }
    if (SpellManager.SelectedSpell == 2)
    {
        GameObject bullet = Instantiate(poisondartBulletPrefab,
firePoint.position, firePoint.rotation);
        Rigidbody2D rb = bullet.GetComponent<Rigidbody2D>();
        rb.AddForce(firePoint.up * autobulletForce,
ForceMode2D.Impulse);
    }
    if (SpellManager.SelectedSpell == 3)
```

```
{  
    GameObject bullet = Instantiate(electricBulletPrefab,  
firePoint.position, firePoint.rotation);  
    Rigidbody2D rb = bullet.GetComponent<Rigidbody2D>();  
    rb.AddForce(firePoint.up * autobulletForce,  
ForceMode2D.Impulse);  
}  
if (SpellManager.SelectedSpell == 4)  
{  
    GameObject bullet = Instantiate(poisonBulletPrefab,  
firePoint.position, firePoint.rotation);  
    Rigidbody2D rb = bullet.GetComponent<Rigidbody2D>();  
    rb.AddForce(firePoint.up * poisonbulletForce,  
ForceMode2D.Impulse);  
}  
}  
}
```

## bilag 30 - script PauseMenu

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
using UnityEngine.SceneManagement;  
  
public class PauseMenu : MonoBehaviour  
{  
  
    public static bool GameIsPaused = false;  
  
    public GameObject pauseMenuUI;  
    public GameObject playerprefab;  
  
    void Update()  
    {  
        // Tjekker om brugeren trykker escape.  
        if (Input.GetKeyDown(KeyCode.Escape))  
        {  
            // Hvis spillet allerede er pauset, så skal den fortsætte  
            spillet.  
        }  
    }  
}
```

```
        if (GameIsPaused)
        {
            Resume();
        }

        // Hvis spillet ikke er pauset, så skal den pause spillet.
        else {
            Pause();
        }
    }
}

// Sætter spillet igang igen
public void Resume()
{
    GameIsPaused = false;
    pauseMenuUI.SetActive(false);
    Time.timeScale = 1f;
}

// Pauser spillet
void Pause()
{
    GameIsPaused = true;
    pauseMenuUI.SetActive(true);
    Time.timeScale = 0f;
}

// Skifter til main menu scenen.
public void LoadMenu()
{
    Time.timeScale = 1f;
    SceneManager.LoadScene("MainMenu");
}

// Går helt ud af spillet
public void QuitGame()
{
    Application.Quit();
}
}
```

## bilag 31 - script Qest1GM

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class Quest1GM : MonoBehaviour
{
    //Instantiate prefab
    public GameObject PlayerPrefab;
    public GameObject Monster1Prefab;
    public GameObject Monster2Prefab;
    public GameObject Monster3Prefab;
    public GameObject Monster4Prefab;

    private GameObject player;
    private GameObject MonsterIns;

    public Sprite player1;
    public Sprite player2;
    public Sprite player3;
    public Sprite player4;
    public Sprite player5;
    public Sprite player6;

    // Næsten det samme script som det originale GameManager script.
    public void Start()
    {
        GameObject PlayerIns = Instantiate(PlayerPrefab, new Vector3(0,
0, 0), Quaternion.identity);

        if (MenuController.CurrentCharacterValue == "player1")
        {
            PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 1);
        }
        else if (MenuController.CurrentCharacterValue == "player2")
        {
            PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 2);
        }
        else if (MenuController.CurrentCharacterValue == "player3")
        {

```

```
PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 3);
    }
    else if (MenuController.CurrentCharacterValue == "player4")
    {

PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 4);
    }
    else if (MenuController.CurrentCharacterValue == "player5")
    {

PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 5);
    }
    else if (MenuController.CurrentCharacterValue == "player6")
    {

PlayerIns.GetComponent<Animator>().SetInteger("PlayerSelected", 6);
    }

    if (MenuController.CurrentMonsterValue == "monster1")
    {
        GameObject MonsterIns = Instantiate(Monster1Prefab, new
Vector3(1f, -0.1f, 0), Quaternion.identity);
        MonsterIns.transform.parent = PlayerIns.transform;
    }
    else if (MenuController.CurrentMonsterValue == "monster2")
    {
        GameObject MonsterIns = Instantiate(Monster2Prefab, new
Vector3(1f, -0.1f, 0), Quaternion.identity);
        MonsterIns.transform.parent = PlayerIns.transform;
    }
    else if (MenuController.CurrentMonsterValue == "monster3")
    {
        GameObject MonsterIns = Instantiate(Monster3Prefab, new
Vector3(1f, -0.1f, 0), Quaternion.identity);
        MonsterIns.transform.parent = PlayerIns.transform;
    }
    else if (MenuController.CurrentMonsterValue == "monster4")
    {
        GameObject MonsterIns = Instantiate(Monster4Prefab, new
Vector3(1f, -0.1f, 0), Quaternion.identity);
        MonsterIns.transform.parent = PlayerIns.transform;
    }
}
```



```
}  
}
```

## bilag 32 - script Q1Gem

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
using UnityEngine.SceneManagement;  
  
public class Q1Gem : MonoBehaviour  
{  
  
    public int seeDistance;  
    private bool PlayerCloseEnough;  
    private bool FoundPlayer;  
    private bool GameIsStarted;  
    public Transform player;  
    public GameObject GemUI;  
  
    // Start is called before the first frame update  
    void Start()  
    {  
        PlayerCloseEnough = true;  
        FoundPlayer = false;  
    }  
  
    // Update is called once per frame  
    void Update()  
    {  
        // Find spiller  
        if (FoundPlayer == false)  
        {  
            player =  
GameObject.FindGameObjectWithTag("Player").transform;  
            FoundPlayer = true;  
            GameIsStarted = true;  
        }  
  
        if (GameIsStarted)
```

```
{  
    if (GameObject.FindGameObjectWithTag("Player") != null)  
    {  
        // Hvis spilleren er tæt nok på, skal den vise et GUI  
        som fortæller spilleren er færdig med questen.  
        if (Vector2.Distance(transform.position,  
player.position) < seeDistance)  
        {  
            if (PlayerCloseEnough == true)  
            {  
                GemUI.SetActive(true);  
                PlayerCloseEnough = false;  
            }  
            // Hvis spilleren ikke er tæt nok på, så vises GUI'en  
ikke.  
            else  
            {  
                if (PlayerCloseEnough == false)  
                {  
                    GemUI.SetActive(false);  
                    PlayerCloseEnough = true;  
                }  
            }  
        }  
    }  
}  
  
    // Funktion som sender spilleren tilbage til startområdet, og gør  
at questen ikke længere er aktiv.  
    public void Quest1Done()  
    {  
        NPCQ1.q1 = false;  
        SceneManager.LoadScene("MainGame");  
    }  
}
```

## bilag 33 - script QuestLog

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class QuestLog : MonoBehaviour
{
    public GameObject Questlog;
    public static bool ShowingLog = false;

    // Update is called once per frame
    void Update()
    {
        // Hvis man trykker på L og spellbook ikke vises i forvejen.
        if (Input.GetKeyDown(KeyCode.L) && SpellBook.ShowingBook ==
false)
        {
            // Hvis man ikke kigger i questloggen.
            if (!ShowingLog)
            {
                // Aktiver book
                Questlog.SetActive(true);
                ShowingLog = true;
            }
            else
            {
                // Deaktiver book
                Questlog.SetActive(false);
                ShowingLog = false;
            }
        }
    }
}
```

## bilag 34 - script QLogCheck

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class QLogCheck : MonoBehaviour
```

```
{

    public GameObject noq;
    public GameObject q1;

    void Update()
    {
        // Hvis quest1 er aktiv
        if (NPCQ1.q1 == true)
        {
            // Så skal det vises i questloggen.
            noq.SetActive(false);
            q1.SetActive(true);
        }
    }
}
```

## bilag 35 - script SpellBook

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class SpellBook : MonoBehaviour
{

    public GameObject Spellbook;
    public static bool ShowingBook = false;

    // Update is called once per frame
    void Update()
    {
        // Hvis man trykker på P og questlog ikke vises i forvejen.
        if (Input.GetKeyDown(KeyCode.P) && QuestLog.ShowingLog ==
false)
        {
            // Hvis man ikke kigger i spellbook.
            if (!ShowingBook)
            {
                // Aktiver book
            }
        }
    }
}
```

```
        Spellbook.SetActive(true);  
        ShowingBook = true;  
    }  
    else  
    {  
        // Deaktiver book  
        Spellbook.SetActive(false);  
        ShowingBook = false;  
    }  
}  
}
```

## bilag 36 - script HealthBar

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
using UnityEngine.UI;  
  
public class HealthBar : MonoBehaviour  
{  
    public Slider slider;  
  
    public void SetMaxHealth(float health)  
    {  
        // Sætter barens maks liv  
        slider.maxValue = health;  
        slider.value = health;  
    }  
  
    public void SetHealth(float health)  
    {  
        // Justerer barens liv.  
        slider.value = health;  
    }  
}
```

## bilag 37 - script HealthPotion

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class HealthPotion : MonoBehaviour
{
    Player player;
    private bool playerfound;
    public static float HP;

    public HealthBar healthBar;

    void Start()
    {
        HP = 25f;
        playerfound = false;
    }

    void Update()
    {
        // Find spilleren.
        if (playerfound == false)
        {
            player = FindObjectOfType<Player>();
            playerfound = true;
        }
    }

    void OnTriggerEnter2D(Collider2D main)
    {
        // Hvis spilleren går ind i den.
        if (main.gameObject.tag == "Player")
        {
            // Hvis spillerens liv er mindre end det liv som den giver.
            if (player.currentHealth < player.maxHealth - HP)
            {
                player.currentHealth += HP;
            }

            // Hvis spillerens liv er mere eller lig det liv den giver.
            else if (player.currentHealth >= player.maxHealth - HP)
            {
                // (Empty block)
            }
        }
    }
}
```

```
{  
    player.currentHealth = player.maxHealth;  
}  
  
    // Herefter ødelæg det.  
    Destroy(gameObject);  
}  
}  
}
```

## bilag 38 - script CameraMovement

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
  
public class CameraMovement : MonoBehaviour  
{  
  
    public Transform Player;  
  
    void FixedUpdate()  
    {  
        // Sætter kameraet position til spillerens position.  
        transform.position = new  
Vector3(Player.position.x,Player.position.y,transform.position.z);  
    }  
}
```